

Digital Pendulum System

TEACHING MANUAL 2

33-005-5M5

MATLAB 5 Version



Feedback

Feedback Instruments Ltd, Park Road, Crowborough, E. Sussex, TN6 2QR, UK.

Telephone: +44 (0) 1892 653322, Fax: +44 (0) 1892 663719.

email: feedback@fdbk.demon.co.uk World Wide Web Homepage: <http://www.fbk.com>

Manual: 33-005-5M5 Ed01 990115

Printed in England by FI Ltd, Crowborough

Feedback Part No. 1160-330055M5

Notes



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

Preface

THE HEALTH AND SAFETY AT WORK ACT 1974

We are required under the Health and Safety at Work Act 1974, to make available to users of this equipment certain information regarding its safe use.

The equipment, when used in normal or prescribed applications within the parameters set for its mechanical and electrical performance, should not cause any danger or hazard to health or safety if normal engineering practices are observed and they are used in accordance with the instructions supplied.

If, in specific cases, circumstances exist in which a potential hazard may be brought about by careless or improper use, these will be pointed out and the necessary precautions emphasised.

While we provide the fullest possible user information relating to the proper use of this equipment, if there is any doubt whatsoever about any aspect, the user should contact the Product Safety Officer at Feedback Instruments Limited, Crowborough.

This equipment should not be used by inexperienced users unless they are under supervision.

We are required by European Directives to indicate on our equipment panels certain areas and warnings that require attention by the user. These have been indicated in the specified way by yellow labels with black printing, the meaning of any labels that may be fixed to the instrument are shown below:



CAUTION -
RISK OF
DANGER

Refer to accompanying documents



CAUTION -
RISK OF
ELECTRIC SHOCK



CAUTION -
ELECTROSTATIC
SENSITIVE DEVICE

PRODUCT IMPROVEMENTS

We maintain a policy of continuous product improvement by incorporating the latest developments and components into our equipment, even up to the time of dispatch.

All major changes are incorporated into up-dated editions of our manuals and this manual was believed to be correct at the time of printing. However, some product changes which do not affect the instructional capability of the equipment, may not be included until it is necessary to incorporate other significant changes.

COMPONENT REPLACEMENT

Where components are of a 'Safety Critical' nature, i.e. all components involved with the supply or carrying of voltages at supply potential or higher, these must be replaced with components of equal international safety approval in order to maintain full equipment safety.

In order to maintain compliance with international directives, all replacement components should be identical to those originally supplied.

Any component may be ordered direct from Feedback or its agents by quoting the following information:

- | | |
|------------------------|----------------------------|
| 1. Equipment type | 2. Component value |
| 3. Component reference | 4. Equipment serial number |

Components can often be replaced by alternatives available locally, however we cannot therefore guarantee continued performance either to published specification or compliance with international standards.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

Preface



DECLARATION CONCERNING ELECTROMAGNETIC COMPATIBILITY

Should this equipment be used outside the classroom, laboratory study area or similar such place for which it is designed and sold then Feedback Instruments Ltd hereby states that conformity with the protection requirements of the European Community Electromagnetic Compatibility Directive (89/336/EEC) may be invalidated and could lead to prosecution.

This equipment, when operated in accordance with the supplied documentation, does not cause electromagnetic disturbance outside its immediate electromagnetic environment.

COPYRIGHT NOTICE

© **Feedback Instruments Limited**

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted, in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without the prior permission of Feedback Instruments Limited.

ACKNOWLEDGEMENTS

Feedback Instruments Ltd acknowledge all trademarks.

IBM, IBM - PC are registered trademarks of International Business Machines.

MICROSOFT, WINDOWS 95, WINDOWS 3.1 are registered trademarks of Microsoft Corporation.

MATLAB is a registered trademark of Mathworks Inc.



TABLE OF CONTENTS

1 Introduction	1-1
2 Rule-Based Control	2-1
2.1 RULE-BASED CONTROL SIMULATION.....	2-4
2.1.1 Simulation: example 1	2-19
2.1.2 Simulation: example 2	2-21
2.1.3 Simulation: example 3	2-25
2.1.4 Simulation: example 4	2-30
2.2 RULE-BASED CONTROL PRACTICAL	2-35
2.2.1 Practical: example 1	2-37
2.2.2 Practical: example 2	2-41
2.2.3 Practical: example 3	2-46
2.2.4 Practical: example 4	2-50
3 Fuzzy Control	3-1
3.1 CRANE MODE.....	3-10
3.1.1 FUZZY SET	3-10
3.1.2 Simulation example	3-18
3.1.3 REAL – TIME EXAMPLE.....	3-21
3.2 STABILISING MODE	3-23
3.2.1 FUZZY SET	3-23
3.2.2 Simulation example 1	3-27
3.2.3 Simulation example 2	3-30
3.2.4 REAL-TIME: EXAMPLE 1.....	3-31
3.2.5 REAL-TIME EXAMPLE 2.....	3-35



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

Contents

3.3	SWING-UP MODE.....	3-38
3.3.1	FUZZY SET	3-38
3.3.2	Simulation example	3-45
3.3.3	REAL-TIME EXAMPLE.....	3-50
3.4	HOW TO BUILD AND RUN YOUR OWN FUZZY CONTROLLER.....	3-56
4	Installation Instructions	4-1
5	Literature	5-1



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 1

Introduction

1 Introduction

This manual is divided into two sections: rule-based and fuzzy controllers. We would like to review the control topics and problems connected with the pendulum-cart system and refer you to appropriate manuals in which you can find corresponding material. The *Reference Guide* manual describes how to get access to the control parameters embedded in the unique Real-Time Kernel.

Key words	TM1 Modelling, Identification, PID and LQ Controllers	TM2 Rule-based and Fuzzy Controllers	TM3 Neural Controllers
Control Problems	Crane, Upright stabilisation	Steering from point to point, Crane, Upright stabilisation	Steering from point to point, Crane, Upright stabilisation
Quality Criteria	Quadratic	Quadratic	Quadratic
Controllers	PID, LQ	Intelligent	Intelligent
Tools applied	Mathematical modelling	Fuzzy logic Rule-based algorithms Mathematical modelling	Neural networks, Mathematical modelling
Programming Environment	MS-Windows 95 MATLAB 5.1 Simulink 2.1 Control Signal Toolbox 4.01 Signal Processing 4.0.1 Nonlinear Control Design 1.1.1 Real-Time Kernel External Interface	MS-Windows 95 MATLAB 5.1 Simulink 2.1 Control Signal Toolbox 4.01 Signal Processing 4.0.1 Fuzzy-Logic Toolbox 1.0.3 Nonlinear Control Design 1.1.1 Real-Time Kernel External Interface(only for rule- based control applications)	MS-Windows 95 MATLAB 5.1 Simulink 2.1 Control Signal Toolbox 4.01 Signal Processing 4.0.1 Neural Network Toolbox 2.0.4 Nonlinear Control Design 1.1.1 Real-Time Kernel
Enhanced Programming Environment requires extra:	Real-Time Workshop 2.1 C Watcom compiler 11.0	Real-Time Workshop 2.1 C Watcom compiler 11.0	Real-Time Workshop 2.1 C Watcom compiler 11.0

Table 1 Content of the manuals



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 1

Introduction

The *Getting Started* manual guides you through the real-time control of pendulum-cart system, shown in the demo form. The *External Interface* manual shows how to build a controller of your own design in the C language. Table 1 describes the content of each Teaching Manuals (TM). Any controller comprises two essential components: a mechanism for knowledge representation and a mechanism for decision-making. With conventional controllers a mathematical model (such as a set of differential equations or transfer function) represents knowledge and decision-making based on optimisation.

When the system is complex or less precisely known it becomes difficult to characterise the knowledge adequately with conventional means. One must find alternative ways to encode the available knowledge about the system. It can result in the development of intelligent controllers. To this group, among others, belong: rule-based and fuzzy controllers.

Before exploring the examples you need to know if the system model is compatible with reality. The default parameters are stored in the model but it cannot be guaranteed that they are well identified. It is recommended to go through the identification procedure and experiments described in Teaching Manual 1 “Modelling, Identification, PID and LQ Controllers”.

The position measurement signals from the encoders play a crucial role in achieving the control goal. Measurement technique is another factor influencing the quality of control. The velocities in the system are not measured, they are observed. A poor observation algorithm may be a source of serious errors.

Remark. In many cases users would like not only to modify parameters of rule-based controllers embedded in the Real Time Kernel but also to develop their own algorithms of control. Therefore we shall try to clearly explain how to do it. The methodology used for LQ, PID or Fuzzy controllers (see the next section) can not be used directly for Rule-based controllers. For LQ, PID or Fuzzy techniques we are able to give a ready controller structure written in C. It does not matter if the controller algorithm is prepared as a fixed DLL or an external DLL file. The user can take advantage of the flexibility of the structure to create his own algorithms and is only limited to a number of input/output signals defined in a given LQ, PID or Fuzzy structure.

The rule-based toolbox structure offers also certain flexibility allowing to modify control algorithms (limited to parameter change). However, it might not be sufficient for more creative users.



There are two ways to build a rule-based controller significantly different from the toolbox rule-based controller:

- implement it in C using the External Interface method,
- implement it in MATLAB/Simulink using RTW (Real-time Workshop from MATLAB) and RT-CON Professional (from INTECO-Poland).

After these remarks we are ready to describe the model of the toolbox rule-based controller and then its correspondence to the real system. At the beginning of this manual you find the model of the cart-pendulum system. It contains several dedicated blocks added to obtain the best compatibility with the real laboratory system.

This model was presented in Teaching Manual 1 and it is invoked by writing: *model* in the MATLAB Command Window. In the Rule-based Controllers section a number of controller models are given.

These controller models are built to be compatible with the real system controllers. Sampling and quantization effects are also modelled to correspond to sampling introduced by the computer into the control algorithm and a finite accuracy of sensors (encoders) with discrete signals.

The goal of modelling is to achieve time responses of the model and the real system close to each other, as far as the shapes and amplitudes of signals are concerned. Only then can we trust our modelling of rule-based controllers. Usually, to build a controller one starts from simulation models. Then the controllers are implemented in the real system. Finally the simulation models are verified and modified to match reality as accurately as possible.

The fuzzy controllers in section 3 are presented in the simulation and real environments. Three controller demos are given. The *Crane Fuzzy Controller* is the first and simplest example. It contains only seven rules related to two input variables and one control output. It has been built to show the fuzzy technique rather than to develop the most accurate crane control policy. The second demo – *Stabilising Fuzzy Controller* emphasises a difference between linear (rule-based) and non-linear (fuzzy) stabilisation policies.

The last demo – *Swing-up Fuzzy Controller* encourages the user to design more complex controllers using the fuzzy technique.

The user is equipped with simulation tools to build his own fuzzy controller. Then he can try to operate the simulated fuzzy controller with the real system. One can access *pd_fuzzy.m* from the MATLAB Command Window, where the external interface file for fuzzy control is stored. Before starting the practical it is only necessary to load the generated fismatrix into the *fismat* fuzzy controller block.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 1

Introduction

Typographical conventions

The following conventions are used throughout this manual.

<i>Italics</i>	Variables, Simulink blocks, files, function and button names.
Courier New	Text typed in the MATLAB Command Window
Arial	Results shown in the MATLAB Command Window



2 Rule-Based Control

The material presented in this chapter consists of two parts 2.1 and 2.2. Part 2.1 corresponds to simulation of rule-based control. Part 2.2 refers to practicals.

The pendulum-cart system is an interesting laboratory model that can be controlled in many ways. Four state variables and only one control characterise this non-linear system. The first variable, cart position, is constrained by the rail length. The control, force pushing the cart back and forth on the rail, is also constrained.

One can ask if it is possible that a human being can serve as a controller for the system? The answer is positive if the control algorithm is not too complex. Even such a control task as pole balancing by hand in its unstable upright position can be satisfactorily done by a human being. Unfortunately we are not able to balance the pendulum by pushing back and forth the cart. We fail because we do not respond quickly enough to keep the pendulum in the unstable equilibrium point. However our experiments are not useless. We can gather knowledge about a possible control algorithm, we can also build a model of the controller and simulate its behaviour. We can study time and frequency responses of the system, generate additional characteristics such as potential or kinetics energy of the system etc. This information and our reasoning may result in a number of rules finally put together in the form of a control algorithm.

The **rule-based** control does not correspond to any of quality criteria such as time-optimal, linear quadratic etc. It is rather a policy that achieves the control goal and is robust and insensitive to parameter changes and disturbances. With the pendulum-cart system when the control goal is defined as “swing-up and upright stabilisation” we deal with a case when a trade off between two control actions is concerned. Namely, centring the cart and swinging-up the pendulum contradicts each other. To swing-up the pendulum we must put the cart in motion. Control rules applied to the cart centring may contradict raising-up the pendulum. Despite such effects the control goal has to be achieved finally. We cannot guess control rules they are obtained by a reasoning process. Therefore the rule-based approach is classified as an intelligent method. When conventional control algorithms fail intelligent methods seem to be more promising in solving complex control problems.

The rule-based controller is invoked by writing:

```
>>pd_rules
```

in the *MATLAB Command Window*

The following window opens - Figure 1

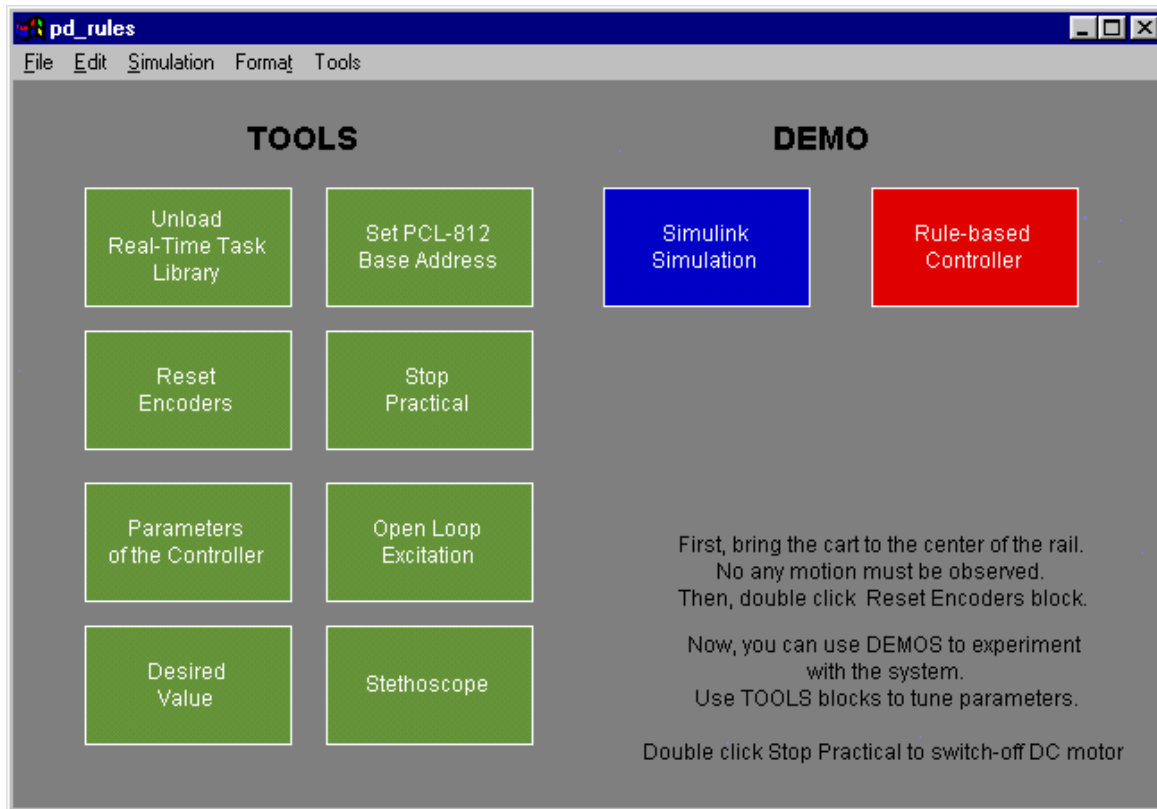


Figure 1: The *pd_rules* window

A number of tools are given to facilitate experiments in real time. The *Reset Encoders* block must be obligatory used to start an experiment. Other tools have been prepared to check or modify hardware and software configuration, to tune and set parameters and to simplify observations of the measured data. You can double click each of the Tools buttons to perform the action corresponding to the chosen button. These actions are described in Table 2.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

TOOLS	FUNCTION
Unload Real-Time Task Library	The Real-Time Kernel library is removed from the memory
Set PCL-812Base Address	You can check or modify the I/O board address in the memory. This should be done every time Matlab is started.
Reset Encoders	This action is performed at the beginning of an experiment. In this way the initial values of the pendulum angle and cart position are set. In the case when unexpected measurements have been collected it is recommended to repeat this action.
Stop Practical	The experiment is stopped.
Parameters of the Controller	The parameter window appears. One can modify values of parameters in two ways: by dragging sliders or typing new numerical values. Next, one of two modes of tuning can be chosen: "set" or "immediate". In the case of "set" one must acknowledge the modified values clicking the Set button. In the case of "immediate" when the Immediate button is marked the modifications are applied immediately.
Open Loop Excitation	The desired cart position value generator activates the open-loop system. A number of shapes of the desired value signal are available. The generator window is a source of typical families of signals i.e. square wave, saw-tooth etc. Note: The external generator flag in the Real-time Task block must be marked.
Desired Value	The desired cart position value is set by the use of slider. Note: The external generator flag in the Real-time Task block must be marked.
Stethoscope	It is recommended to check the state variables and generated control while the DC motor is switched off. This tool is dedicated to performing such checking.

Table 2 Tools for *pd_rules*



2.1 RULE-BASED CONTROL SIMULATION

Double click the *Simulink Simulation* button to open the rule-based control model - Figure 2.

There are generally three control policies modelled in Figure 2.

- swinging-up the pendulum and centring the cart to the middle of the rail
- stabilising the pendulum in its upright position and keeping the cart in the middle of the rail
- limiting the cart position

Therefore, the entire model consists of sub-models: in the form of the Simulink blocks given in Figure 2.

- the dynamics of the cart-pendulum system is represented by the *Dynamics* block
- the swinging and centring controller is modelled as the *Swinging & Centring Controller, Coulomb Friction & Swinging Control Gain*,
- the stabilising controllers are modelled as the *Linear pendulum controller, Linear cart controller, Sum & saturation* and *Coulomb Friction & Linear Control Gain* blocks,
- the limits of the cart position on the rail are modelled in the *Limit Guard* block
- the control switching decisions activating and not activating the controllers are represented by the following blocks: *Soft landing arbiter, Upper zone arbiter* and *Limitter*.

Red block – Upper Zone Arbiter – selects the control output between a swinging control process (see Blue Block - Swinging and Centring Controller which has an on-off characteristic) when the pendulum angle is in down position (i.e. $x_2 = \pi/2$) and a linear control process (see Pink Blocks - Linear Pendulum and Cart Controller which has a linear feedback characteristic) when the pendulum is in vicinity of up-right (i.e. $x_2 = 0$)

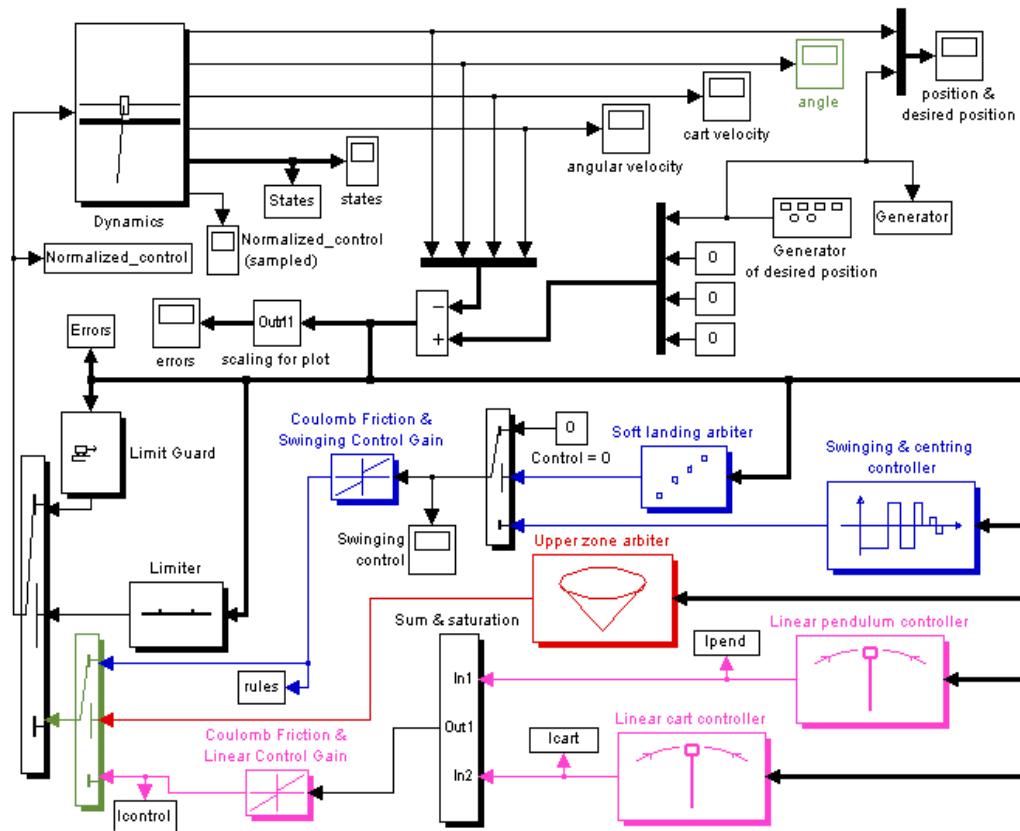


Figure 2: The rule-based controller model, *pd_mrules.mdl*

The interior of the *Dynamics* block is given in Figure 3. The pendulum-cart dynamics was described in *Teaching Manual 1*(330054m5.pdf). In this simulated model the *Dynamics* block is additionally equipped with the model of the computer sampling and the model of *Cart Position & Pendulum angle* quantization. These blocks are shown in Figure 3.

The interiors of the quantization blocks are given in Figure 4. Real pendulum-cart system has two state variables: the pendulum angle and cart position are measured. They are provided by the encoders mounted on the pendulum and motor shaft. Both encoders have identical resolution 2048 increments per one revolution. The angle of the pendulum is measured with the resolution equal to 0.003068 rad ($2\pi/2048$) and the cart position with the resolution of 0.076 mm (one revolution of the motor encoder) corresponds to 0.156 m of the cart travel. While the cart travels from one end of the rail to the other the encoder makes four full revolutions.



The computer sampling period has also a significant influence on the measured and control signals. The sampling period is fixed to 0.01 s. There are several reasons to choose this value. The sampling period 0.01 s means the control frequency 100 Hz. It is sufficient for the system with the natural frequency of about 1 Hz. Further shortening of the sampling period is not recommended. It must be a trade off between sampling and quantization. Quantization is defined by a construction of sensing devices: the encoders.

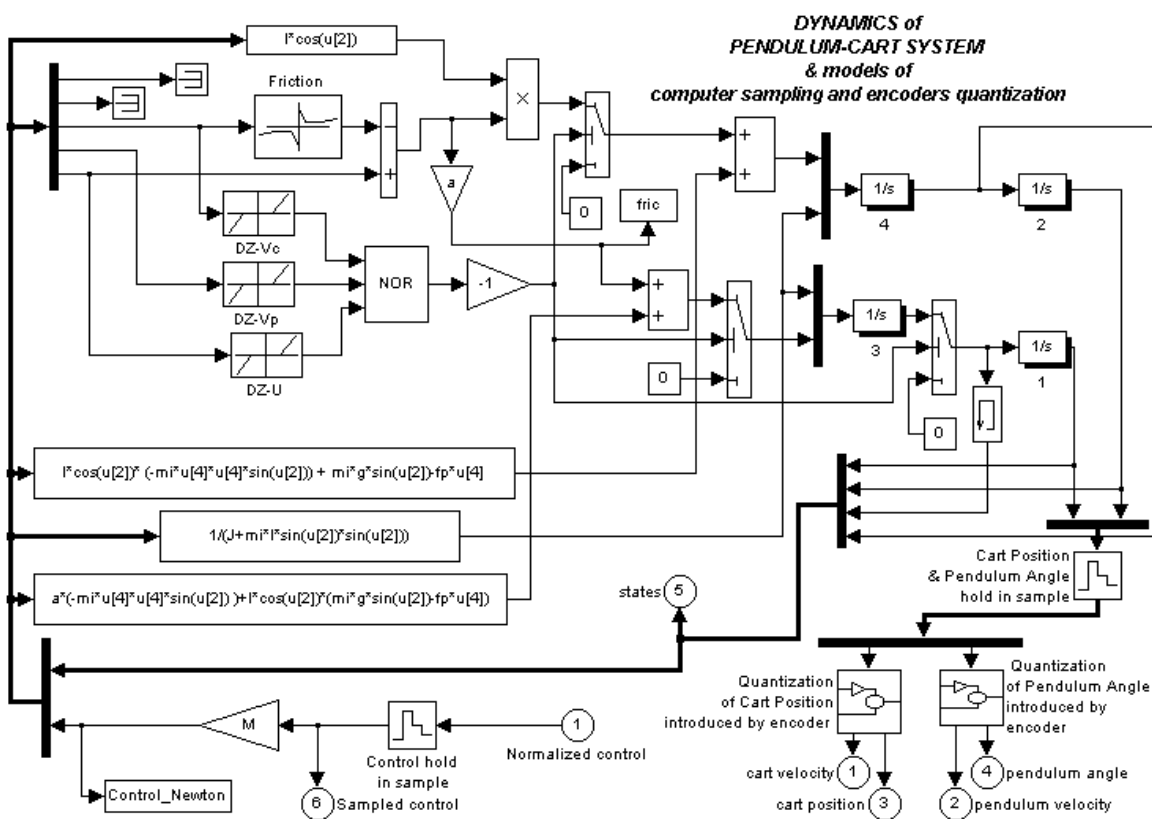
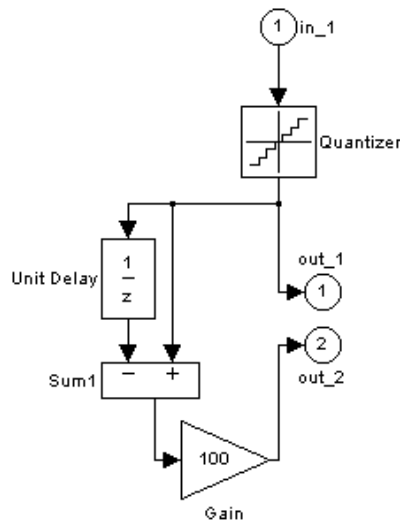


Figure 3: Interior of the Dynamics block

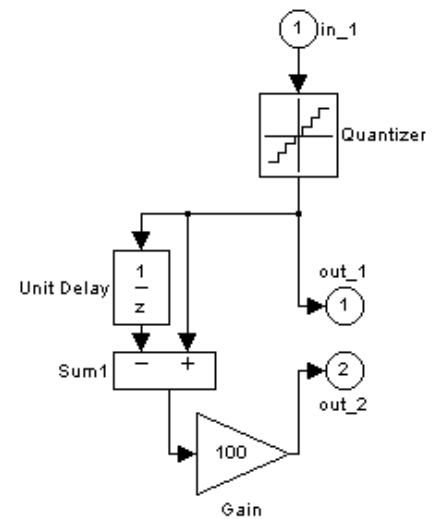
It is worth noting that too short a sampling period is a source of difficulties in co-operation between the Real-Time Kernel (RTK) under MS WINDOWS and the visualisation processes and also leaves too little time for complex control algorithms.



The continuous pendulum-cart dynamics process is modelled and simulated with the variable-step method (Dormand-Prince *ode45* method) – see Figure 5. The *sample hold* and *quantization* blocks are added to make modelling compatible with the real system conditions.



Quantization value - 7.6×10^{-5}
of Cart Position
introduced by encoder



Quantization value - 306.8×10^{-5}
of Pendulum Angle
introduced by encoder

Figure 4: Quantization blocks

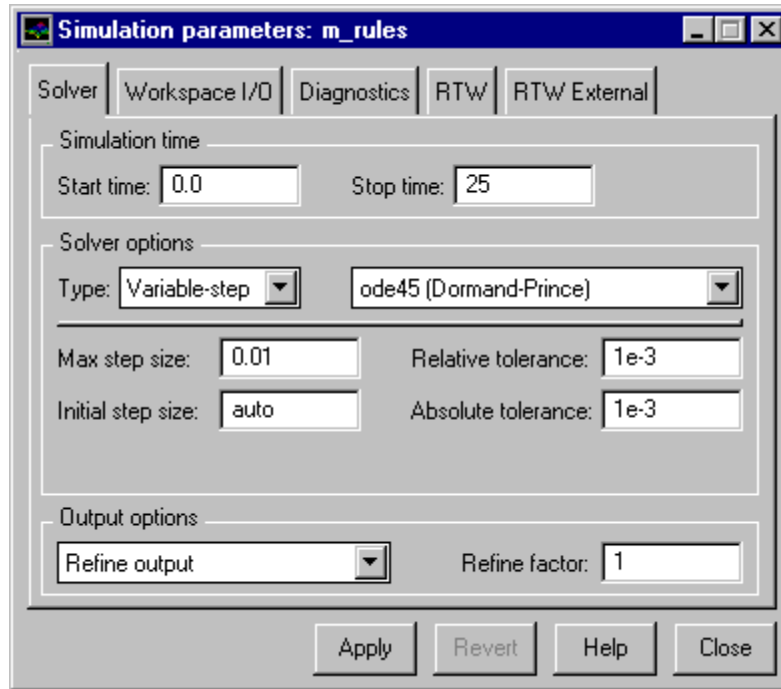


Figure 5: Simulation parameters of the rule-based controller

We recall here the mathematical model given in *Teaching Manual 1*. The state equations are:

$$\begin{aligned}
 \dot{x}_1 &= x_3, \\
 \dot{x}_3 &= \frac{a(F - T_c - \mu x_4^2 \sin x_2) + l \cos x_2 (\mu g \sin x_2 - f_p x_4)}{J + \mu l \sin^2 x_2} \\
 \dot{x}_2 &= x_4, \\
 \dot{x}_4 &= \frac{l \cos x_2 (F - T_c - \mu x_4^2 \sin x_2) + \mu g \sin x_2 - f_p x_4}{J + \mu l \sin^2 x_2}
 \end{aligned} \tag{2.1}$$



where

$$a = l^2 + \frac{J}{m_c + m_p}, \quad \mu = (m_c + m_p)l. \quad (2.2)$$

x_1, x_2, x_3 and x_4 denote: cart position, pendulum angle, cart velocity and pendulum angular velocity respectively

$(\dot{})$ denotes the first derivative with respect to time.

The control force F , parallel to the rail, applied to the cart. The mass of the cart is denoted by m_c and the mass of the pendulum, by m_p . l is the distance from the axis of rotation to the centre of mass of the pendulum-cart system. J is the moment of inertia of the pendulum-cart system with respect to the centre of mass.

T_c denotes the friction in the motion of the cart, and D_p is the moment of friction in the angular motion of the pendulum, proportional to the angular velocity: $D_p = f_p x_4$,

g denotes gravity.

The admissible controls F are bounded

$$|F(t)| \leq M. \quad (2.3)$$

The cart friction T_c in the model is a non-linear function of the cart velocity x_3 ,



The rule-based algorithms are as follows - Table 3

If $ x_2 - \text{Stab.zone} < 0$	no.1
then if $\text{abs}(K_1x_1 + K_2x_2 + K_3x_3 + K_4x_4) > 1$	no.2
then $u =: \text{sign}(K_1x_1 + K_2x_2 + K_3x_3 + K_4x_4)$	no.3
else $u =: K_1x_1 + K_2x_2 + K_3x_3 + K_4x_4$	no.4
else if $\{0.5 \cdot (x_4)^2 + 9.81 \cdot k \cdot R[\cos(x_2) - 1]\} > 0$	no.5
then $u =: 0$	no.6
else if $\text{sign}[x_4 \cdot (x_2 - \pi/2) \cdot x_1] < 0$	no.7
then $u =: \text{sign}[x_4 \cdot (x_2 - \pi/2)]$	no.8
else if $ \pi/2 - x_2 < \gamma$	no.9
then $u =: -\text{sign}(x_1)$	no 10
else $u =: \text{sign}[x_4 \cdot (x_2 - \pi/2)] [1 - \alpha \cdot x_1 - \beta \cdot (x_3)^2]$	no 11
if $u > 0$	no 12
then $u =: U_{\max} \cdot u + \text{Friction}$	no 13
else $u =: U_{\max} \cdot u - \text{Friction}$	no 14
if $ u > 1$	no 15
then $u =: \text{sign}(u)$	no 16
if $ x_1 - \text{Rail mg} < 0$	no 17
then $u =: \text{sign}(x_1)$	no 18

Table 3 Rule-based control algorithm

The detailed rule base control algorithm (see Table 3) has the **if...then...else** form. All the lines of the code are numbered from 1 to 18.



If the pendulum is in a vicinity of its upright position inside the stabilisation cone (no.1) then a linear controller is applied (no.4). $u = K_1x_1 + K_2x_2 + K_3x_3 + K_4x_4$ where K is the feedback gain matrix obtained by solving an appropriate LQR problem off-line. If the linear control approaches the boundaries of control (no.2) then it saturates (no. 3). The Simulink models of linear controllers for the system are given in Figure 6 and Figure 7. The *Sum & saturation* block comprise the linear control (Figure 8).

If the pendulum is outside the stabilisation zone (condition no.1 not satisfied in the *Upper zone arbiter* block) it must be swung up. The algorithm checks if swinging up the pendulum does not result in overriding the upper unstable equilibrium point.

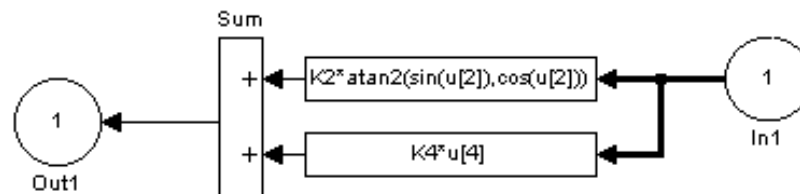


Figure 6: Linear pendulum controller

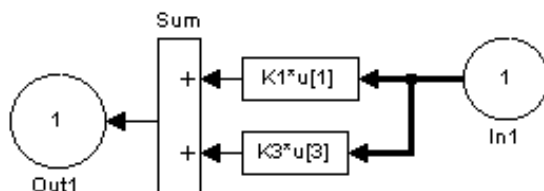


Figure 7: Linear cart controller

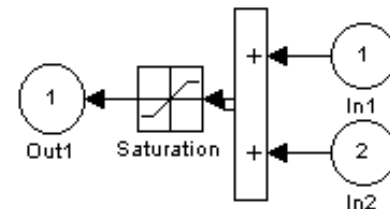


Figure 8: Sum & saturation

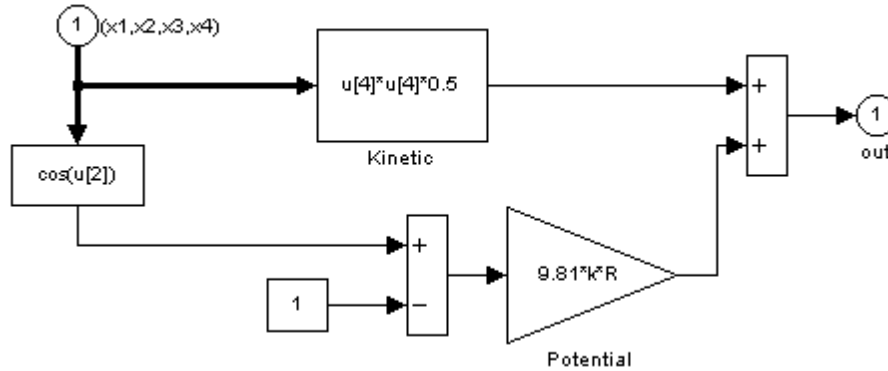


Figure 9: Soft landing arbiter

To achieve a soft landing in the vicinity of the upright position, the algorithm checks whether the kinetic energy of the pendulum minus energy loss due to friction forces is sufficient to rise up the centre of gravity of the pendulum to its upright position. Look at condition no.5 in the *Soft landing arbiter* block (Figure 9). If the condition is satisfied then the control is set to zero (no.6).

The swinging rule has a very simple form (no. 8). It is “bang-bang” with switching moments related to the sign of the angular velocity and angle position, which points up, or down. The cart centring action is based on a heuristic rule. The magnitude of the control force is reduced only if the actual force would make the cart to run away too far from the centre of the rail (no.7).

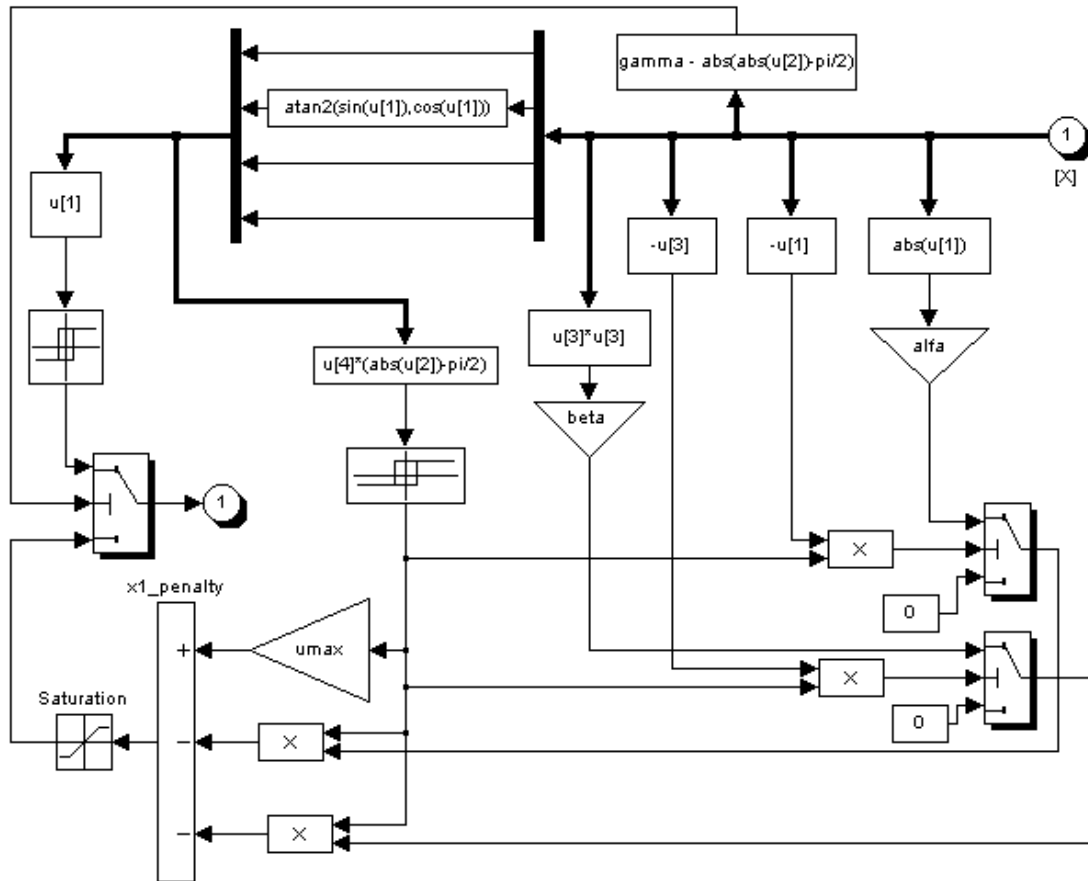


Figure 10: Swinging & centring controller

The magnitude is reduced proportionally to the cart position x_1 with the coefficient α and to the quadrate of the cart velocity x_3 with the coefficient β (no.11).

How to obtain the best pair (α, β) is an open question. There is a trade-off between the centring action and swinging up the pendulum. Observations of the system behaviour encourage us to introduce a third centring possibility. Pushing the cart back and forth has little effect on the pendulum when the pendulum is in the horizontal position (x_2 equals to $\pi/2$). We can assume a small angle γ in the neighbourhood of $\pm\pi/2$ and apply the centring action if necessary (no.9 and no.10). We can decide also not to apply any of the centring rules setting the parameters: α, β, γ equal to zero. In this case the algorithm swings up the pendulum, increasing its total energy, with no regard to the cart position (no.8). Then the control is pure “bang-bang”. The *Swinging & centring controller* model is shown in Figure 10.



There are also a number of effects difficult for modelling. For example, the effects related to Coulomb friction of the cart. We can compensate these effects by control. If control is positive (no.12) we can make it “more positive” by adding an amount of control to overcome the Coulomb friction (no.14). Similarly, we make a negative control “more negative” (no.13). The *Coulomb Friction & Swinging Control Gain* block perform this action (Figure 11). Then the control is normalised to stay in the range [-1, +1] (no.15 and no.16). Usually the normalisation performed by the *Saturation* block is not active because the maximum control value is reduced by the U_{max} parameter. However usually we do not use the maximum force M generated by the DC motor. Thus in our disposal we have the parameter U_{max} to reduce this force proportionally, ($0 \leq U_{max} \leq 1$). This control signal is sampled and multiplied by M inside the *Dynamics* block. The real maximum control magnitude M is equal to 16.5 N.

In all examples and experiments we assume $U_{max}=0.25$ so we use only 25% of the DC motor power, in fact 35% as the normalised u value is enlarged to 0.35 ($0.25+0.1$) due to the Coulomb friction effect, modelled by the *Friction* parameter ($Friction=0.1$).

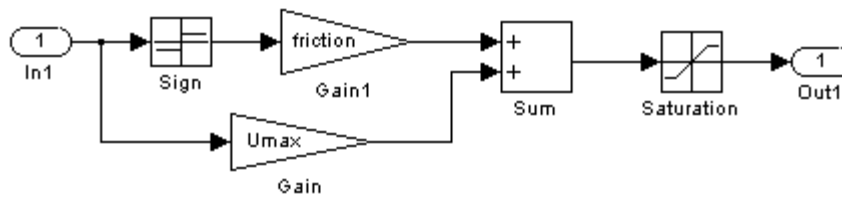


Figure 11: *Coulomb Friction & Swinging Control Gain* controller

The similar block to the one given in Figure 11, but with different parameters is applied at the output of the linear controller. Its name is *Coulomb Friction & Linear Control Gain*. In the linear case we assume: $U_{max}=1$ and $Friction=0.1$. It means that we use the maximum force ($U_{max}=1$) which can be generated by the DC motor to stabilise the system when the upright stabilisation zone is achieved. The *Friction* value is also a significant parameter for the control goal. We have oscillations of amplitude ± 0.1 m of the cart position around the desired cart position value for $Friction=0.1$ and we observe damped oscillations for $Friction>0.1$, for example $Friction=0.15$. These observations are valid for both the simulated and real-time responses of the system.

Finally, the *Limit Guard* block (Figure 12) is added to reinforce centring of the cart and prevent it from overrunning the ends of the rail. The rule is very simple. If the range given by the *Rail mg* "rail limit" parameter in the *limiter* block is exceeded (no.17), then the opposite to the direction of motion maximum steering force (i.e. $u_{max2}=1$) is applied to the cart (no.18).

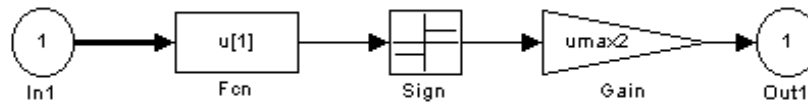


Figure 12: Limit Guard

Examples

We present four simulation examples of the rule-based controller actions. These experiments are then repeated in Part B for the real-time control of the laboratory model.

Any **simulation experiment** can be performed using [pd_mrules.mdl](#) Simulink file or selecting this , by double clicking on the *Simulink Simulation* blue button in [pd_rules](#) main menu.

Real-time experiments can be performed using [pd_rules_controller.mdl](#) Simulink file or selecting this , by double clicking on the *Rule-based controller* red button in [pd_rules](#) main menu.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

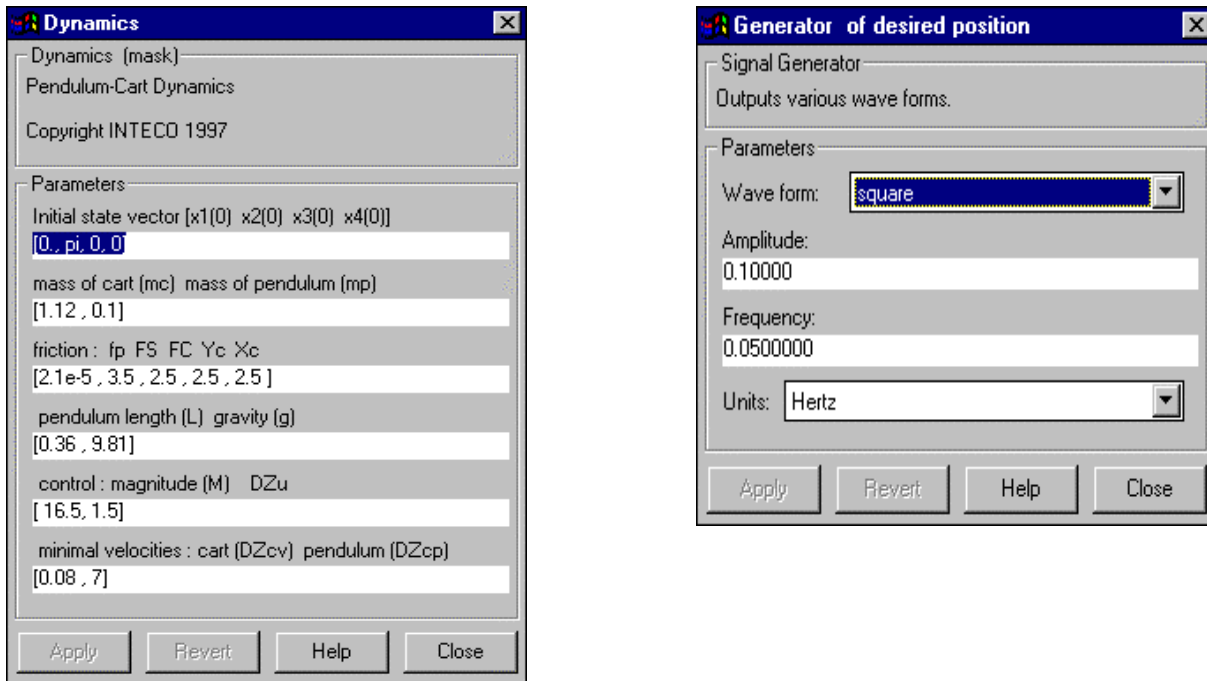


Figure 13: The *Dynamics* and *Generator of desired position* blocks

The controller blocks are masked. Double clicking on each of the controller blocks results in opening the mask and gives a simple window for parameter setting and tuning. The parameter names are standardised – the same notations are used in the control algorithm equations in the simulation model and in the monitor model of the real-time experiments. The *Dynamics* and *Generator of desired position* blocks are shown in Figure 13. The controller masks are presented in Figure 14 and Figure 15..



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

Limiter

Fcn (mask)

Parameters

Rail mg - rail limit

0.35

Apply Revert Help Close

Soft landing arbiter

Soft landing (mask)

Soft landing

Parameters

k coefficient

8.0

R value

0.4

Apply Revert Help Close

Linear pendulum controller

linear (mask)

Parameters

K2 - angle feedback

8

K4 - angular velocity feedback

0.6

Apply Revert Help Close

Coulomb Friction & Linear Control Gain

Coulombic and Viscous Friction (mask)

A discontinuity offset at zero models coulomb friction.
Linear gain models viscous friction.
 $y = \text{sign}(x) * (U_{\text{max}} * \text{abs}(x) + \text{friction})$

Parameters

Friction

0.1

Umax

1

Apply Revert Help Close

Linear cart controller

linear (mask)

Parameters

K1 - position feedback

-0.8

K3 - velocity feedback

-1.4

Apply Revert Help Close

Figure 14: Controller masks



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

Coulomb Friction & Swinging Control Gain

Coulombic and Viscous Friction (mask)
A discontinuity offset at zero models coulomb friction.
Linear gain models viscous friction.
 $y = \text{sign}(x) * (U_{\text{max}} * \text{abs}(x) + \text{friction})$

Parameters

Friction
0.1

Umax
0.25

Apply Revert Help Close

Sum & saturation

Subsystem (mask)

Parameters

max control
1

Apply Revert Help Close

Upper zone arbiter

ZONE (mask)
ZONE

Parameters

Stab.zone - width of stabilization zone [rad]
0.3

Apply Revert Help Close

Swinging & centring controller

swing (mask)
swing

Parameters

maximal control umax
1

ALPHA - centering cart factor related to position
0.0

BETA - centering cart factor related to velocity
0.0

GAMMA - centering cart factor related to $+$ - $\pi/2$ angle
0.0

Apply Revert Help Close

Figure 15: Controller masks (cont.)



2.1.1 SIMULATION: EXAMPLE 1

Swinging up and stabilisation in the upright position with centring set to zero ($\alpha, \beta, \gamma=0$) and the desired cart position signal equal to zero

$X1=0$ to $x1=0$
 $X2=\pi/2$ $x1=0.$

We start with the motionless system, the pendulum is hanging down, the cart is in the middle of the rail. The goal of control is to steer the pendulum to the upright unstable equilibrium point and the cart to the middle of the rail (see Figure 16). To visualise the control output and pendulum angle open The Swinging Control scope and Angle scope respectively from the Matlab *pd_mrules.mdl* file.

Set the desired cart position in the Generator block to a value of zero, as by default this is set to a square wave of amplitude 0.1 and frequency equal to 0.05Hz.

The cart-centring algorithm is switched off except for centring which is active in the stabilisation zone. The *Swinging up* controller generates “bang-bang” control of 6 N. We also observe that pulses of 16.5 N signal produced by the *Limit Guard* at 0.7 s due to the cart crossing 0.35 m position on the rail (see Figure 17). Due to the *Soft Landing Arbiter* action there are periods of time in which the control is set to zero. In the stabilisation interval (4.2 to 6 s) the average value of control is positive. The explanation is the following.

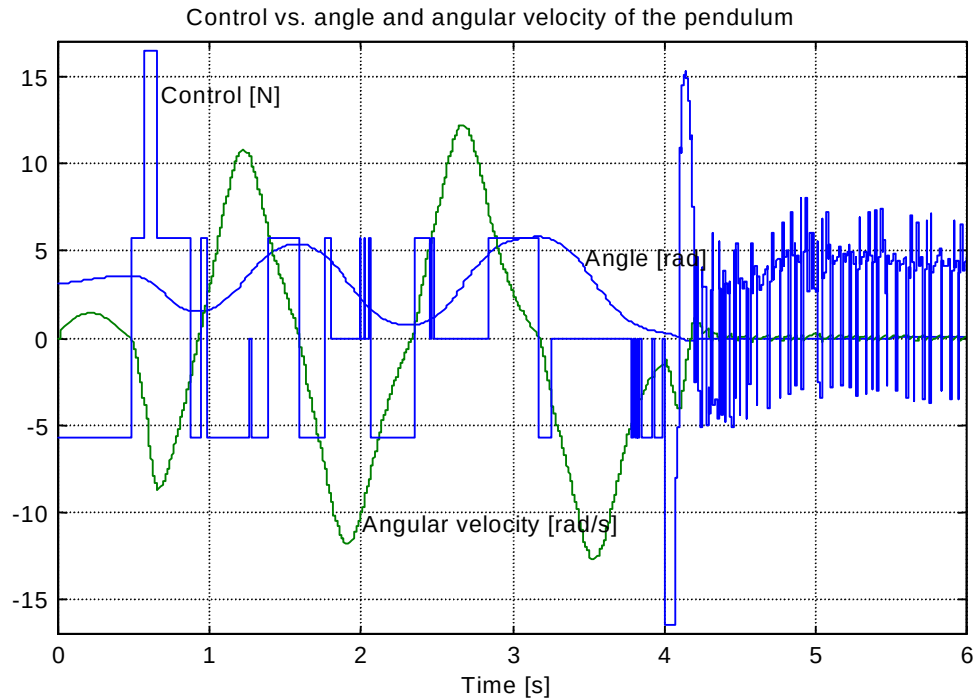


Figure 16: States related to the pendulum and control

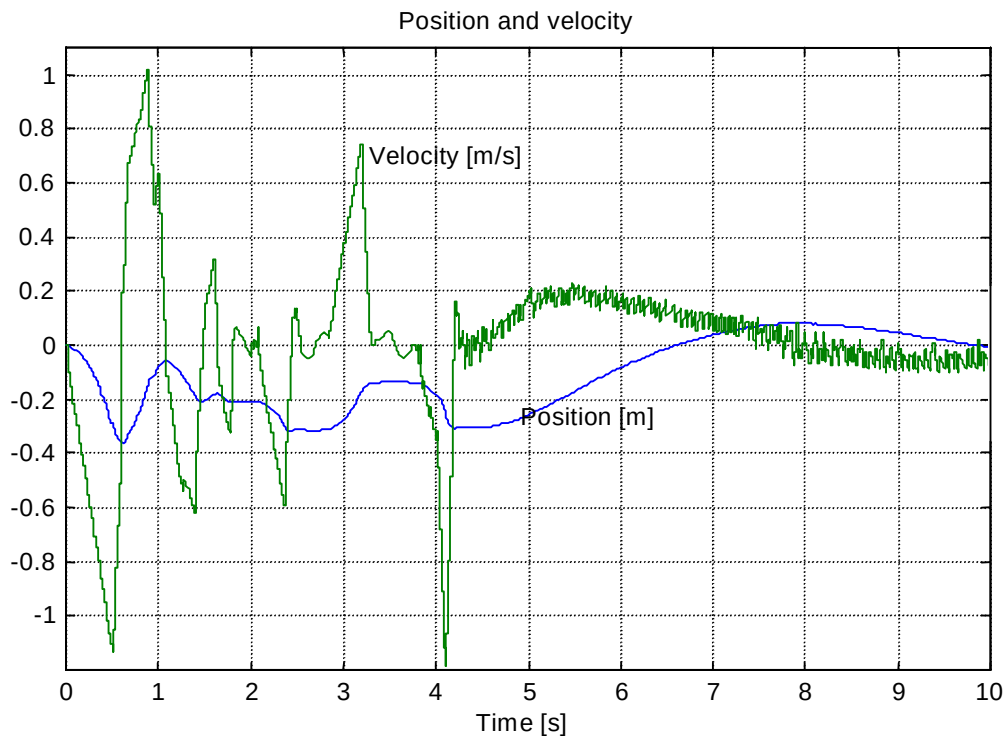


Figure 17: States related to the cart



Both linear controllers are active. The pendulum has achieved the vicinity of the upright position. The cart position is still negative and the cart must be steered to the middle of the rail. Therefore the positive average value of control is generated by the *Linear cart controller* block.

2.1.2 SIMULATION: EXAMPLE 2

Swinging up and stabilisation in the upright position with centring set to zero ($\alpha, \beta, \gamma=0$) and the desired cart position signal in a square-wave form

X1=... to x1=square wave

X2= $\pi/2$ x2=0.

In this example the situation is almost identical as in the first example. The only difference is that the desired cart position signal is non-zero. It is defined as a square-wave of the frequency of 0.05 Hz and the amplitude of 0.1 m. The time responses of the simulated plant & controller are given in Figure 18 and Figure 19.

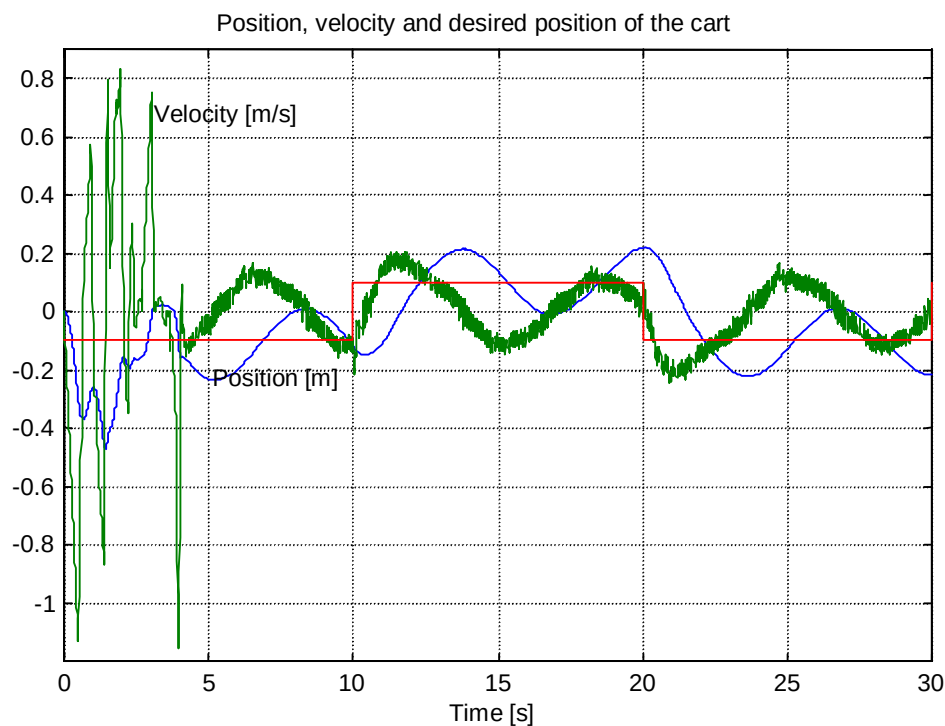


Figure 18: States related to the cart and desired cart position

(Friction=0.1 in the stabilisation zone)

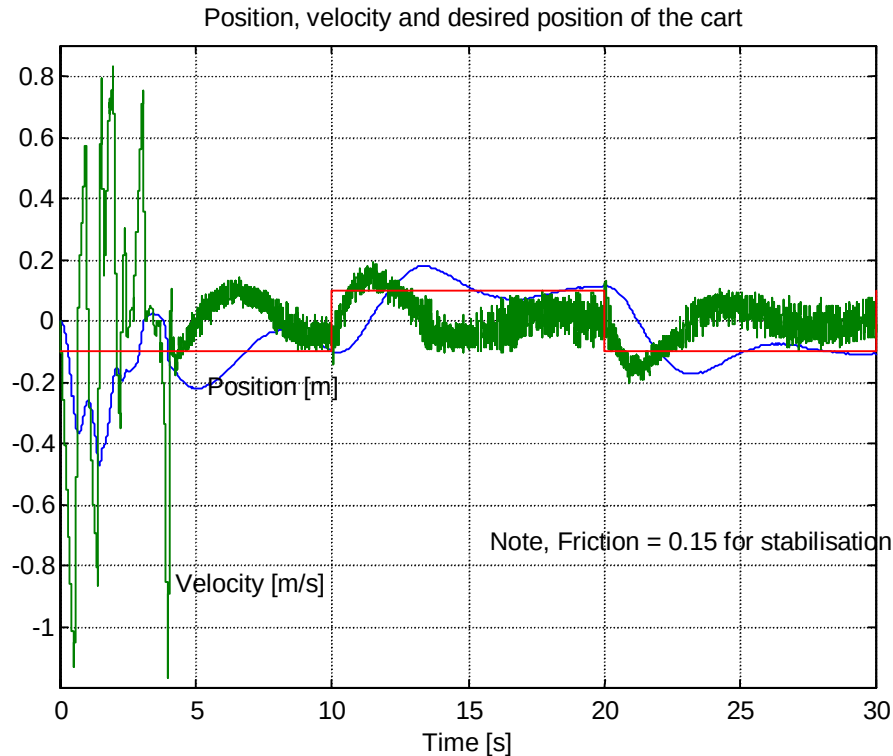


Figure 19: States related to the cart and desired cart position

(Friction=0.15 in the stabilisation zone)

We can notice that the cart is oscillating around the desired value signal. The tracking error is large. To explain this effect we review the parameter settings. Let us take attention to the mask of the *Dynamics* block (Figure 13). The following parameter values are set: *magnitude* (*M*)=16.5 N and *DZu* = 1.5 N, in the mask of the *Coulomb Friction & Linear Control Gain* (see Figure 14) *Friction*=0.1 and *Umax*=1. It means that the control is increased or decreased by 10% which correspond to the 0.1 Coulomb friction value. The Linear Control Gain *Umax* is equal 1. Therefore the magnitude *M* of control is not reduced.

A simple calculation gives: $16.5 \cdot 10\% = 1.65$ At the same time the *DZu* parameter (dead zone of control) is equal to 1.5. The modification of the control value equal to 1.65 N modelled in the *Coulomb Friction & Linear Control Gain* block is not sensitive enough for the *Dynamics* model (dead zone is set to 1.5 N). If we increase the *Friction* parameter to 0.15 then a more accurate tracking of the desire value signal is observed, see Figure 19.

Figure 20 and Figure 21 shows the time taken to bring the pendulum arms to the up position and the control signal changes between on/off(plus or minus 1), zero or linear behaviour.



Various steps of the control algorithm are easy to detect in Figure 21. At the beginning we have a “bang-bang” swing-up policy. After 1.4 s from the start an abrupt control change occurs. This action is produced by *Limit Guard*. The cart is too far from the middle.

In the interval lasting from 3.1 to 3.6 s the control is switched off. This time it is a *Soft landing arbiter* action. The kinetic energy of the pendulum is sufficient to raise it up without the help of the control. Finally the system is stabilised. The control effort is devoted to tracking the desired cart position signal.

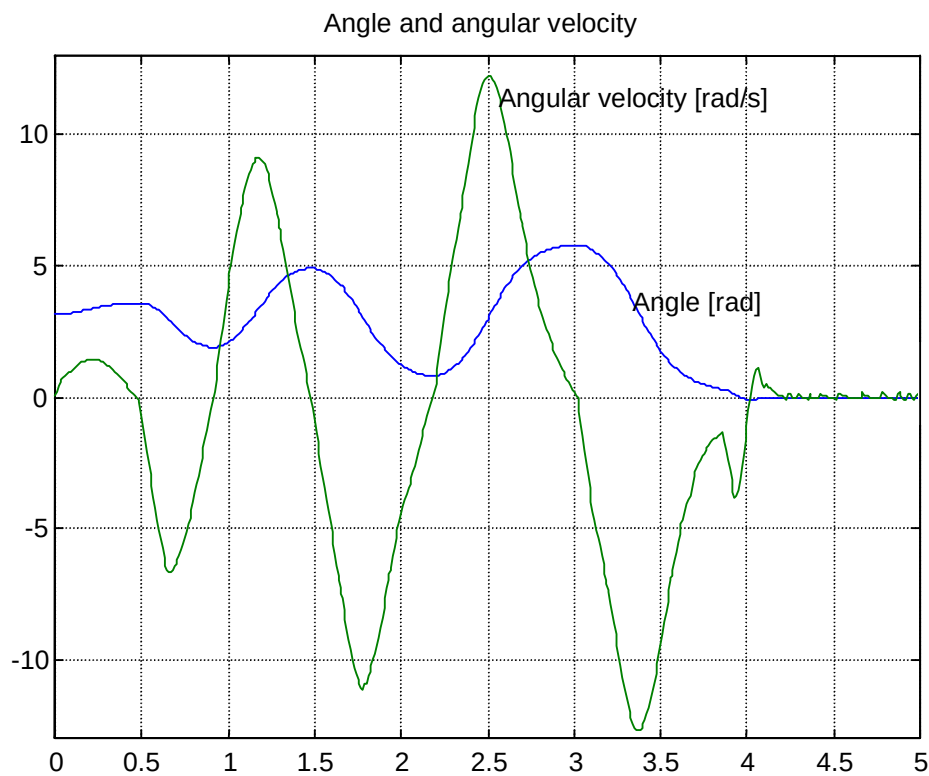


Figure 20: States related to the pendulum

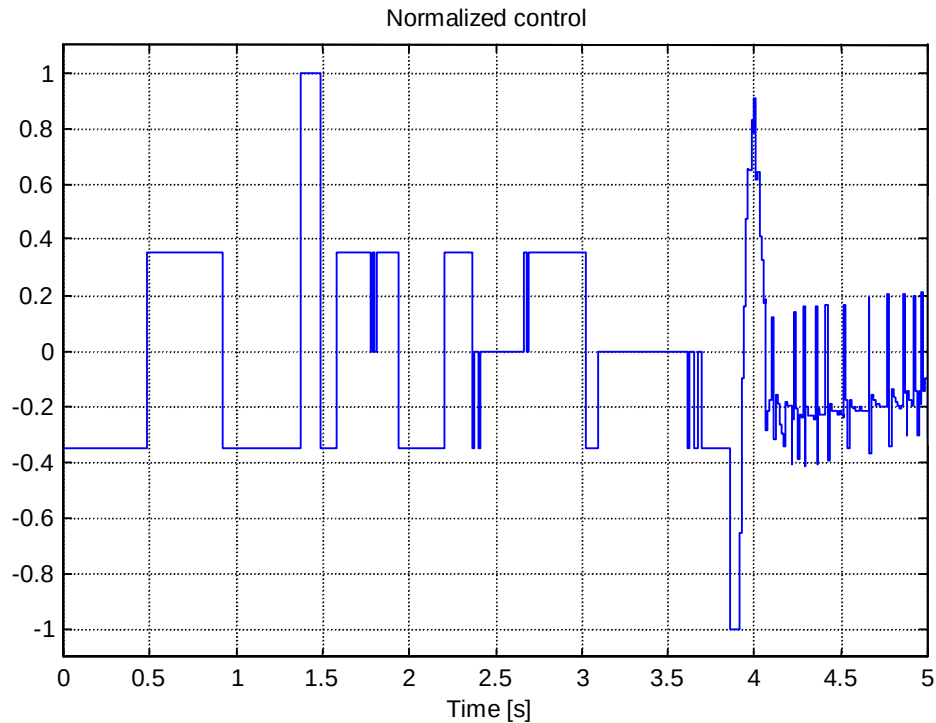


Figure 21: Normalised control

An accurate modelling means that the simulated model approximates the real system as closely as possible. If we visualise the state variables and control at the stabilisation mode on a short time interval then all inaccuracies caused by the encoder finite resolution, sample period introduced by the computer etc. are well visible in Figure 22.

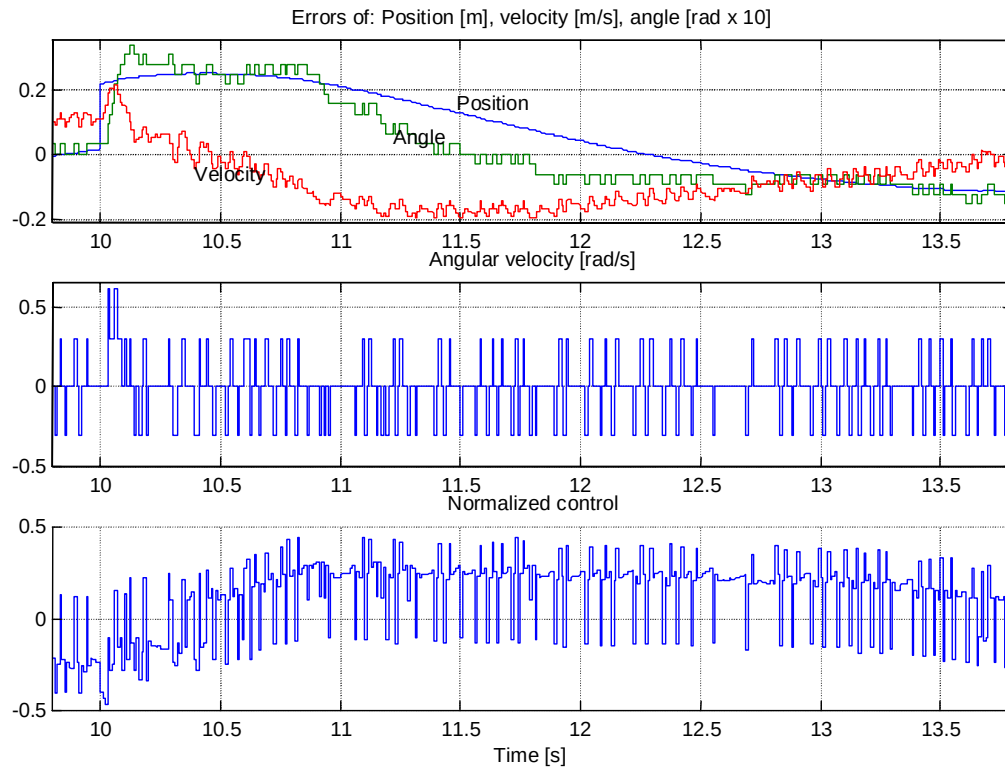


Figure 22: Errors and control in the stabilisation zone

Notice, that the angular velocity signal takes on only three values:

- 0.3, 0, + 0.3 rad/s,

except for the small time interval when the desired value is switched (we observe the + 0.6 rad/s value twice).

2.1.3 SIMULATION: EXAMPLE 3

Swinging up and stabilisation in the upright position with centring parameters:

$\alpha = 0.7$, $\beta = 0.5$, $\gamma = 0$ and the desired cart position signal set to zero.

$x_1=0$ to $x_1=\text{square wave}$

$x_2=\pi/2$ $x_2=0$.



In this experiment the cart centring rules are added. Note that we have not introduced a cart centring mechanism till now. In fact the cart centring takes place only in the stabilisation zone. The centring mechanism is purely heuristic. The magnitude of the control force is reduced only if the actual force would result in driving the cart too far from the centre of the rail.

The magnitude is reduced proportionally to the cart position x_1 with a coefficient α and to the quadrate of the cart velocity x_3 with a coefficient β . The values of α and β must be chosen experimentally.

Set the following values for $\alpha = 0.7$, $\beta = 0.5$, $\gamma = 0$ in “Swinging and Centring Controller” block.

The results of the experiment are given in Figure 23. We notice that the control time required to achieve the stabilisation zone is prolonged but the cart does not move too far away from the middle of the rail. Therefore the *Limit Guard* block is not activated. The control is depicted in Figure 24. The states that relate to the cart from Figure 23 are depicted pair-wise in a zoomed form in Figure 25. Similarly the states which relate to the pendulum in Figure 23 are depicted pair-wise in a zoomed form in Figure 26.

The absolute values of *alpha beta* centring controls are presented in Figure 27. There are time intervals in which both controls are active. There are intervals in which they act separately and there are also intervals when no centring action is required. In that case the swinging control contributes to the cart centring.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

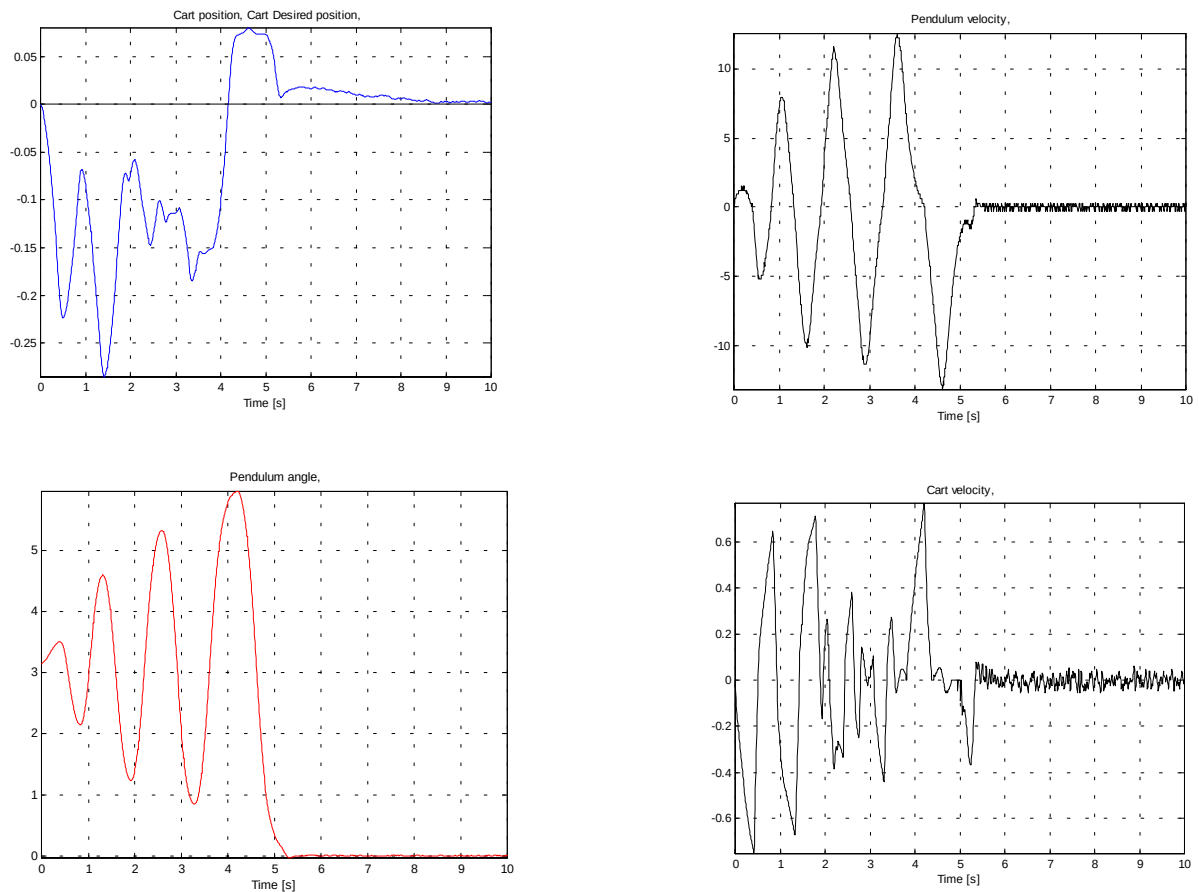


Figure 23: Time diagrams of states

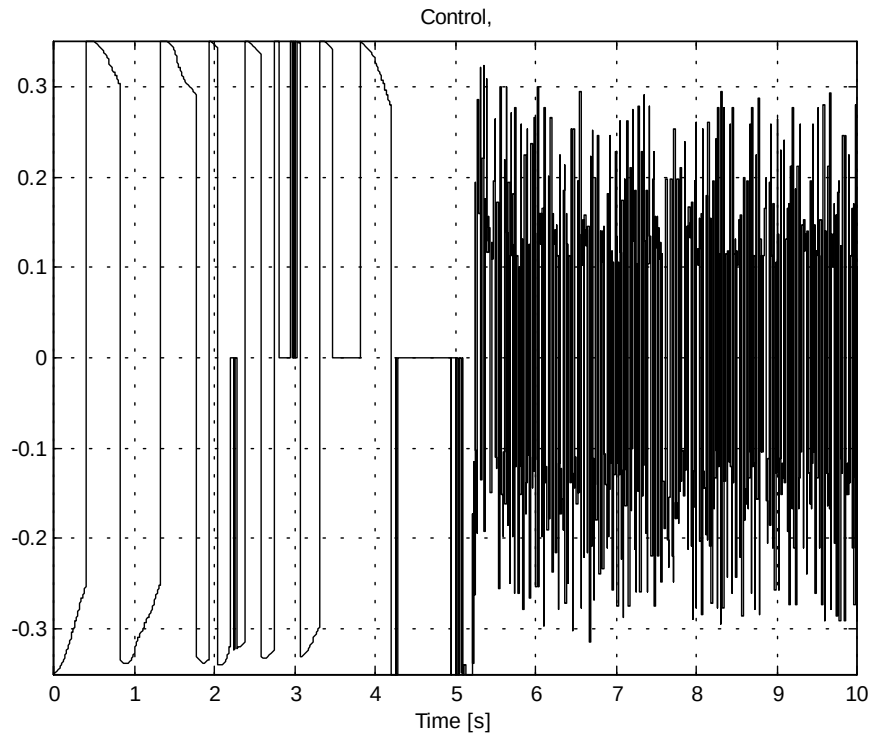


Figure 24: Time diagram of control

The shape of the resultant control in the swing-up mode is modified. The “bang-bang” character is not preserved (Figure 24).

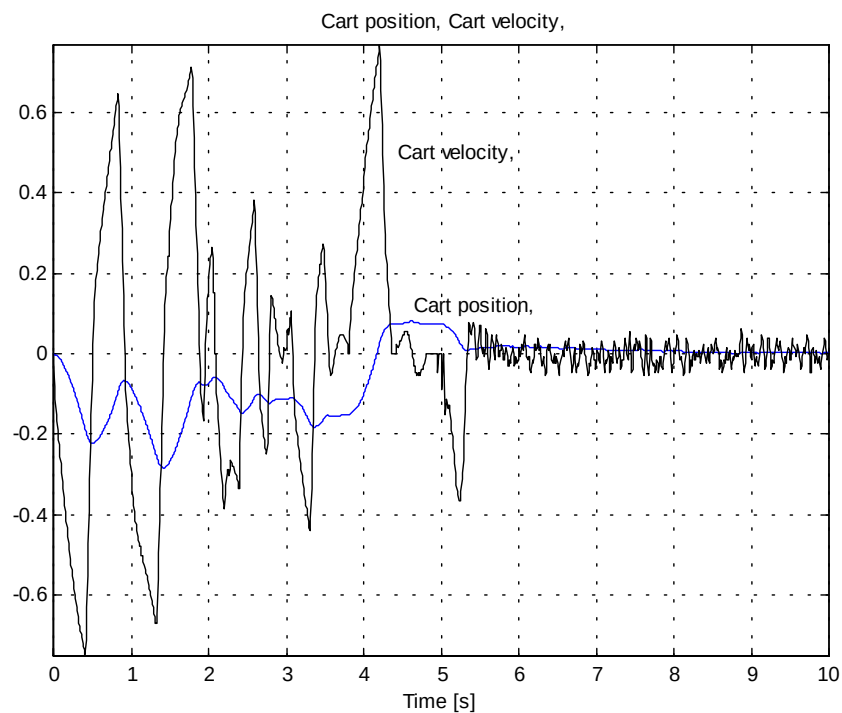


Figure 25: Time diagram of states related to the cart

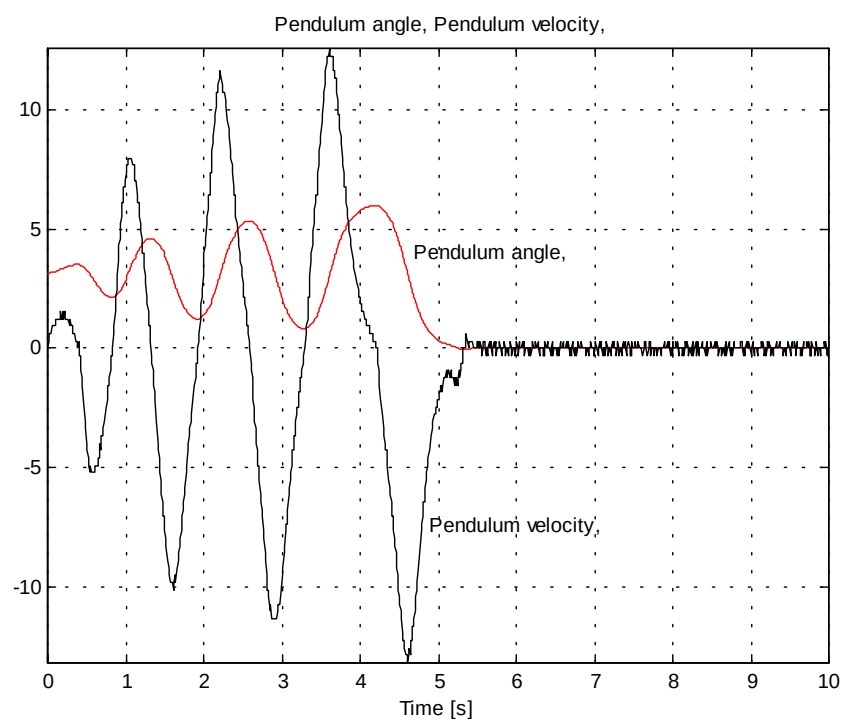


Figure 26: Time diagram of states related to the pendulum

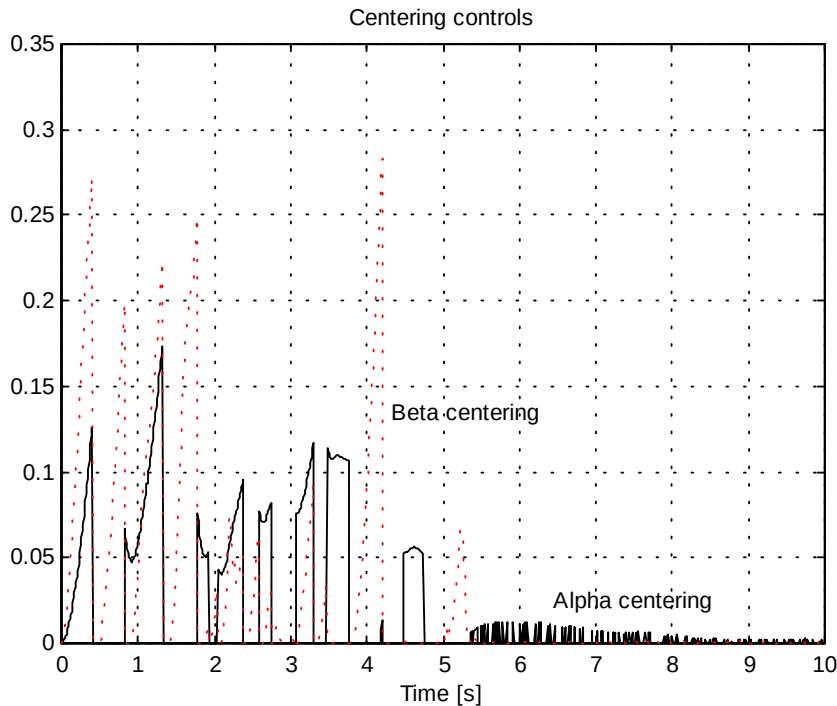


Figure 27: Time diagram of the α , β centring controls

2.1.4 SIMULATION: EXAMPLE 4

Swinging up and stabilisation in the upright position with centring parameters:

$\alpha = 0.8$, $\beta = 0.6$, $\gamma = 0.2$ and the desired cart position signal set to zero.

$x_1=0$ to $x_1=0$

$x_2=\pi/2$ $x_2=0$.

This experiment is performed with all centring rules being active. Introducing the γ parameter value equal to 0.2 we have to change the remaining α , β parameter values. The results of the experiment are shown in Figure 28. The control is depicted in Figure 29. The states that relate to the cart from Figure 28 are depicted pair-wise in a zoomed form in Figure 30. Similarly the states that relate to the pendulum in Figure 28 are depicted pair-wise in a zoomed form in Figure 31. The absolute values of *alpha beta* centring controls are presented in Figure 32 and the absolute values of *alpha beta gamma* centring controls in Figure 33.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

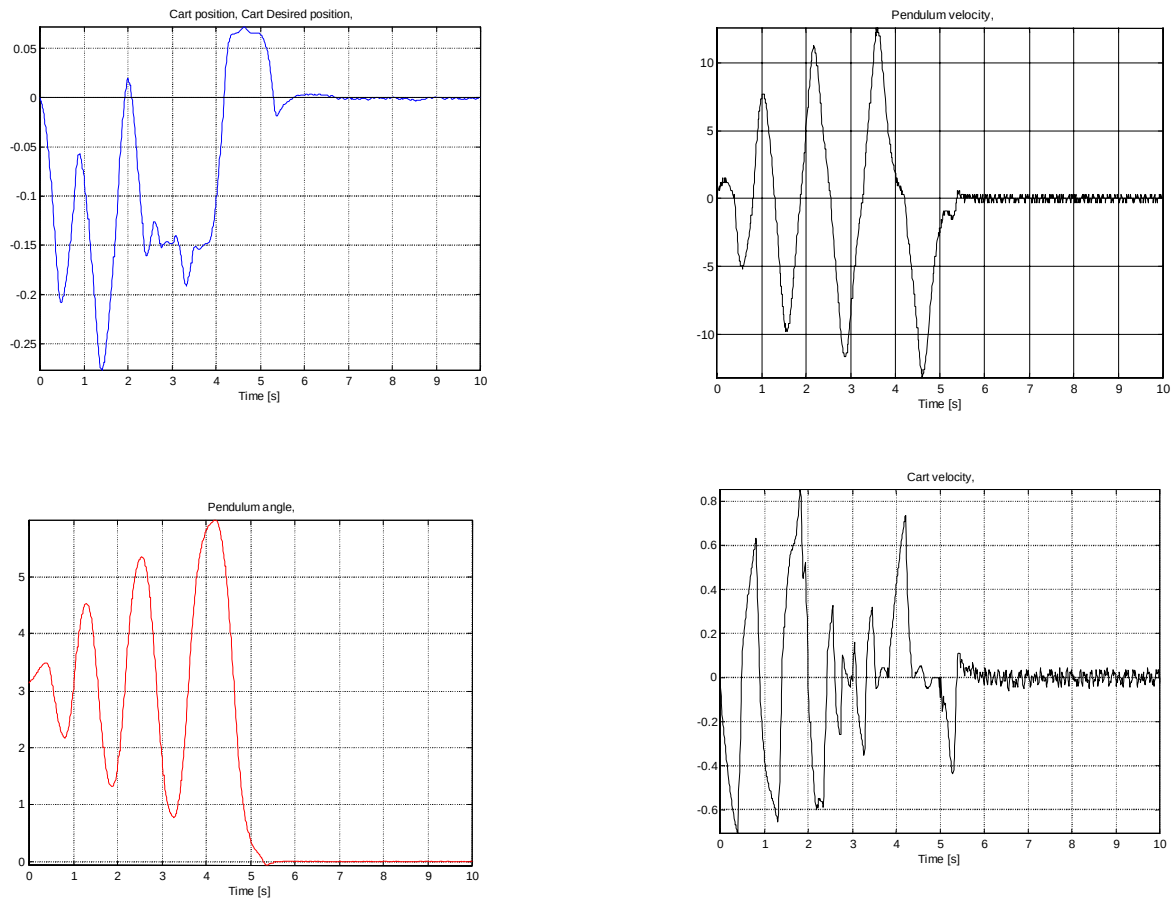


Figure 28: Time diagrams of states

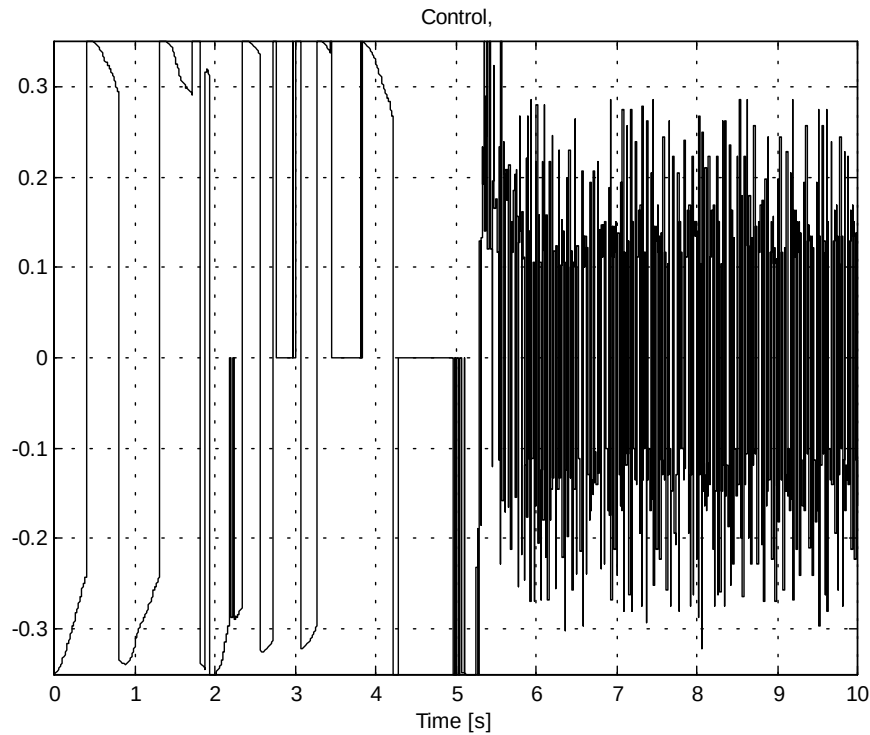


Figure 29: Time diagram of control

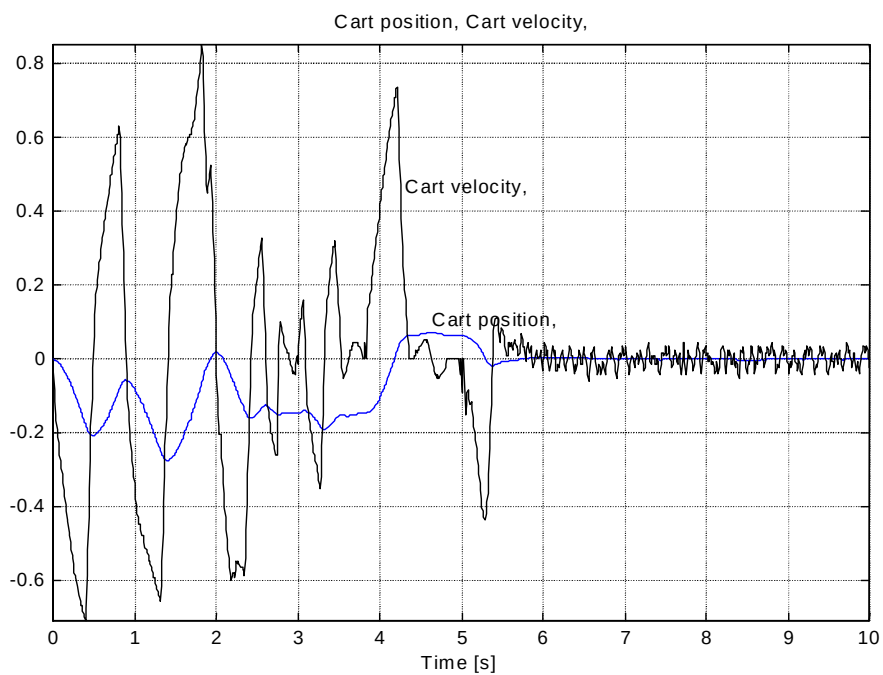


Figure 30: Time diagram of states related to the cart

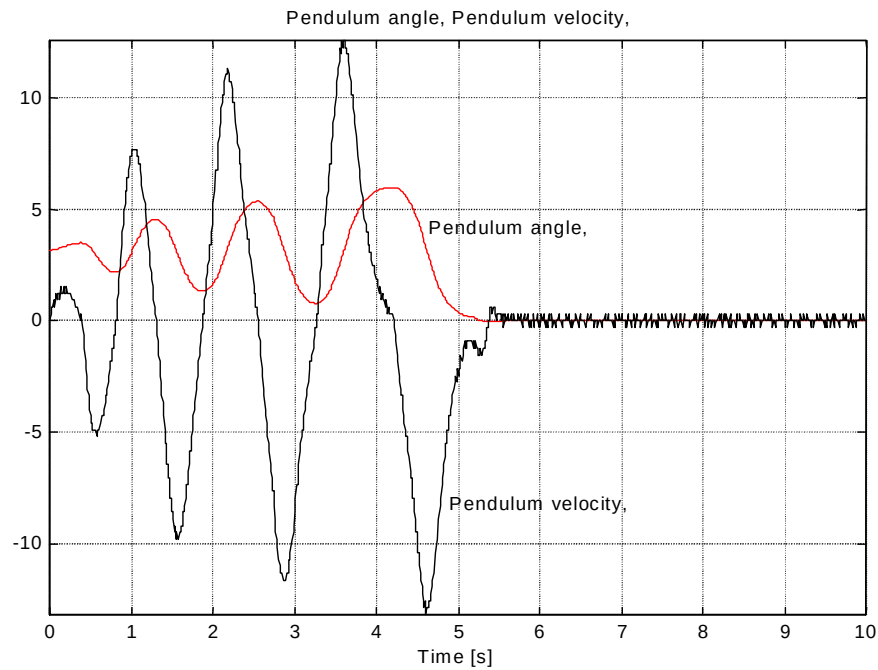


Figure 31: Time diagram of states related to the pendulum

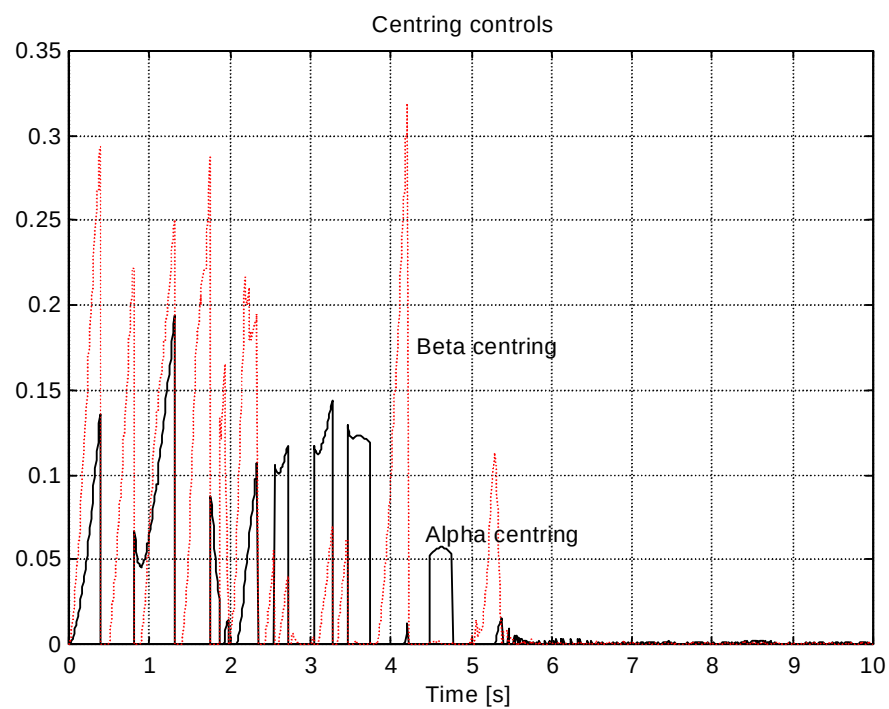


Figure 32: Time diagram of the α , β centring controls

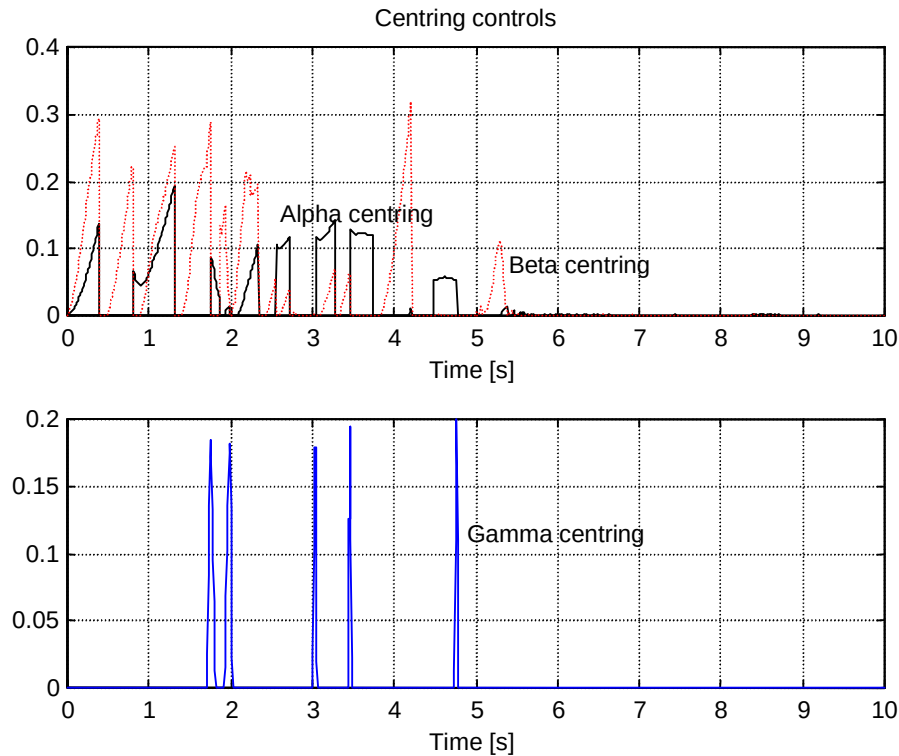


Figure 33: Time diagram of the α , β and γ centring controls

The simulation results of example 4 prove that all centring rules used together are an effective method of centring the cart. We can see in Figure 30 that the maximum cart deviation from the middle of the rail is 0.25m. The stabilisation zone is achieved after 5.4s. We can also notice that the γ control is a narrow spike signal which appears when the pendulum is crossing a small vicinity of the $\pm \pi/2$ angle. The γ centring force applied to the cart at such moments is not disturbing the swinging algorithm much.

Other question is how to choose the parameters α , β and γ ? There are no general rules how to manage the task except for β which is proportional to the cart kinetic energy constant.

Now, equipped with the data and decision making gathered from simulation, we can carry on control experiments with the real pendulum-cart system. Rule-based control experiments will show how far from reality our modelling is.



2.2 RULE-BASED CONTROL PRACTICAL

The results of the simulation experiments from section 2.1 are now compared with the results of the real-time control of plant.

If the previous chapters have not be read or worked through, then the rule-based controllers are invoke by writing:

```
>>pd_rules
```

in the *MATLAB Command Window*

The *pd_rules* window opens (Figure 1) or by double clicking on *Rule-based Controller* block.. All the TOOLS from main menu are needed prior to start a real-time experiment. It is necessary to repeat actions described in the *Getting Started* manual to put the pendulum-cart system into operation. Do not forget to double click on the *Reset Encoders* button. At the beginning it is recommended to check measurements using the *Stethoscope* button with the motor being switched off.

Double click on the *Rule-based Controller* button and you can see the following window (Figure 34). This is the SIMULINK model for monitoring the experiments. **The main goal of the presented model is not to simulate the pendulum-cart system but to transfer data to and from the Real Time Kernel.** This is the significant difference between the presented model and the models in Simulink described in section 2.1.

The visualisation tools are ready to monitor each of the four state variables, the desired position signal, and the control signal. All the variables are stored simultaneously under the name *pd_hist* in the *To workspace* block of Simulink.

Starting a real time experiment is done by selecting from the top-bar menu *Simulation* and then *Start*.

Note that parameters set by default in Real Time Task may not give a good performance for your system, as pendulum physical systems may differ(Fig.35). Follow the identification of pendulum parameters routine form Teaching Manual1. Modify in Pendulum Rule-Based Toolbox the respective parameters in the simulation experiment and then real-time experiment.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

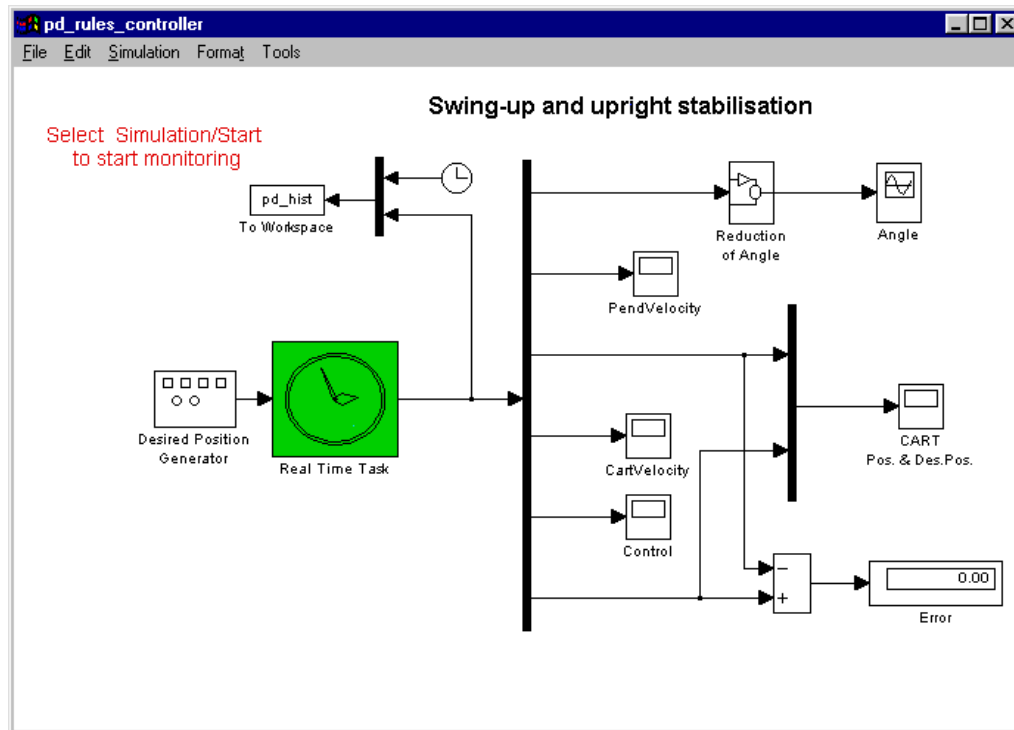


Figure 34: Experiment monitoring window

Real Time Task

Real Time Model Parameters (mask)
Parameters for the Real Time SIMULINK Model

Parameters

Friction:
0.1

LQ parameters [K1 K2 K3 K4]:
[0.8 8.0 1.4 0.6]

Rail limit:
0.35

Stb. zone:
0.3

Centering factors [alpha, beta, gamma]:
[0.8 0.6 0.1]

Downsampling ratio:
10

☐ External generator usage flag

☒ Stop practical after Simulation/Stop

Apply Revert Help Close

Block Parameters: Real Time Task

Real Time Model Parameters (mask)
Parameters for the Real Time SIMULINK Model

Parameters

Friction:
0.23

LQ parameters [K1 K2 K3 K4]:
[0.65 8.0 1.2 0.8]

Rail limit:
0.35

Stb. zone:
0.3

Centering factors [alpha, beta, gamma]:
[0.5 0.4 0.1]

Sample time:
0.01

Downsampling ratio:
10

☒ Set RTK parameters

☒ External generator usage flag

☒ Stop practical after Simulation/Stop

Apply Revert Help Close

Figure 35: Rule base *Real Time Task* mask



2.2.1 PRACTICAL: EXAMPLE 1

Swinging up and stabilisation in the upright position with centring set to zero (α , β , $\gamma=0$) and the desired cart position signal equal to zero.

We can set the same values of control parameters as in the corresponding simulated experiment shown in Figure 14 or set the values closer to these, an example is shown below. Double click the *Parameters of the controller* button in the *pd_rules* window. The following window opens and makes it possible to change values of the parameters (Figure 36)

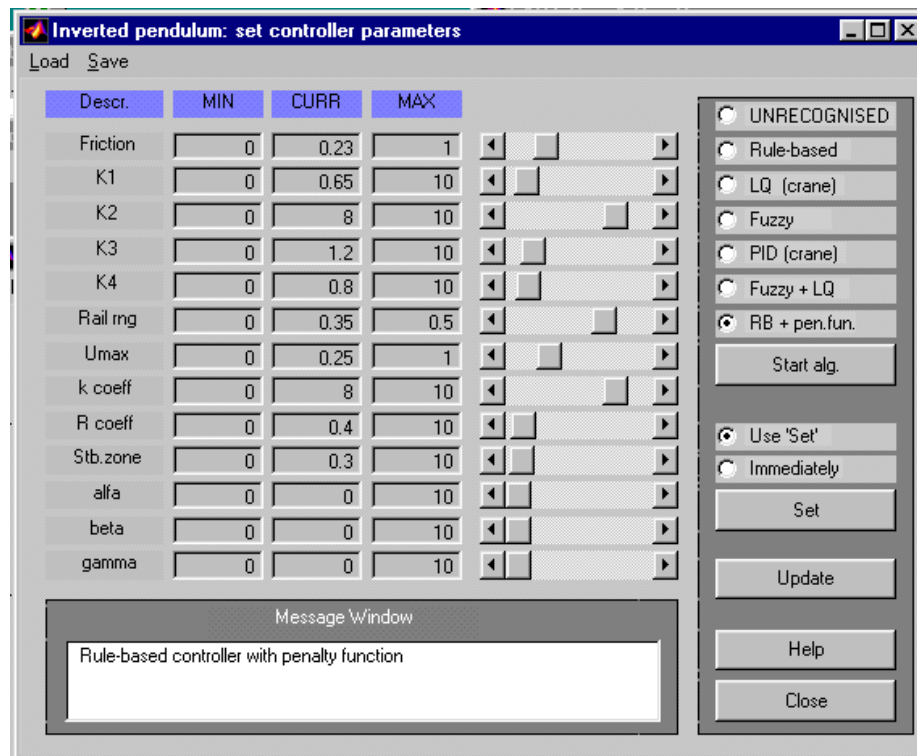


Figure 36: The *Parameters of the controller* window

Start the Real-time experiment from *pd_rules_controller.m* or select *Rule-Based Controller* window. The default control parameters are set in the green box called *Real Time Task* and they can be modified in the above window while the control is running (Figure 36).



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

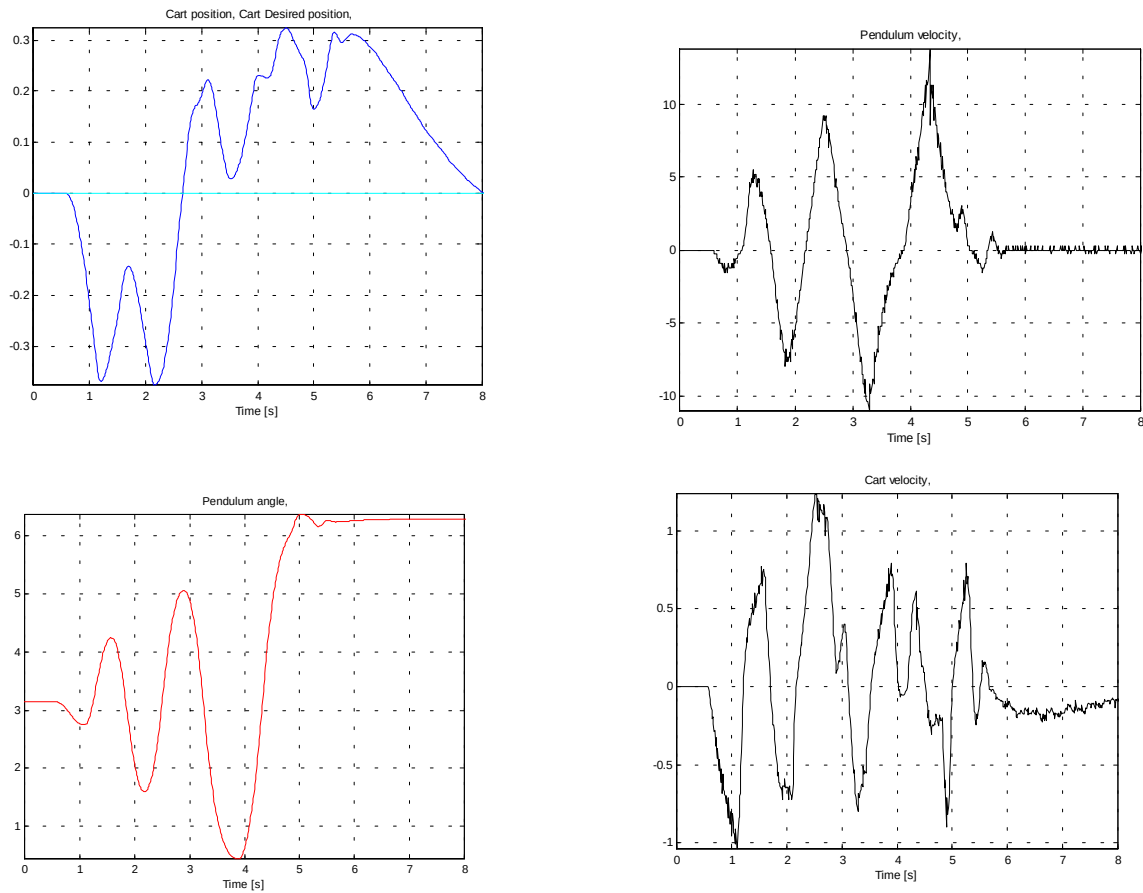


Figure 37: Time diagrams of states

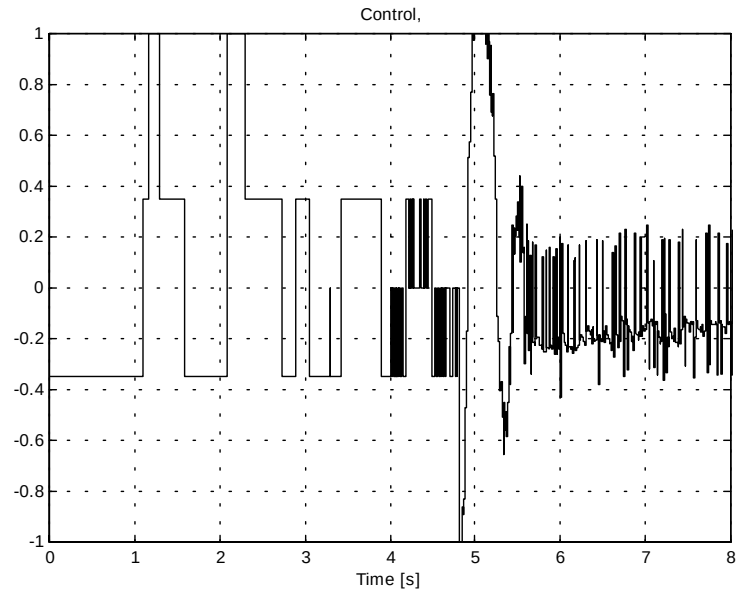


Figure 38: Time diagram of control

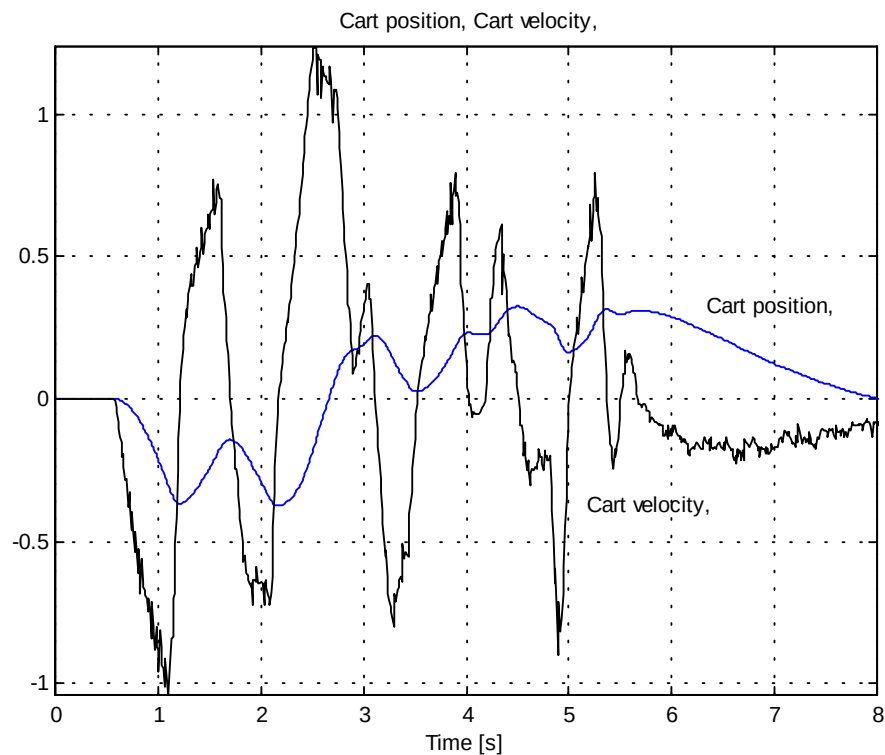


Figure 39: Time diagram of states related to the cart

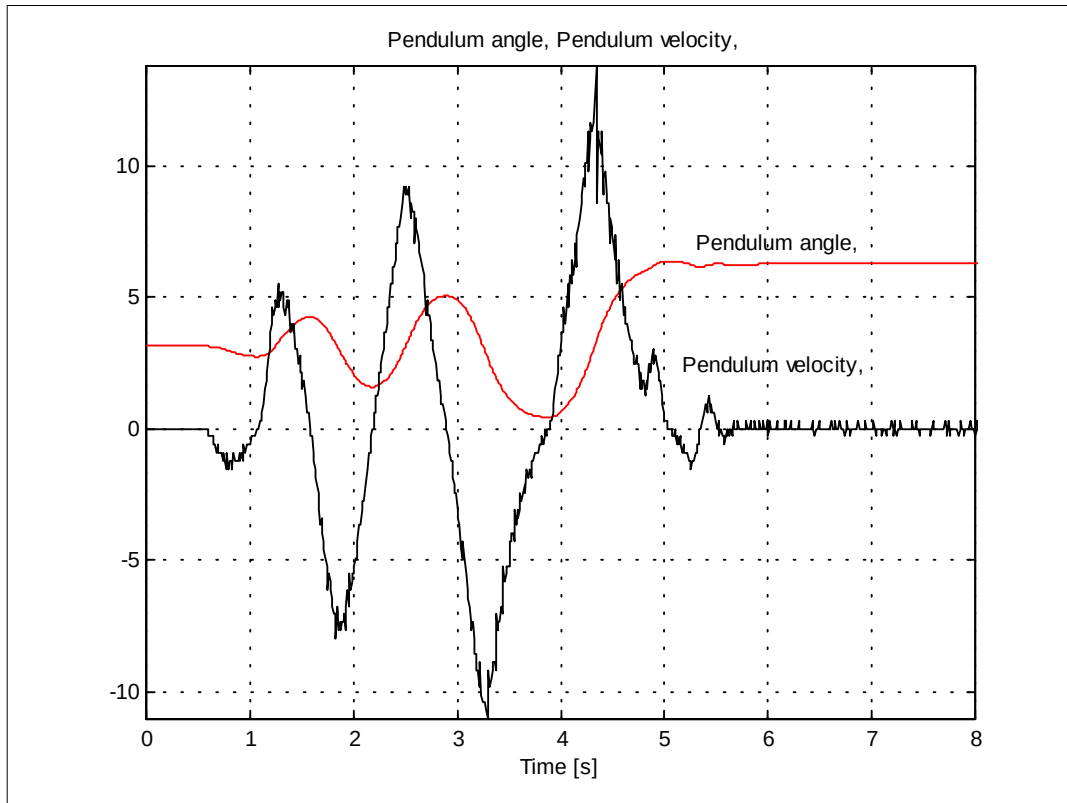


Figure 40: Time diagram of states related to the pendulum

The results of the experiment are given in Figure 37. The control is depicted in Figure 38. The states that relate to the cart from Figure 37 are depicted pair-wise in a zoomed form in Figure 39. Similarly the states that relate to the pendulum in Figure 37 are depicted pair-wise in a zoomed form in Figure 40

Both the simulated model and the real system behave similarly. Of course there are some differences. However, notice that the real system starts at the moment when 0.8s has elapsed. The control times are thus in both models nearly the same. We observe also the same number of swings that are needed to achieve the stabilisation zone. As far as the cart and pendulum maximum velocities are concerned conformity of numerical values for both systems is evident.



2.2.2 PRACTICAL: EXAMPLE 2

Swinging up and stabilisation in the upright position with centring set to zero (α , β , $\gamma=0$) and the desired cart position signal in a square-wave form.

In this example the *Desired Position Generator* block is set to be active. The desired cart position is defined as a square-wave signal of the frequency 0.05 Hz and amplitude equal to 0.1m.

If you click the *Desired Position Generator* block then the following window opens (see Figure 41).

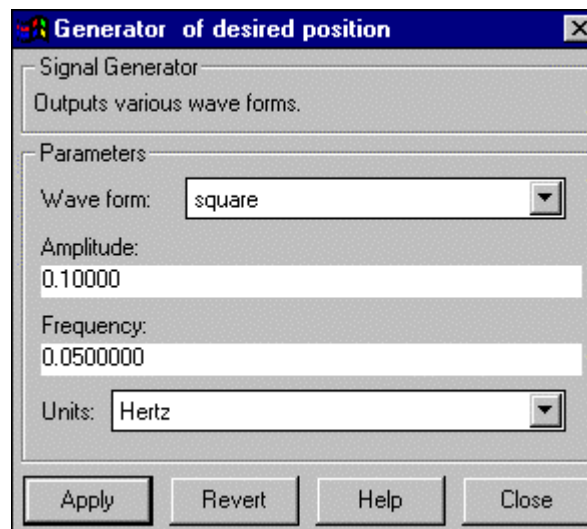


Figure 41: Setting parameters of the *Desired Position Generator* block

If you click the *Real Time Task* block then the window shown in Figure 42 opens. The *External usage generator flag* is set on to activate the signal from the generator. The second flag in this mask is responsible for stopping the experiment if the simulation has been stopped. *Downsampling ratio* is the property of the visualisation task.

Otherwise use the same control parameters as shown in Figure 36.



Figure 42: Mask of the *Real Time Task* block

The following data of the real model are gathered when the experiment starts (see Figure 43 and). Again the real system behaves in a similar way to that of its simulated model. However, comparing Example 1 to Example 2 we can notice that the upright angular position in Example 1 was 2π rad and in Example 2 was 0 rad. In the simulated models the pendulum was steered to 0 rad. If you repeat the experiments you can be sure to obtain the same results in the simulated environment. The experiments in the real time environment due to disturbances may steer the pendulum to one of the two equilibrium points: 0 or 2π rad.

Remember that each time you start an experiment the data is stored in the Workspace of MATLAB in the *pd_hist* vector variable. Be aware of the *Downsampling ratio* parameter value (see Figure 42). A *Downsampling ratio* equal to 10 means that 9 out of 10 consecutive measurements are lost and only one is stored in the *pd_hist* vector variable.

The data that are visualised in all practical experiments has been stored with *Downsampling ratio* equal to 1. No measurements are lost.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

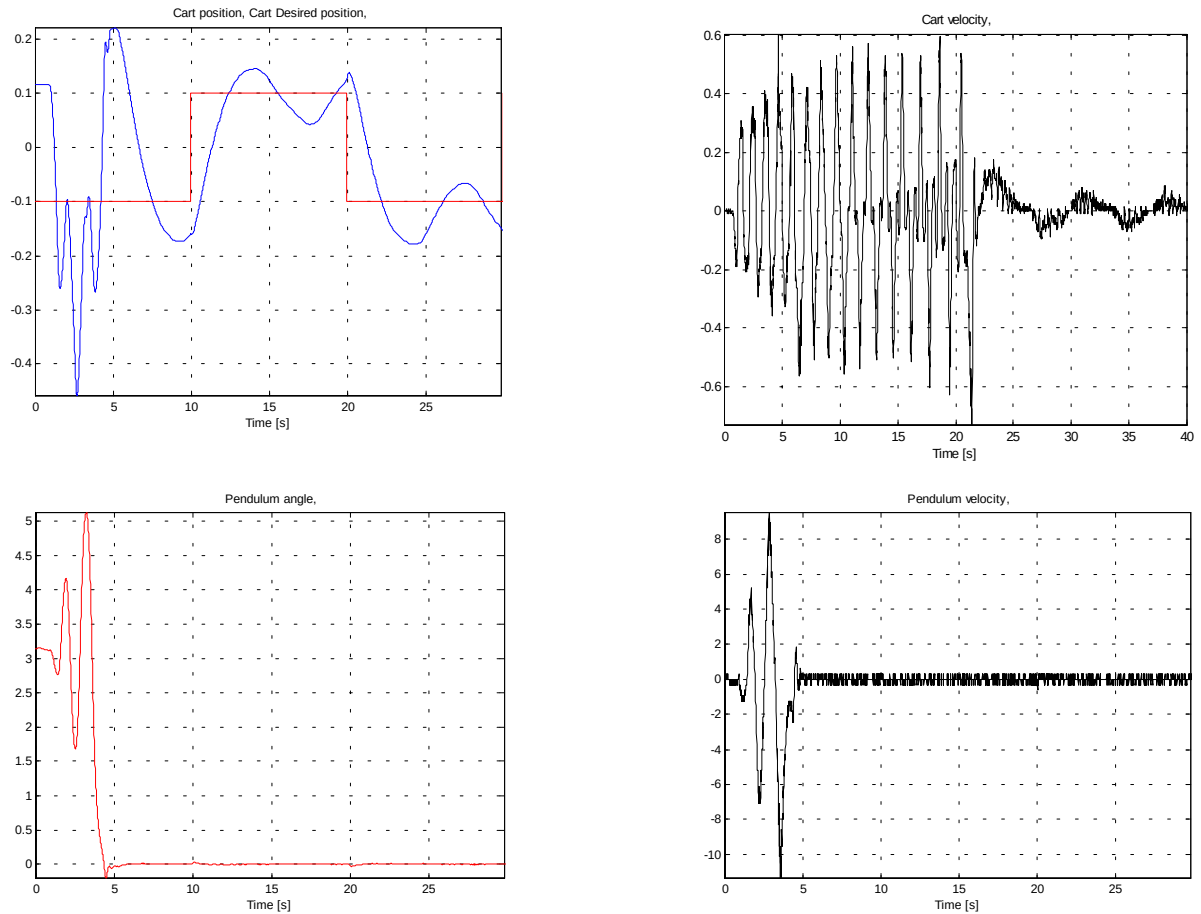


Figure 43: Time diagrams of states

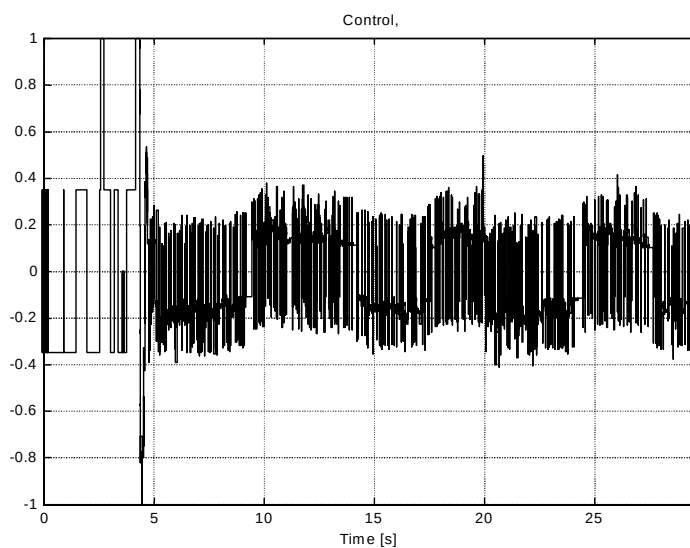


Figure 44: Time diagram of control

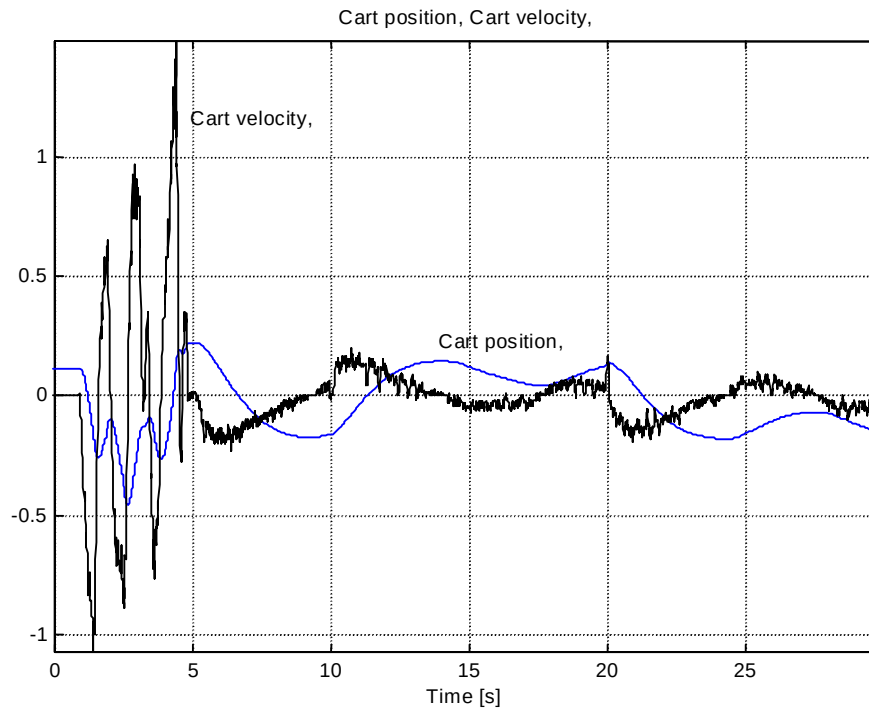


Figure 45: Time diagram of states related to the cart

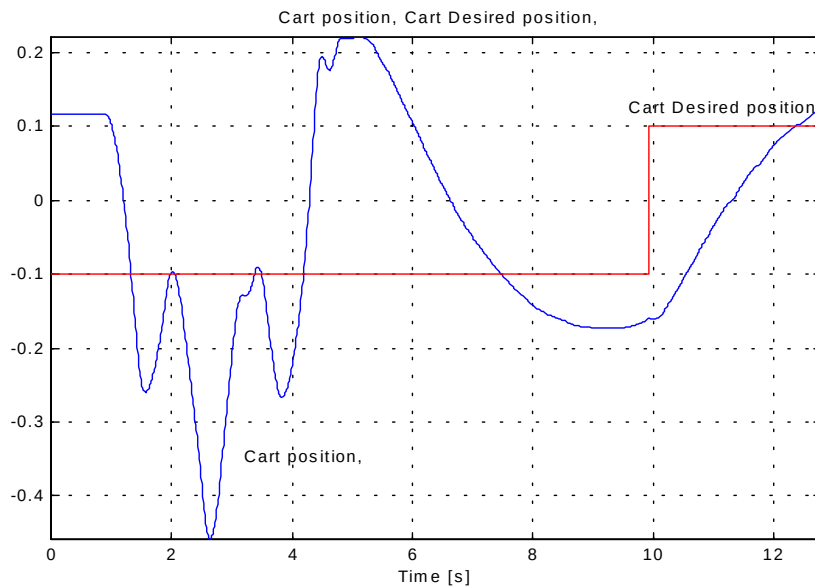


Figure 46: Time diagram of states related to the cart (the first 12 seconds)

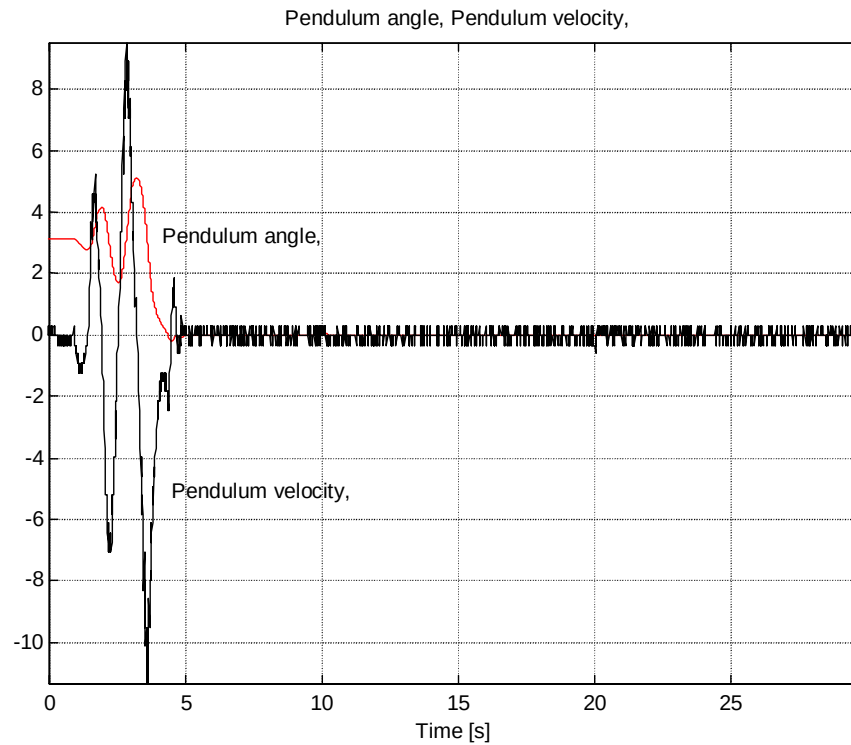


Figure 47: Time diagram of states related to the pendulum

The time diagrams related to the cart variables are given in Figure 45. The first 12 seconds of control corresponding to the cart motion during the swing-up mode is presented in Figure 46. The time diagrams related to the pendulum variables are given in Figure 47. The first 12 seconds of control corresponding to the swing-up policy is presented in Figure 48.

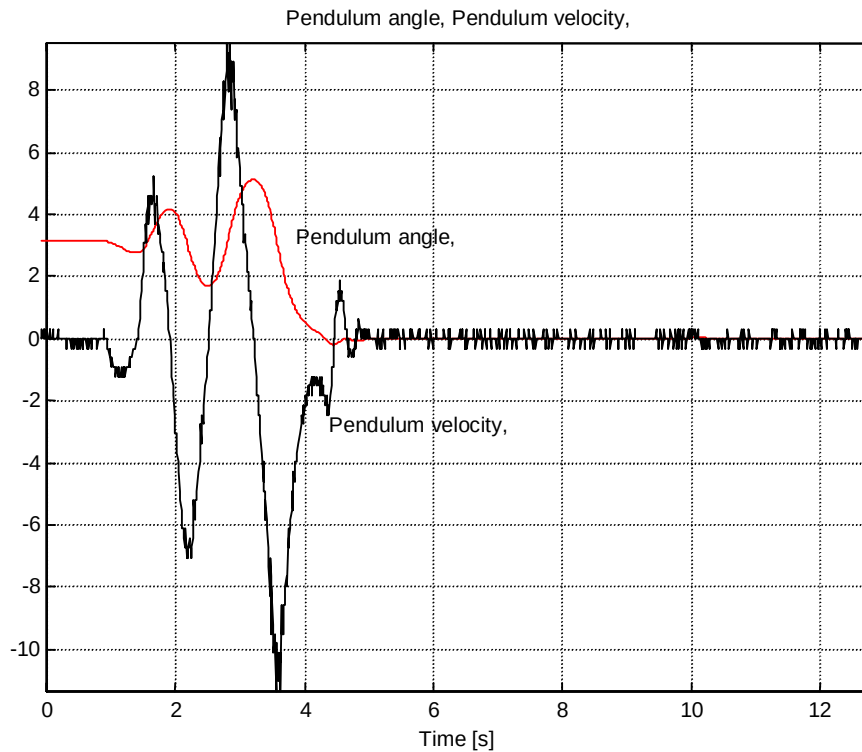


Figure 48: Time diagram of states related to the pendulum (the first 12 seconds)

2.2.3 PRACTICAL: EXAMPLE 3

Swinging up and stabilisation in the upright position with centring parameters:

$\alpha = 0.7$, $\beta = 0.5$, $\gamma = 0$ and the desired cart position signal set to zero

In this example we focus on better centring policy of the cart. Thus the desired cart position signal is set to zero.

Remember to set the α and β parameter values. Click the *Parameters of the controller* button in the *pd_rules* window (see Figure 1).



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 2

Rule-Based Control

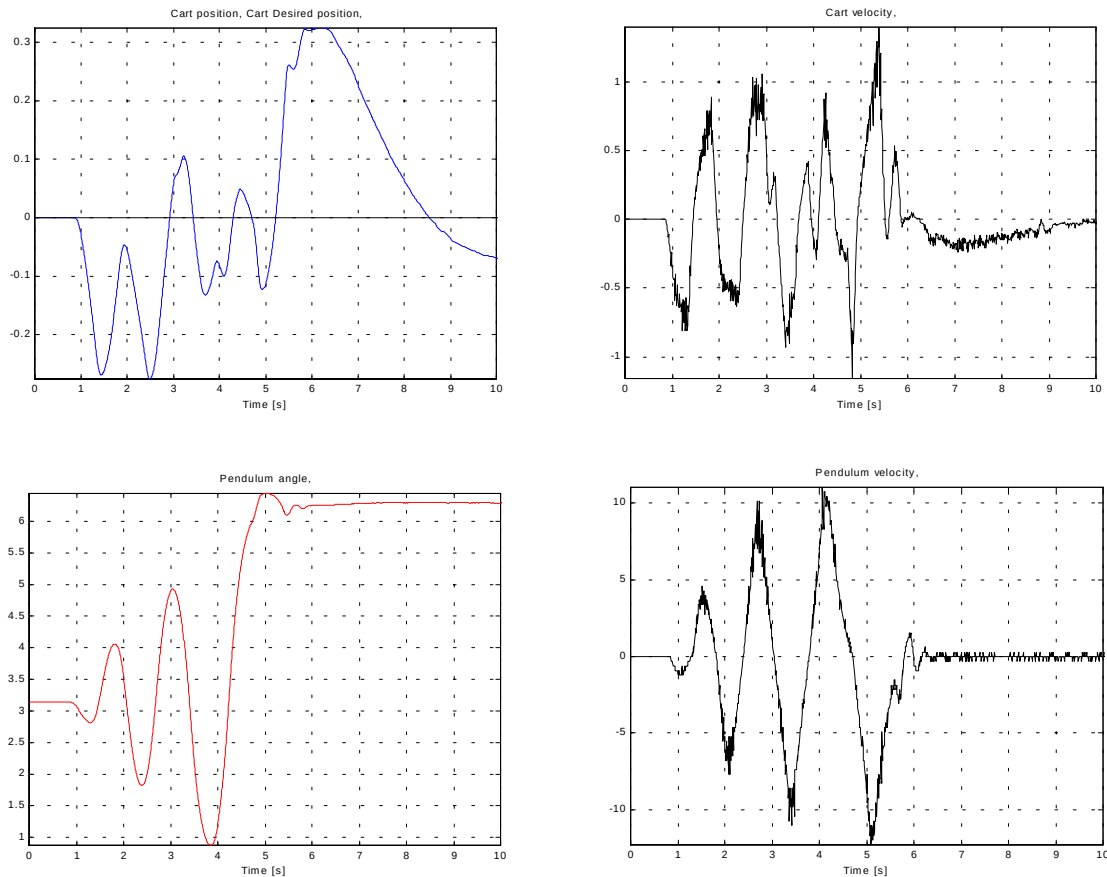


Figure 49: Time diagrams of states

The window given in Figure 36 opens. Now you can enter the α and β values. The real-time control results are given in Figure 49. Control output is shown in Figure 50. The states that relate to the cart from Figure 49 are depicted pair-wise in a zoomed form in Figure 51. Similarly the states that relate to the pendulum in Figure 49 are depicted pair-wise in a zoomed form in Figure 52.

The amplitude of the cart oscillations around the rail centre is smaller. It is well visible in Figure 51. The maximum deviation from the rail centre is less than 0.3 m. Notice also the new shape of the control signal (see Figure 50). The “bang-bang” character is not preserved.

Again a good conformity between the simulation and real-time responses is observed.

This time we present the entire picture of control not dividing it in parts (α - control, β - control). These specific control rules are included in the RTK but are not accessible separately. In practical experiments we can only observe the total control signal.

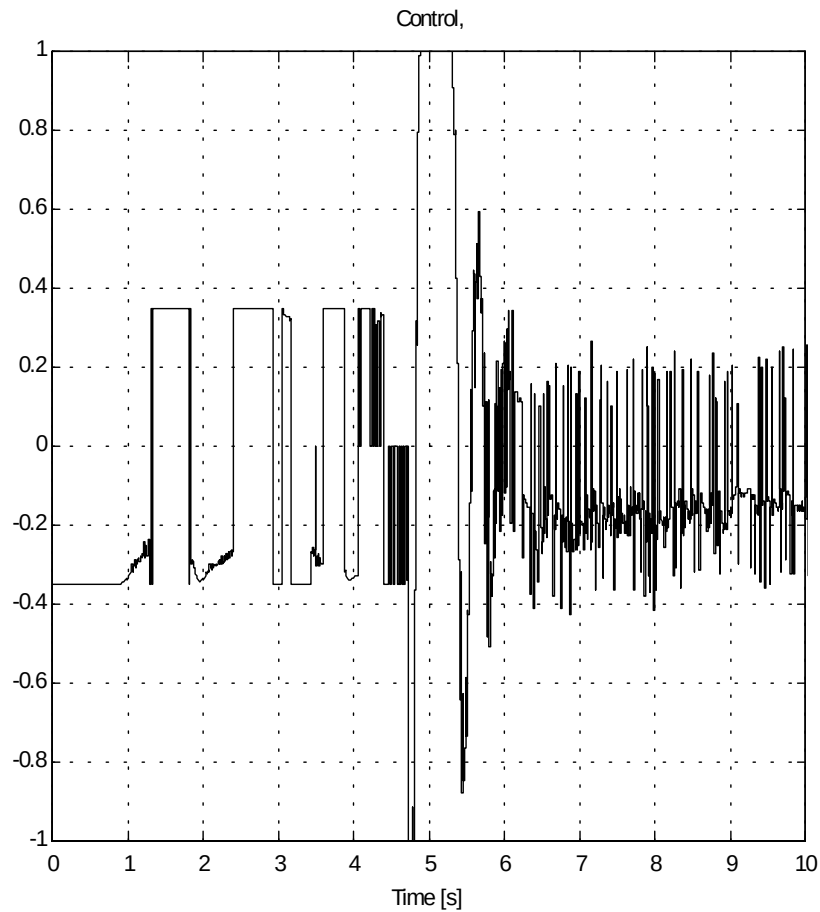


Figure 50: Time diagrams of control

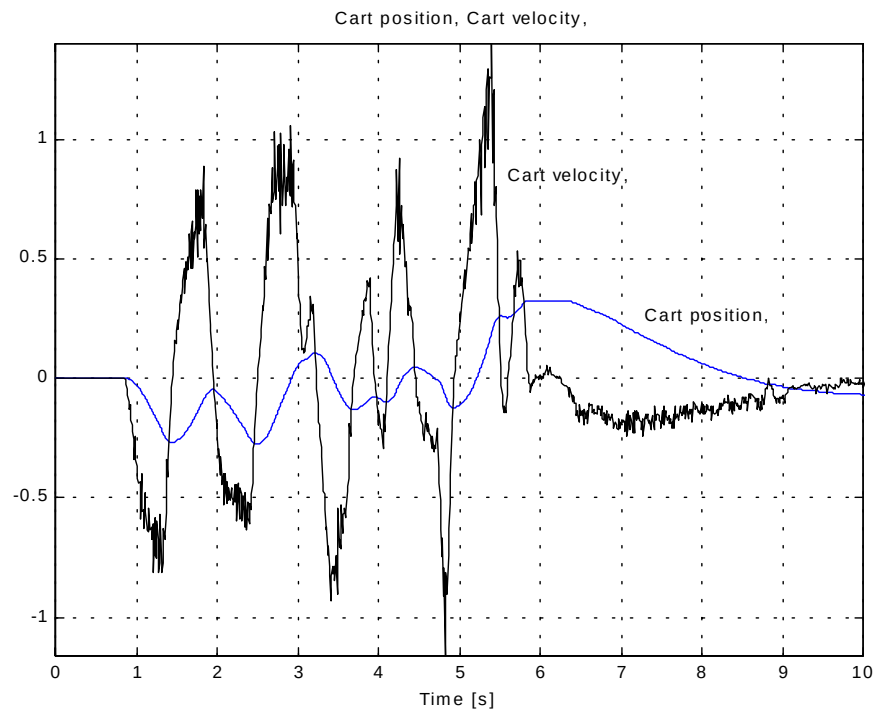


Figure 51: Time diagram of states related to the cart

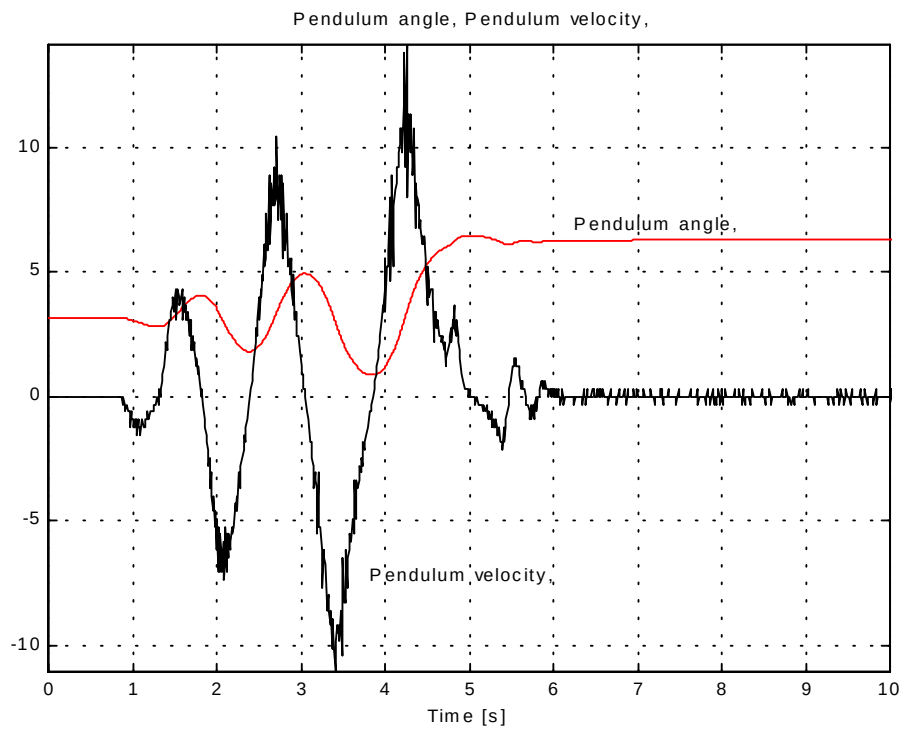


Figure 52: Time diagram of states related to the pendulum



2.2.4 PRACTICAL: EXAMPLE 4

Swinging up and stabilisation in the upright position with centring parameters:

$\alpha = 0.8$, $\beta = 0.6$, $\gamma = 0.1$ and the desired cart position signal set to zero

In the simulated model γ is assumed equal to 0.2. In the real system it is recommended to use a smaller value: $\gamma = 0.1$. All other parameter settings are as in the previous examples. The real-time results are given in Figure 53.

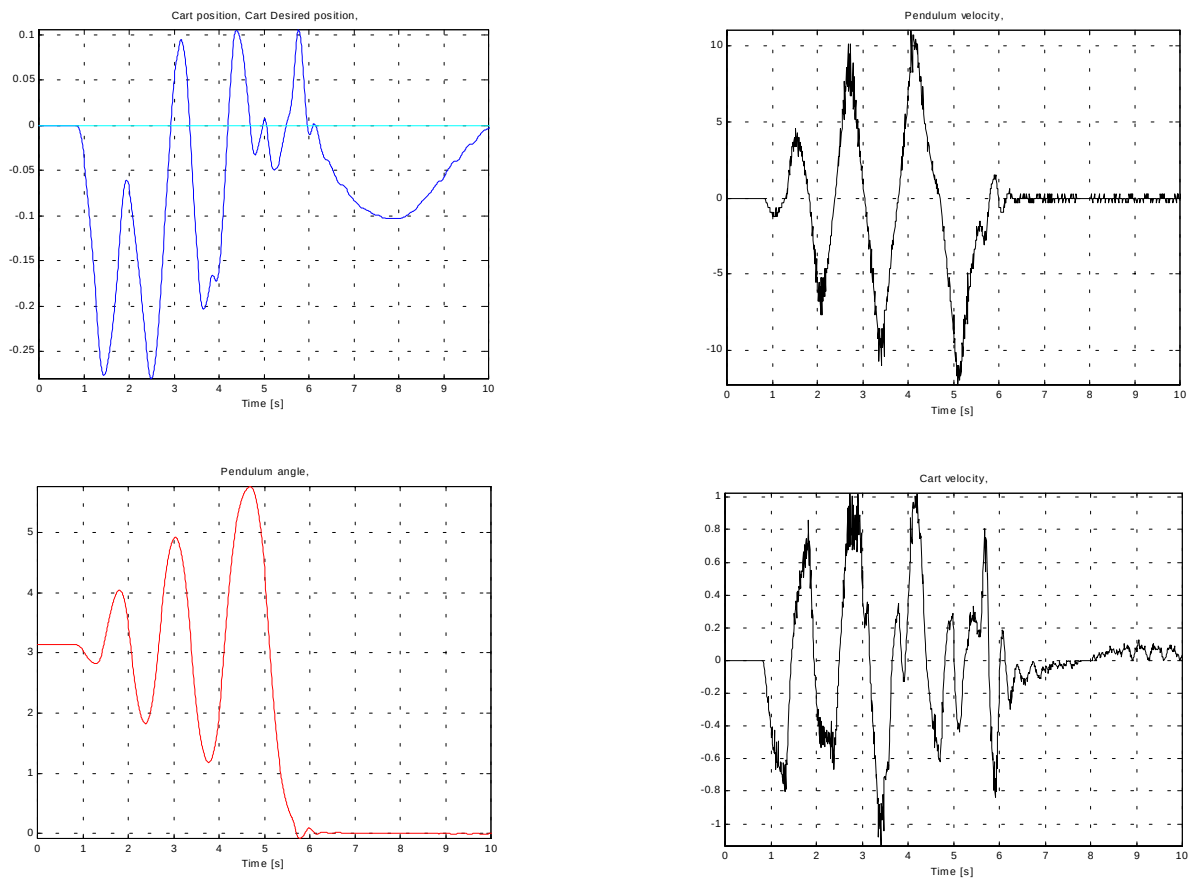


Figure 53: Time diagrams of states



The control is depicted in Figure 54. The states that relate to the cart in Fig.53 are depicted pair-wise in a zoomed form in Figure 53. Similarly the states that relate to the pendulum in Figure 53 are depicted pair-wise in a zoomed form in Figure 55.

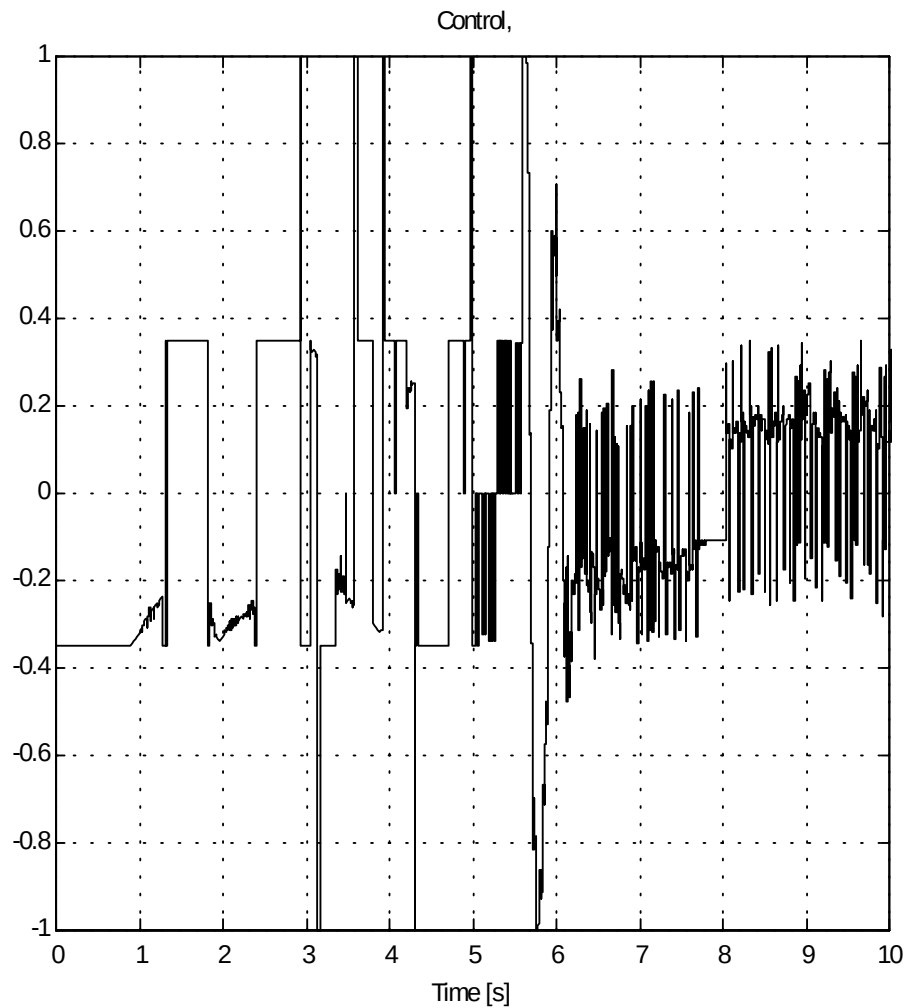


Figure 54: Time diagrams of controls

γ control spikes are visible in Figure 54. The deviations of the cart position corresponding to the middle of the rail are less than 0.25 m.

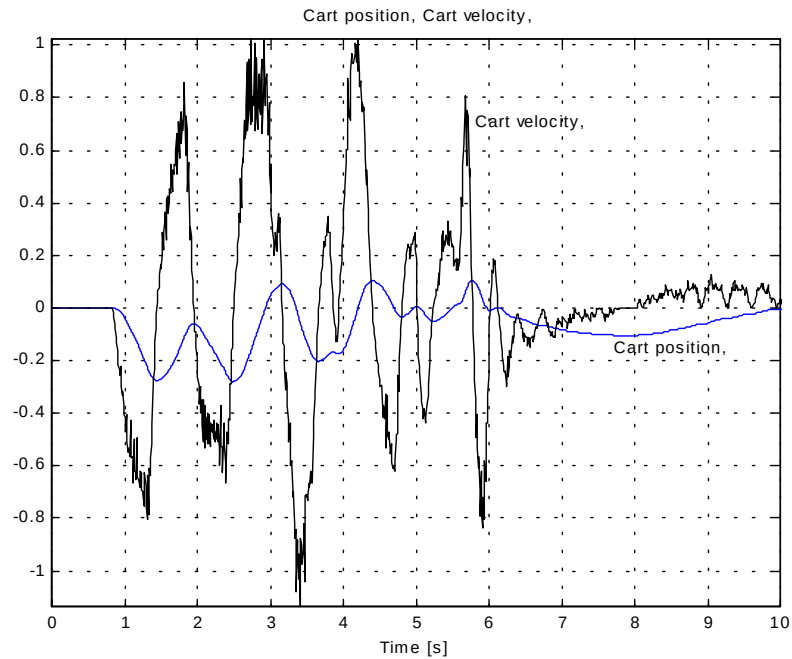


Figure 55: Time diagram of states related to the cart

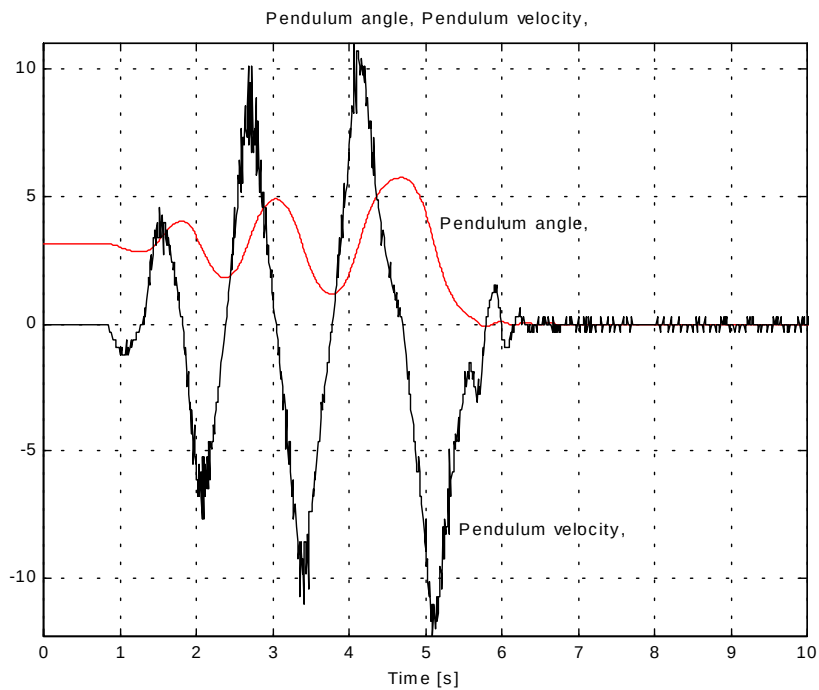


Figure 56: Time diagram of states related to the pendulum



3 Fuzzy Control

We assume that the reader has studied Section 2. It is advisable because throughout the present section we make comparisons and refer to certain details of the previous section. We also assume that the reader is familiar with the *fuzzy set* terminology. Nevertheless we give a brief summary of definitions:

- A *fuzzy set* A , defined over some *universe of discourse*, is a mapping of that universe into the interval $[0, 1]$. The process of creating this mapping is called *fuzzification*.
- A membership function is the curve that defines how true a given statement is for a given input value. The membership takes on the value of zero for no membership and one for full membership. Values in the range 0 to 1 represent partial membership to the given set.
- The *fuzzy set* is defined in terms of a pair $\{x, \mu(x)\}$ for all x in the *universe of discourse*. The *membership function* denoted by $\mu : F \rightarrow [0, 1]$ represents partial or graded membership to the given set A . For example the fuzzy set representing the term *fast* is depicted in Figure 57.

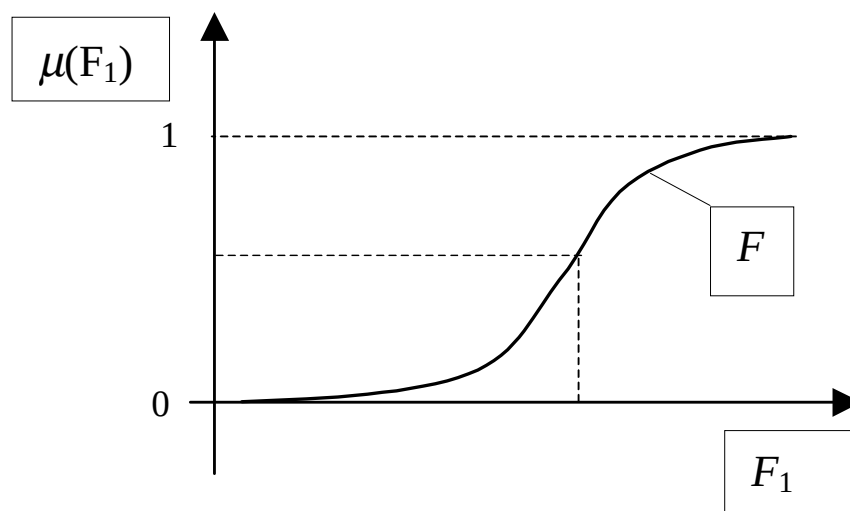


Figure 57: Fuzzy set representing the term *fast*

The curve in Figure 57 defines the transition from *not fast* to *fast*. F_1 denotes velocity measured in m/s.



- A *linguistic variable* takes a linguistic term such as fast, slow, etc.
- Let A and B be fuzzy sets defined over the universe of discourse X . Their intersection, $A \cap B$, is defined in terms of $\mu_A(x)$ and $\mu_B(x)$ as $\mu_{A \cap B}(x) = \min(\mu_A(x), \mu_B(x))$
- Let A and B be fuzzy sets defined over the universe of discourse X . Their union, $A \cup B$, is defined in terms of $\mu_A(x)$ and $\mu_B(x)$ as $\mu_{A \cup B}(x) = \max(\mu_A(x), \mu_B(x))$
- Fuzzy reasoning is called the *inference mechanism*. Fuzzy rules take the form:

If x_A is A and / or x_B is B ... then y_Z is Z

There are fuzzy relations defined over the Cartesian product of the *universes of discourses* of the antecedent and consequent variables. For example, the rule R : If x is A then y is Z , is represented as a fuzzy relation defined as:

$$\mu_R(x, y) = \min(\mu_A(x), \mu_Z(y)).$$

- Interpreting an “If ... then ...” rule involves: evaluating the antecedent (involving *fuzzification*) and applying the result to the consequent (*implication*).
- *Defuzzification* gives a crisp single value of the possibility distribution. The typical defuzzification formula has the form:

$$\text{Mean Of Maximum} = \operatorname{argmax}(\mu_Z(y))$$

Note that possibility distribution is different from probability distribution. It need not satisfy the additivity property of possibility distribution. Possibility distribution places a constraint on the value that may be taken by the variable in question.

The following steps in the design of a fuzzy controller are essential:

- Selection of input/output variables
- Scaling the variables (if necessary)
- Fuzzification: definition of membership functions for all variables
- Fuzzy rules: development of the inference mechanism
- Selection of the defuzzification strategy
- Re-scaling the variables (if necessary).



The point of fuzzy logic is to map the input space to the output space, and the primary mechanism for doing this is the inference system which consists of “if - then” statements called rules. The “if” part of a rule is called the antecedent, while the “then” part of the rule is called the consequent. In fuzzy logic if the antecedent is true to some degree, then the consequent is true to that same degree. More than one rule can be active for the same input value. It means that membership functions partially overlap one another. A few rules can interpolate the *universe of discourse*. The fuzzy control design can be described as building a control algorithm by capturing the knowledge of operation of the process and encoding this knowledge in terms of rules.

In this section we discuss how to create and edit fuzzy inference systems by hand using the MATLAB Fuzzy Logic Toolbox. This toolbox offers two methods of building a fuzzy controller: interactive using the graphical tool and automatic using clustering and the adaptive neuro-fuzzy mechanism. There are also two types of fuzzy inference system: Mamdani and Sugeno. We focus on the Mamdani inference system edited interactively by hand. This approach makes it possible, in a direct way, to understand consequences of modifications introduced into the inference mechanism and membership functions.

The MATLAB Fuzzy Logic Toolbox is designed for simulation purposes. Our goal is to use it also to control the real system. Therefore we have created a toolbox dedicated to the pendulum-cart system. The name of this toolbox is *pd_fuzzy*. The main parts of it are three mechanisms: simulation using the pendulum-cart Simulink model, real-time experiment using the Fuzzy RTK (Real Time Kernel) and the editor of the fuzzy pendulum-cart controller using the MATLAB Fuzzy Logic Toolbox.

In the *pd_fuzzy* toolbox one can also find a number of tools. Some of them are necessary for a real-time control experiment others are optional and auxiliary.

The mechanism used to create a fuzzy controller can be applied successfully if it meets the specifications of the *pd_fuzzy* toolbox. These specifications are reduced to the number and order of input/output signals. A fuzzy controller must be equipped with four inputs (the state variables) and one output (the control). The order of variables is the following: cart position, pendulum angle, cart velocity and pendulum angular velocity.

We start with the demos of three fuzzy controllers (see sections 3.1, 3.2 and 3.3). In section 3.4 we described how to run a controller of your own design.

To start the fuzzy toolbox write:

```
>>pd_fuzzy
```

in the MATLAB Command Window.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3 Fuzzy Control

There are three controller demos: *Stabilising*, *Swing-up* and *Crane* (see Figure 58). Each of the three demos can be activated in the simulated mode (by clicking twice any of the three blue buttons) or in the real-time mode (by double clicking the red button). One can also open the fuzzy editor to visualise the inference fuzzy system (using the black button).

The tools in the *pd_fuzzy* window are similar to these given in the *pd_rules* window (Figure 1) except for the two new tools: *Release FIS System* and *Show FIS Message*.

If you click the *Release FIS System* button then you can perform a new experiment without resetting the computer. The *Show FIS Message* button lists all warnings that occur during a real-time fuzzy experiment. A typical warning is generated by the fuzzy toolbox if the current input or output variable is outside the assumed *universe of discourse*. As far as simulation is concerned warnings do not stop the simulation run. They only slow down the execution of simulation by writing messages in the MATLAB Command Window. In the real-time mode warnings do not stop the run of the system either, they are stored in the memory for further analysis of errors. When the experiment is finished you can read the list of warnings having double clicked the *Show FIS Message* button.

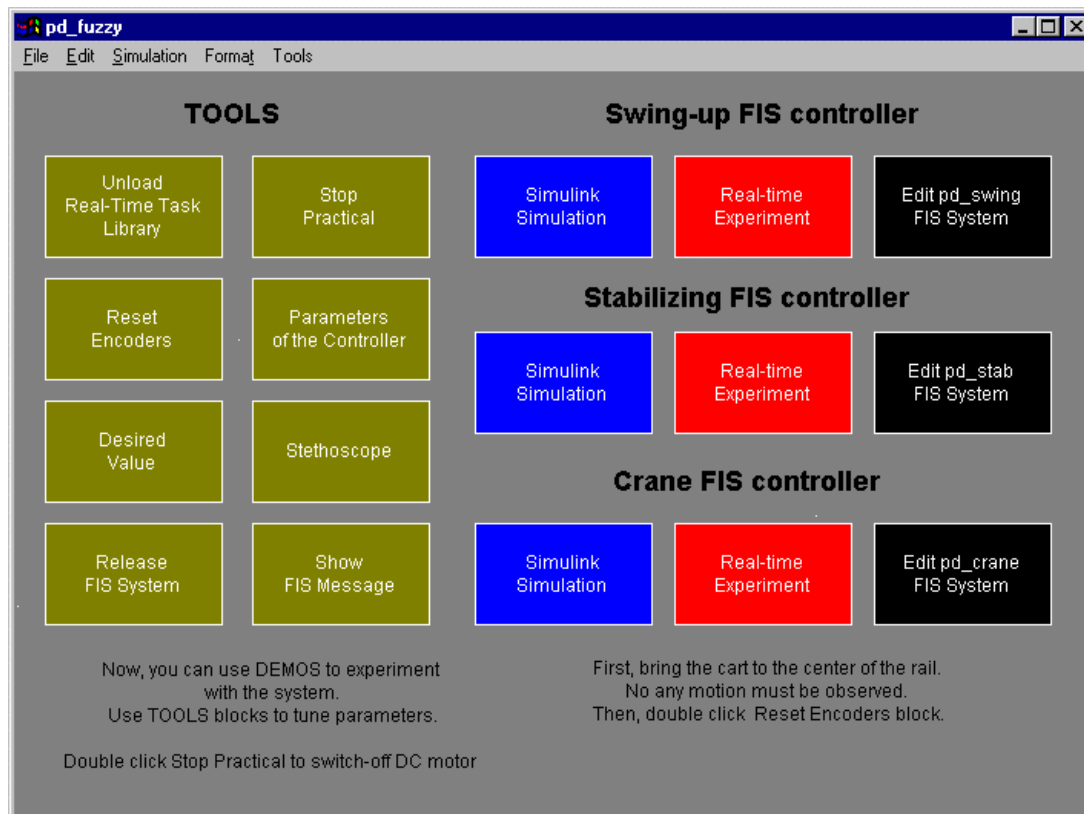


Figure 58: Main fuzzy *pd_fuzzy.mdl* window



We describe below, without going into details, how to run a simulation, how to run a practical and how to use the fuzzy editor tools. The descriptions and visualisations of results related to particular demos are given in sections 3.1, 3.2 and 3.3.

How to run a simulation?

We assume that the *pd_fuzzy* window has been opened. Double click the blue *Simulink Simulation* button to open the corresponding simulation model. For example, if we click the blue *Simulink Simulation* button in the *Crane FIS Controller* row then the window given in Figure 59 opens.

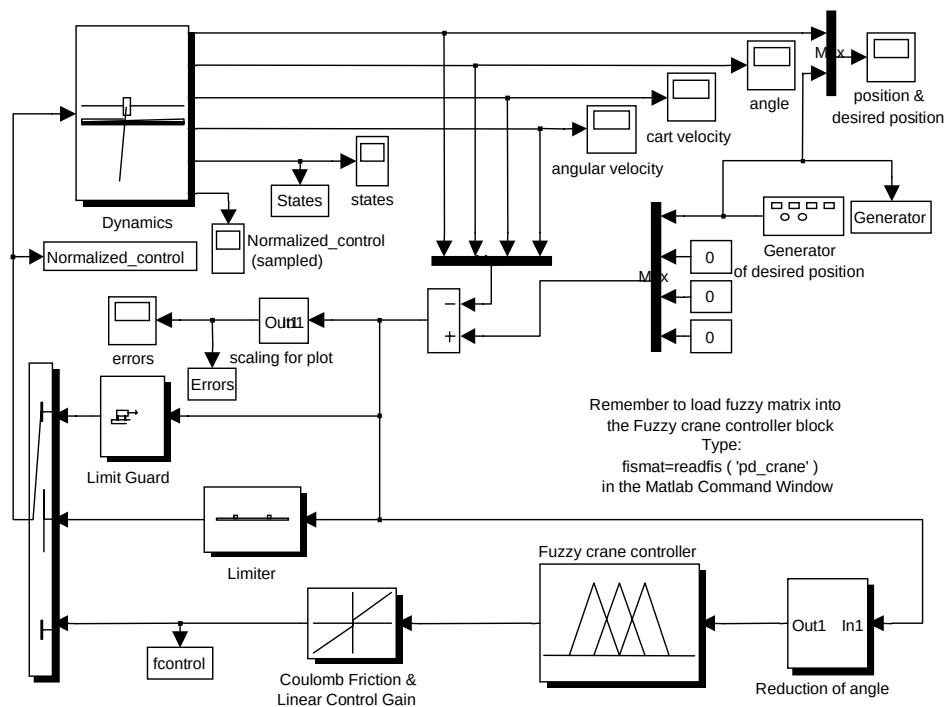


Figure 59: An example of fuzzy simulation. The *pd_mcrane.mdl* crane controller model

Double click on Fuzzy crane controller box will open the following window. This shows the name of the Fuzzy Inference Matrix name that is being used in the simulation process.

Obviously, you can design your own set of rules into a FIS editor, save it as *.fis and then this can be used in any simulated experiments to verify its results.

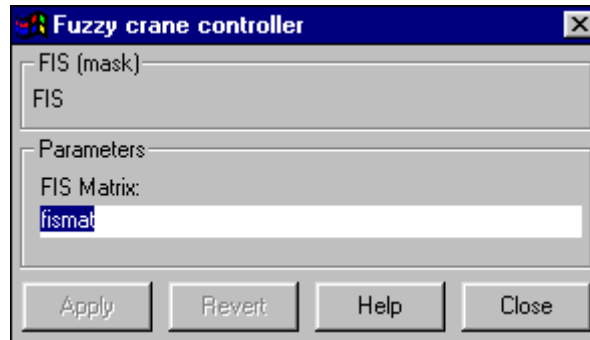
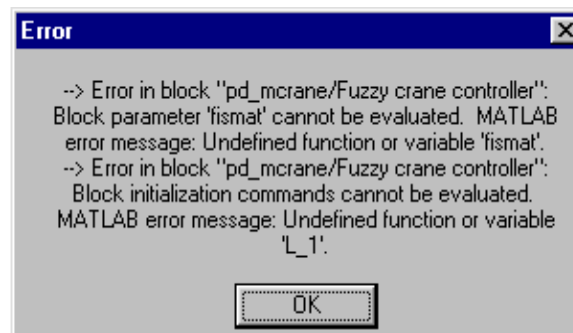


Figure 60: Mask of the *Fuzzy crane controller* block

The *pd_crane.fis* file (the so called Fuzzy Inference System FIS Matrix) is loaded from the disk to the *Fuzzy crane controller* block.

If this file is not loaded to the Matlab workspace, the following error message appears:



To overcome this, type at Matlab prompt : ***fismat = readfis('pd_crane');*** The fuzzy controller is now available for use in *Fuzzy crane controller* block in the Simulink diagram. Start simulation as usually proceeded.

How to run a practical?

We assume that the *pd_fuzzy* window has been opened. Double click the red *Real-time Experiment* button to open the corresponding real-time model. For example, if we click the red *Real-time Experiment* button in the *Crane FIS Controller* row then the window given in Figure 61 opens.

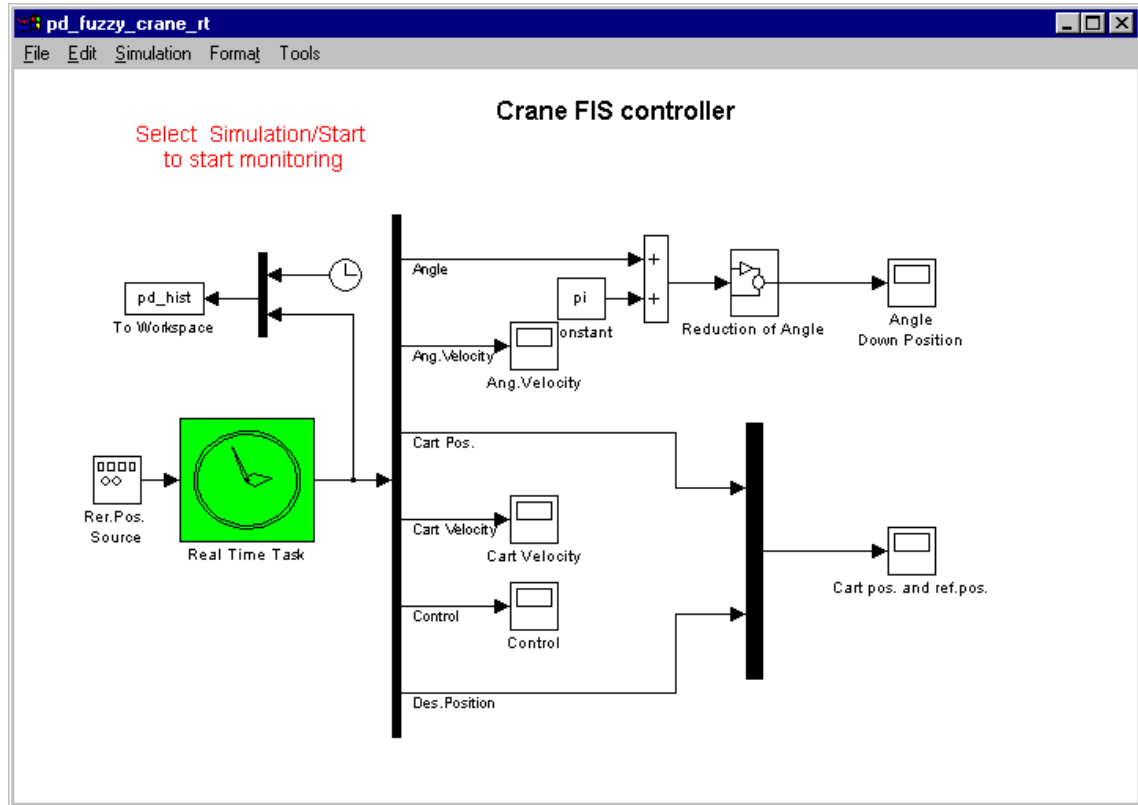


Figure 61: Crane real-time controller

This model looks like a typical Simulink model. However we must remember that it is used for monitoring and parameter change of real pendulum cart system and not for simulation.

Before starting a practical be sure that hardware configuration is correct and *Reset encoder* action has been performed correctly. You can check also the mask of the *Real Time Task* block. Double click this block and the window shown in Figure 62 opens. Most of the parameters are already familiar to the reader. *FIS output gain* and *Alpha offset* are two new parameters. The *FIS output gain* parameter refers to the fuzzy controller as a part of the entire control algorithm while the $U_{\max}$ parameter in the range $[0, 1]$ reduces the maximum control. The *Alpha offset* parameter equal to π indicates that the pendulum is in its down position (the crane mode).

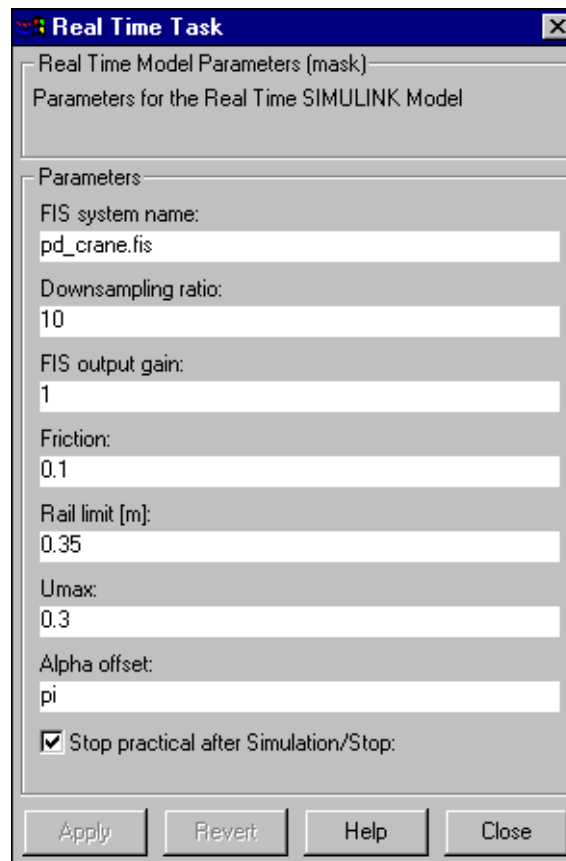


Figure 62: RTK mask with parameters

Note that the fuzzy controller does not replace all other control algorithms. It augments them. We can say that fuzzy control is blended with other control techniques.

Parameter tuning is only required if the system responds wrongly.

Select the **Simulation** pull-down menu, and choose **Start** to run the system. The message window appears (see Figure 63). It means that the code conversion was successful. Click the **OK** button. The new window appears (see Figure 64). The FIS matrix of the fuzzy controller is loaded. The system is ready to start. If you click the **OK** button the real time experiment begins.

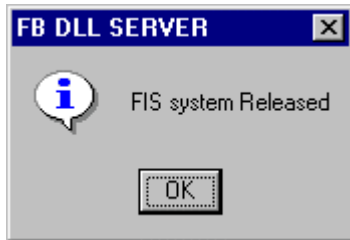


Figure 63: Message after the fis matrix Conversion

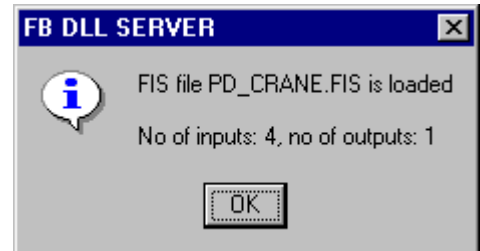


Figure 64: Message related to the start of fuzzy controller

You can stop the experiment in two ways:

- If the flag *Stop practical after Simulation/Stop* is marked then clicking on **Stop** in the **Simulation** pull-down menu
- Otherwise you must click the *Stop Practical* button in the main menu (*pd_fuzzy* window)

During the run of the experiment any vital data can be visualised by opening the respective scopes. All graphic tools are available from MATLAB and Simulink. An example of data visualisation is shown in Figure 65.

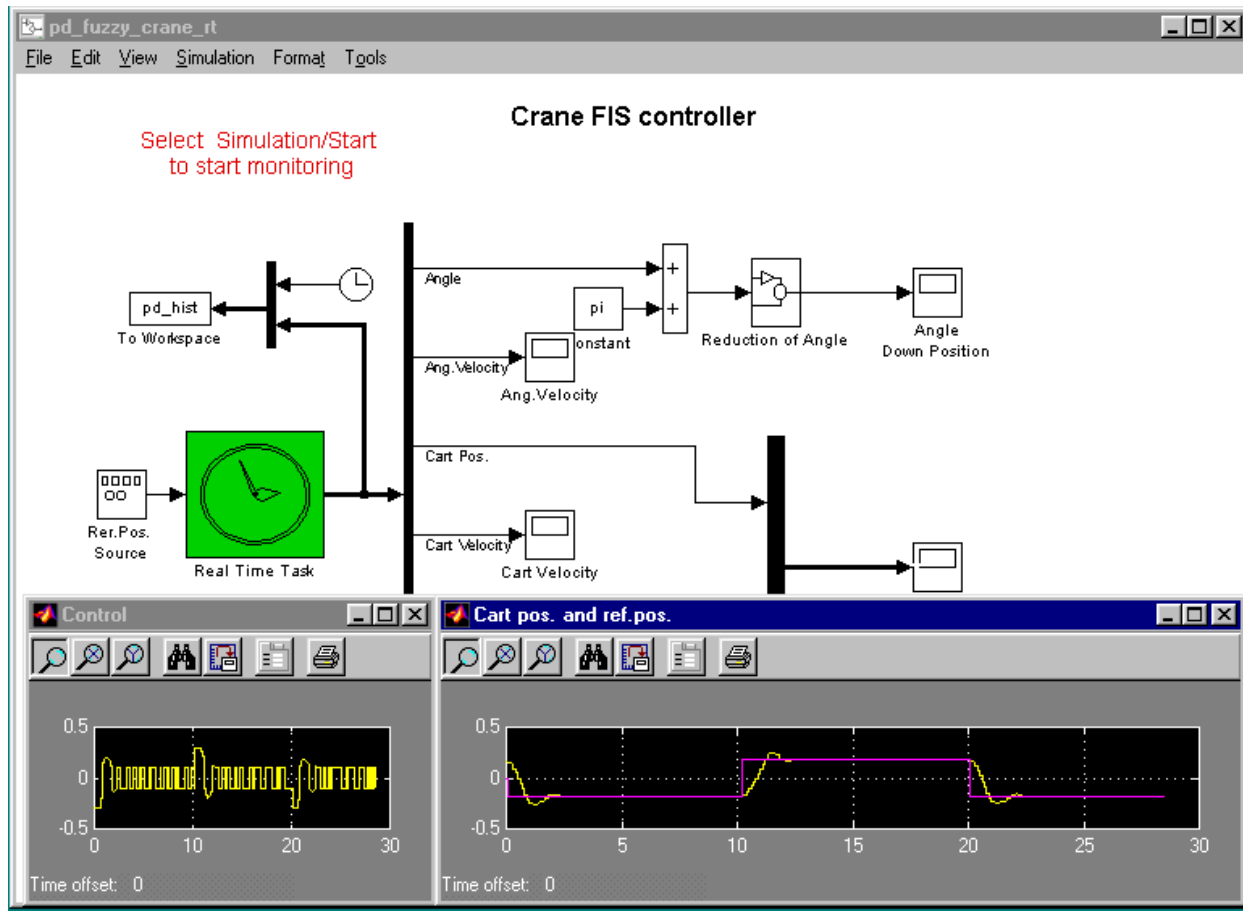


Figure 65: Data captured by real-time controller

3.1 CRANE MODE

3.1.1 FUZZY SET

This exercise looks at the simplest example of control. This is only when cart motion is controlled (one-input, one-output). fuzzy controller is described below. Below is the description of fuzzy control problem: steer the cart from an initial position on the rail to a given final position without taking care of the motion of the pendulum.

If you are in the MATLAB Command Window then write:

`>> fuzzy pd_crane` or click on the black box called 'Edit pd_crane FIS System'.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3 Fuzzy Control

The *FIS Editor Pd_crane* window is opened (Figure 66). From this window you can further inspect and modify the fuzzy crane controller. There are two options: the Matlab editor workspace or using the FIS editor. We recommend to use the FIS option. To access the Matlab editor alternative, type at prompt the following command:

```
>>a=readfis('pd_crane');
```

It results in storing the *pd_crane* fismatrix in the MATLAB Workspace under the variable *a*. If you write:

```
» plotfis(a)
```

then the crane project is shown (given in Figure 68). You can see that *pd_crane* system has 4 inputs, 1 output and 7 rules. Notice that 7 membership functions of the cart position and 9 membership functions of the control force are active. The remaining input variables are neglected. The presence and the right sequence of the input variables corresponding to the RTK task is obligatory. We incorporate an input variable into the project by defining rules corresponding to that variable. Otherwise the variable is left out. In our example we have 7 membership functions of the *position* variable and 7 *If...then* rules each corresponding to one of the 9 membership functions of the *control* variable.

Write:

```
» showfis(a)
```

the following information is given:

- | | | | |
|-------------------|-----------|------------------|-----------------|
| 1. Name | Pd_crane | 9. ImpMethod | min |
| 2. Type | mamdani | 10. AggMethod | max |
| 3. Inputs/Outputs | [4 1] | 11. DefuzzMethod | centroid |
| 4. NumInputMFs | [7 5 5 3] | 12. InLabels | position |
| 5. NumOutputMFs | 9 | 13. | angle |
| 6. NumRules | 7 | 14. | velocity |
| 7. AndMethod | min | 15. | ang_vel |
| 8. OrMethod | max | 16. OutLabels | control |
| | | 17. InRange | [-1 1] |



DIGITAL PENDULUM SYSTEM
Rule-Based And Fuzzy Controllers

CHAPTER 3
Fuzzy Control

- | | | | |
|-----|-------------------|-----|----------------------------|
| 18. | [-3.142 3.142] | 50. | c_m_3 |
| 19. | [-1 1] | 51. | InMFTypes gaussmf |
| 20. | [-5 5] | 52. | gaussmf |
| 21. | OutOfRange [-1 1] | 53. | gaussmf |
| 22. | InMFLabels p_m_1 | 54. | smf |
| 23. | p_c | 55. | zmf |
| 24. | p_p_1 | 56. | gaussmf |
| 25. | p_p_3 | 57. | gaussmf |
| 26. | p_m_3 | 58. | trapmf |
| 27. | p_m_2 | 59. | trapmf |
| 28. | p_p_2 | 60. | trapmf |
| 29. | mf1 | 61. | trapmf |
| 30. | mf2 | 62. | trapmf |
| 31. | mf3 | 63. | trimf |
| 32. | mf4 | 64. | trimf |
| 33. | mf5 | 65. | trimf |
| 34. | v_m_1 | 66. | trapmf |
| 35. | v_center | 67. | trapmf |
| 36. | v_p_1 | 68. | gbellmf |
| 37. | v_m_2 | 69. | gbellmf |
| 38. | v_p_2 | 70. | gbellmf |
| 39. | mf1 | 71. | OutMFTypes trimf |
| 40. | mf2 | 72. | trimf |
| 41. | mf3 | 73. | trimf |
| 42. | OutMFLabels c_p_1 | 74. | trimf |
| 43. | c_m_1 | 75. | trapmf |
| 44. | c_p_2 | 76. | trapmf |
| 45. | c_m_2 | 77. | trimf |
| 46. | c_m_4 | 78. | trimf |
| 47. | c_p_4 | 79. | trimf |
| 48. | c_center | 80. | InMFParams [-0.1 -0.2 0 0] |
| 49. | c_p_3 | 81. | [0.1 0 0 0] |



DIGITAL PENDULUM SYSTEM
Rule-Based And Fuzzy Controllers

CHAPTER 3

Fuzzy Control

82.	[0.1 0.2 0 0]	99.	[2.5 2.5 5 0]
83.	[0.3 0.7 0 0]	100. OutMFPParams	[0 0.25 0.5 0]
84.	[-0.7 -0.3 0 0]	101.	[-0.5 -0.25 0 0]
85.	[-0.06 -0.4 0 0]	102.	[0.25 0.5 0.75 0]
86.	[0.06 0.4 0 0]	103.	[-0.75 -0.5 -0.25 0]
87.	[-4.555 -3.299 -2.985 -1.728]	104.	[-5 -5 -1 -0.75]
88.	[-2.985 -1.728 -1.414 -0.1571]	105.	[0.75 1 5 5]
89.	[-1.414 -0.1571 0.1571 1.414]	106.	[-0.25 0 0.25 0]
90.	[0.1571 1.414 1.728 2.985]	107.	[0.5 0.75 1 0]
91.	[1.728 2.985 3.299 4.555]	108.	[-1 -0.75 -0.5 0]
92.	[-0.5 -0.25 0 0]	109. RuleList	[2 0 0 0 7 1 1]
93.	[-0.25 0 0.25 0]	110.	[3 0 0 0 2 1 1]
94.	[0 0.25 0.5 0]	111.	[1 0 0 0 1 1 1]
95.	[-5 -5 -0.5 -0.25]	112.	[7 0 0 0 9 1 1]
96.	[0.25 0.5 5 5]	113.	[6 0 0 0 8 1 1]
97.	[2.5 2.5 -5 0]	114.	[4 0 0 0 5 1 1]
98.	[2.5 2.5 -5.551e-017 0]	115.	[5 0 0 0 6 1 1]



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3

Fuzzy Control

Thus you obtain the entire description of the *pd_crane* fuzzy system. All names, shapes of membership functions, rules, etc. are visible in the numerical forms. You can trace these data in the *Fis Editor* window (Figure 66). You can [edit](#) membership functions or rules and can [view](#) rules or the surface of the controller.

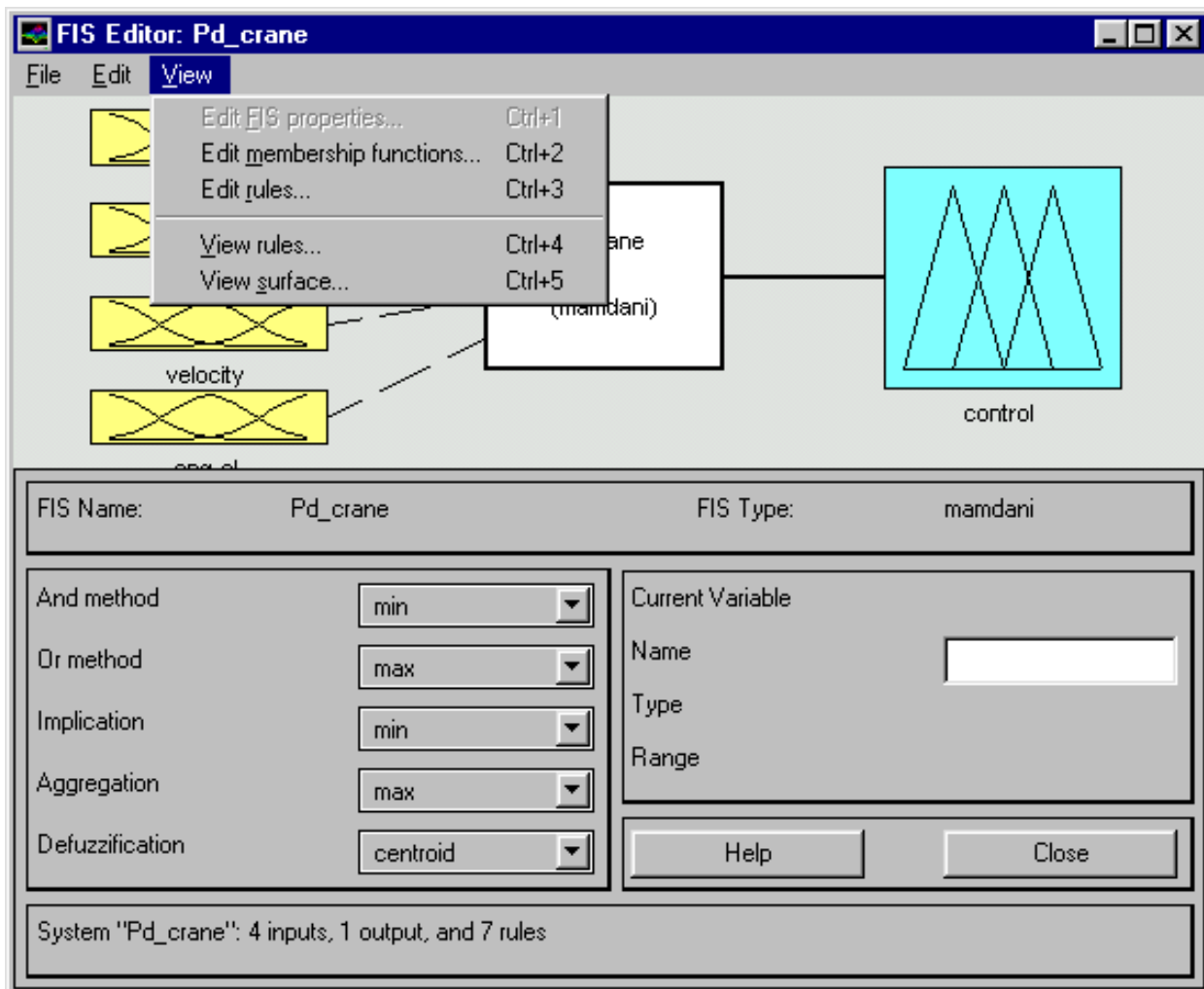


Figure 66: The *Fis Editor: Pd_crane* window

The membership functions for cart position(one of the inputs) and control(output) are presented in Figure 67.



DIGITAL PENDULUM SYSTEM

Rule-Based And Fuzzy Controllers

CHAPTER 3

Fuzzy Control

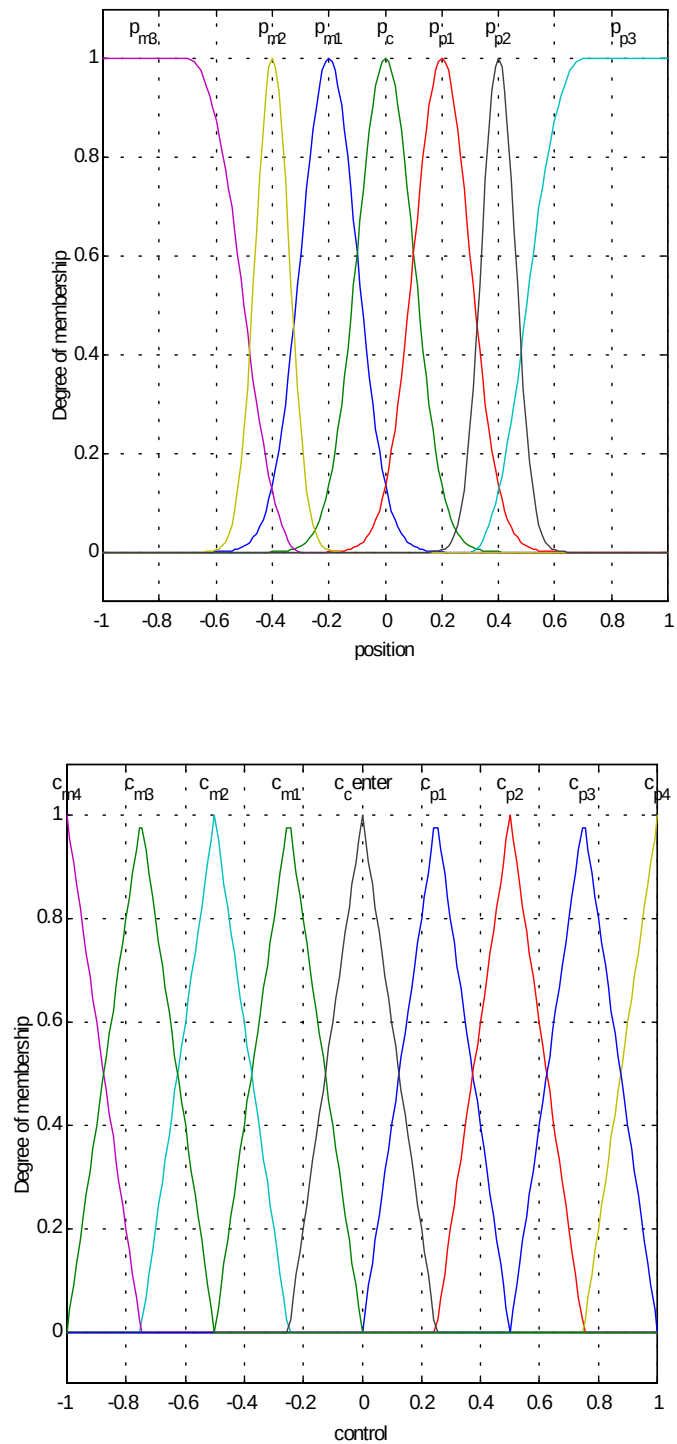
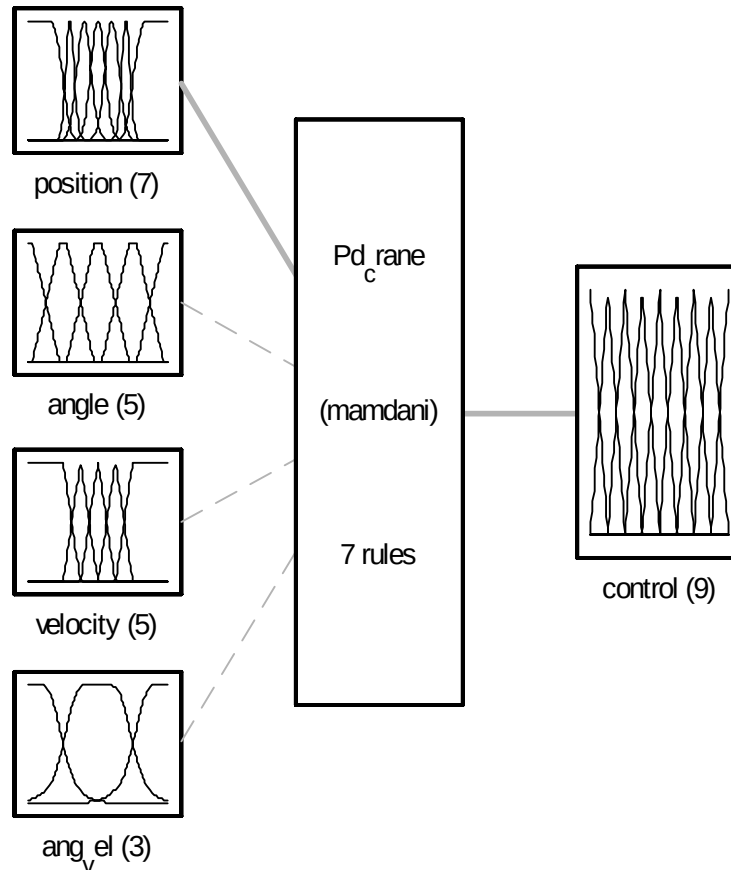


Figure 67: Membership functions



System $Pd_{c \text{ rane}}$: 4 inputs, 1 outputs, 7 rules

Figure 68: Crane project

Parameter classes are set to be evenly distributed in the universe of discourse. The rules are presented in Table 4. The *If...then* inference mechanism is well visible in the verbose form. The index form of rules is an equivalent form of description. The index (1) in the first rule means that the weight of this rule is set equal to 1. The index form: 2 0 0 0, 7 (1) : 1 can be explained as follows: the 2nd input membership function for the first input variable **and** none “0” input membership function for the second input variable **and** none “0” input membership function for the third input variable **and** none “0” input membership function for the fourth input variable implies 7th output membership function for the output variable. The weight of this rule is set to one (1). The “and” logic quantifier is defined as “:1”.



Verbose form

1. If (position is p_c) then (control is c_center)
(1)
2. If (position is p_p_1) then (control is c_m_1)
(1)
3. If (position is p_m_1) then (control is c_p_1)
(1)
4. If (position is p_p_2) then (control is c_m_3)
(1)
5. If (position is p_m_2) then (control is c_p_3)
(1)
6. If (position is p_p_3) then (control is c_m_4)
(1)
7. If (position is p_m_3) then (control is c_p_4)
(1)

Index form

2 0 0 0, 7 (1) : 1
 3 0 0 0, 2 (1) : 1
 1 0 0 0, 1 (1) : 1
 7 0 0 0, 9 (1) : 1
 6 0 0 0, 8 (1) : 1
 4 0 0 0, 5 (1) : 1
 5 0 0 0, 6 (1) : 1

Table 4 Rules for the fuzzy crane controller

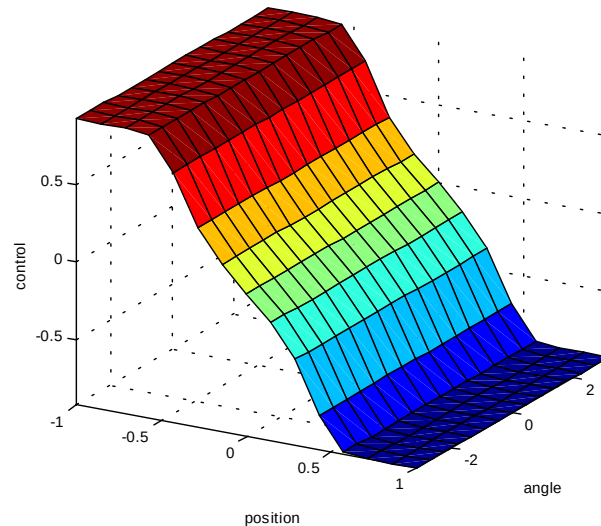


Figure 69: Fuzzy surface

The *control* vs. *position* and *angle* surface is given in Figure 69. To obtain this use FIS Editor window for *pd_crane* and select View-View Surface.

3.1.2 SIMULATION EXAMPLE

This section demonstrate how the cart-pendulum system can be used as a crane model. Our fuzzy controller steers only the cart ignoring the pendulum motion.

Open the simulation model from the main fuzzy *pd_fuzzy.mdl* window (see Figure 58) or double click on the *Simulink Simulation* button. The model depicted in Figure 59 appears.

Remember to write:

```
>> fismat=readfis('pd_crane');
```

in the *MATLAB Command Window*. It loads the fuzzy interference matrix into workspace. Check the desired position signal and initial values of the state variables. We must use the *Reduction of angle* block (see Figure 70) in a new form. The state variables related to the cart have the negative sign to meet the features of the system modelling. Compare the *Linear cart controller* and *Linear pendulum controller* masks given in Figure 14.

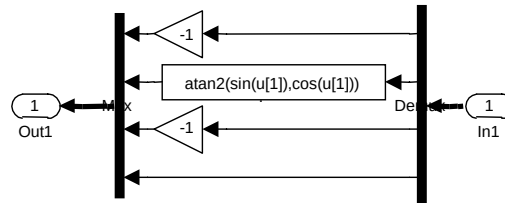


Figure 70: The *Reduction of angle* block

The crane mode simulation results of the are given in Figure 71, Figure 72, Figure 73, and Figure 74. You can notice that the cart tracks the desired position square-wave signal while the pendulum motion is not controlled. The ± 0.1 chattering of the control signal is well visible in Figure 74. Assuming the *Friction* parameter value equal to 0.1 causes this effect. The control is trying to overcome the static friction of the cart by adding 0.1 to a positive control value and subtracting 0.1 from a negative control value.

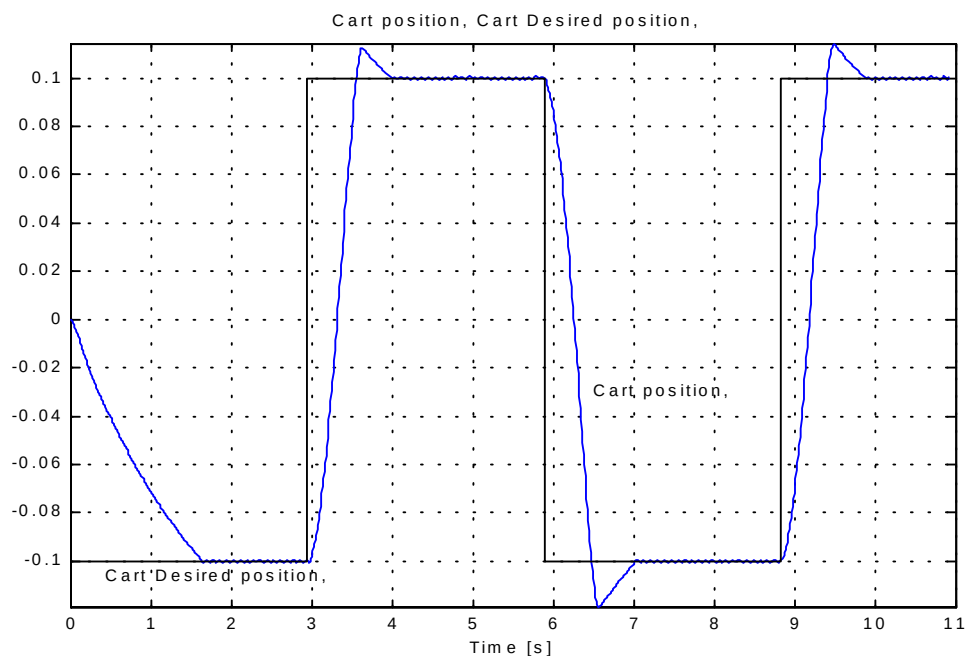


Figure 71: Simulation, time diagram of the cart position

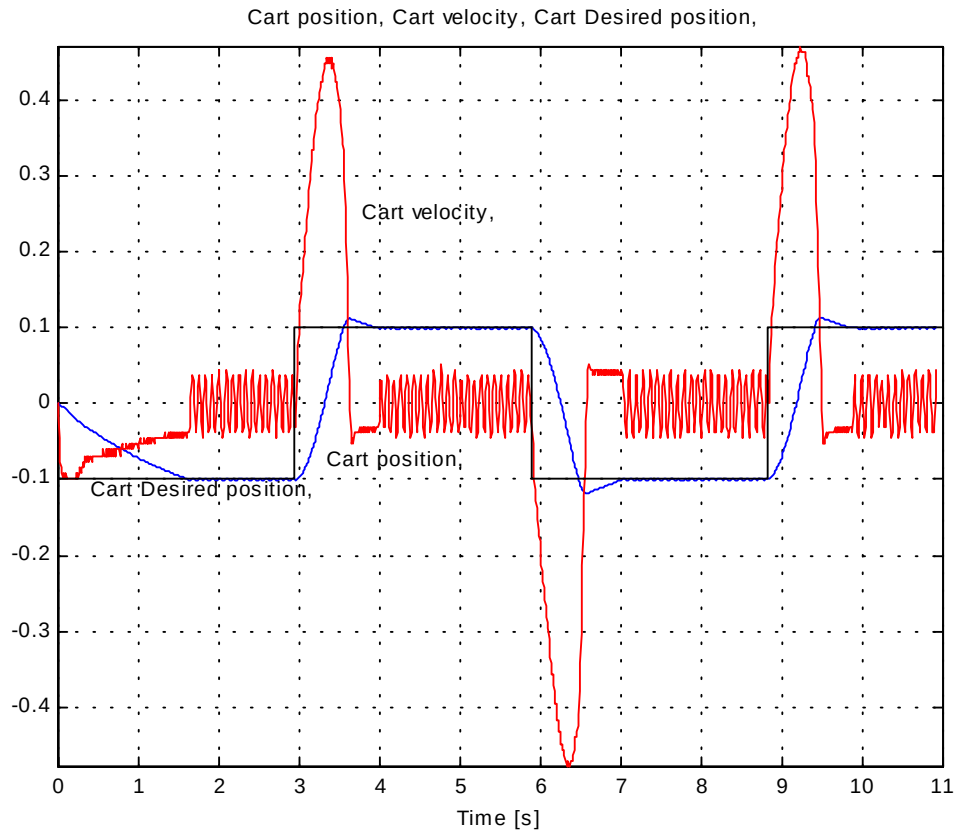


Figure 72: Simulation, time diagram of the cart position and velocity

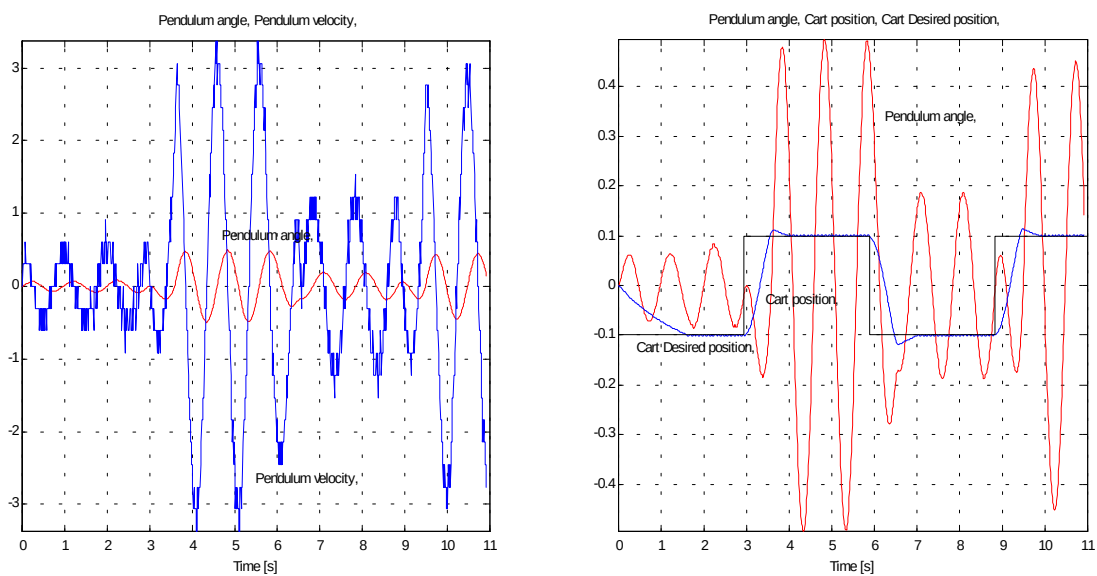


Figure 73: Simulation, time diagram of the pendulum variables

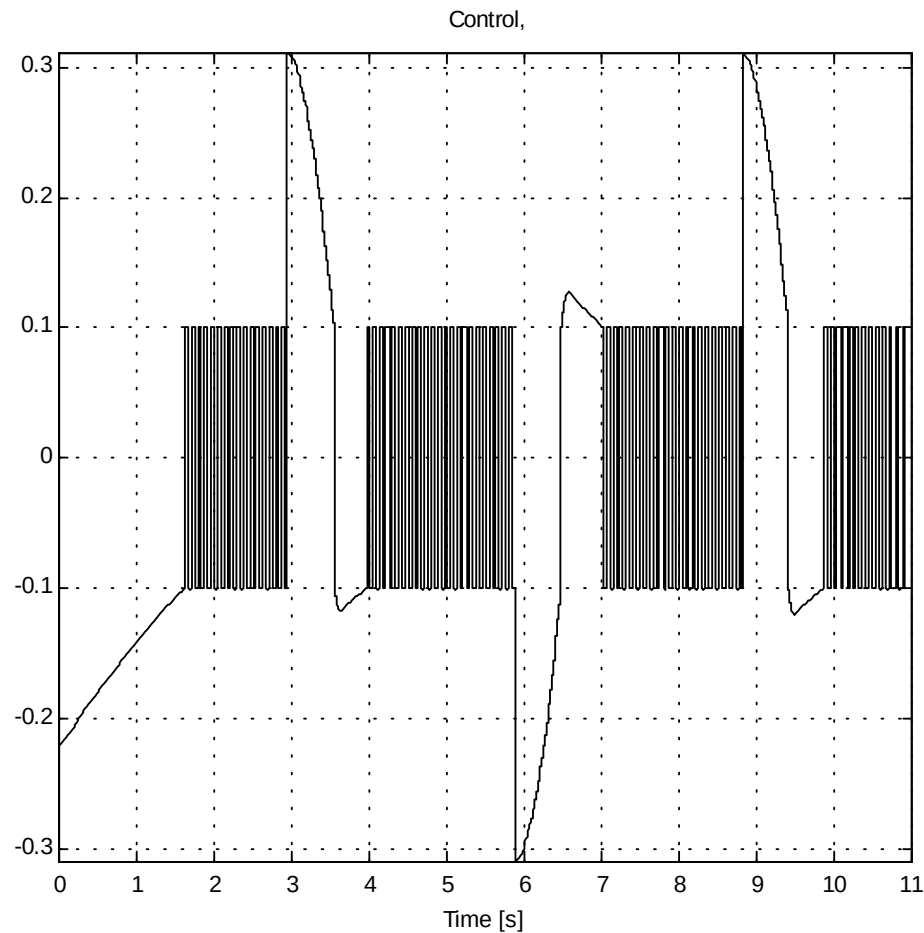


Figure 74: Simulation, time diagram of the control

3.1.3 REAL – TIME EXAMPLE

This experiment concerns the real-time control of the cart. To achieve this open the main menu of Fuzzy Toolbox for pendulum(*pd_fuzzy*) if not already open, see Figure 58. Double click on the '*Real-time Experiment*' button under Crane FIS Controller.

The Window shown in Figure 65 opens. The experiment can be started from here. This file is also open to parameter variations by accessing the green block called Real-Time task.

Time responses of the real system under the *pd_crane* fuzzy controller algorithm are presented in Figure 75.

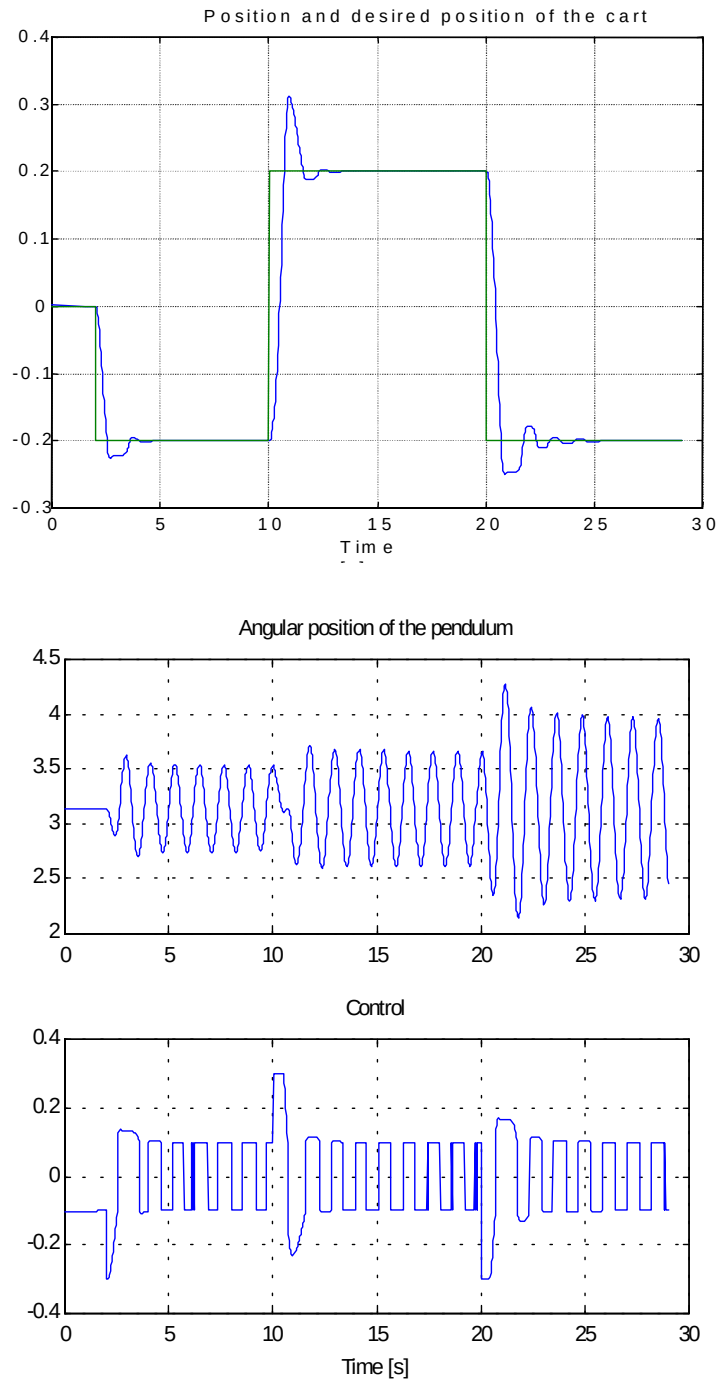


Figure 75: Control data stored from the real system



The fuzzy controller is tracking the square wave signal of the desired cart position. The angular position of the pendulum is not controlled. We start with the pendulum hanging down (π - angle value), then the pendulum starts to swing because of the cart inertia.

The magnitude of the pendulum oscillations changes rapidly when an abrupt change of the cart position occurs. It is well visible in the second diagram of Figure 75. If the error of the cart position signal is positive/negative but has a small nearly equal to 0 value then the control signal, visible in the third diagram, is negative/positive with a value nearly equal to $-0.1/+0.1$ but less/greater than that value. To move the cart we must apply a force greater in magnitude than the static friction. If you double click the *Real Time Task* block in the *pd_fuzzy_crane_rt* window then the parameter mask opens. You can see that the *Friction* parameter is 0.1. If the static friction effect is neglected by setting *Friction* to 0, then the desired cart position signal will be tracked with a static error. Poor tracking occurs also for a *Friction* parameter value much different from the static friction of the real system.

Notice the difference between the simulation and practical control signals. The simulated control shows a high frequency chattering which does not occur in reality.

3.2 STABILISING MODE

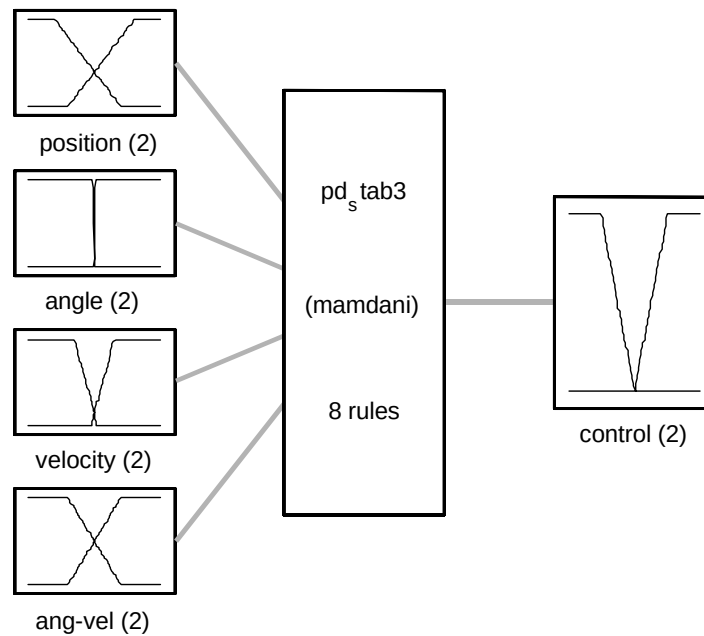
3.2.1 FUZZY SET

The stabilising control problem refers to the situation when the pendulum is in its unstable upright position and the cart is tracking a desired signal (remember to start the system with the pendulum in its upright position). In the *Rule-based controller* section a linear proportional controller is applied as a stabilisation controller. In this section we apply a fuzzy non-linear controller.

If you are in the MATLAB Command Window then write:

```
>>fuzzy pd_stab
```

The *Fis Editor Pd_stab* window opens. The same result you will achieve when double clicking the 'Edit *pd_stab FIS System*' button in the main *pd_fuzzy* window. This time all four state variable inputs are used. We obtain fuzzy variable by constructing the minimum set of membership functions: two membership functions for each variable. The same applies also to the output control variable. The eight rules are sufficient to achieve stabilisation. The stabilisation project is shown in Figure 76.



System $pd_s \text{ tab3}$: 4 inputs, 1 outputs, 8 rules

Figure 76: Stabilisation project

The membership functions are presented in Figure 77.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3

Fuzzy Control

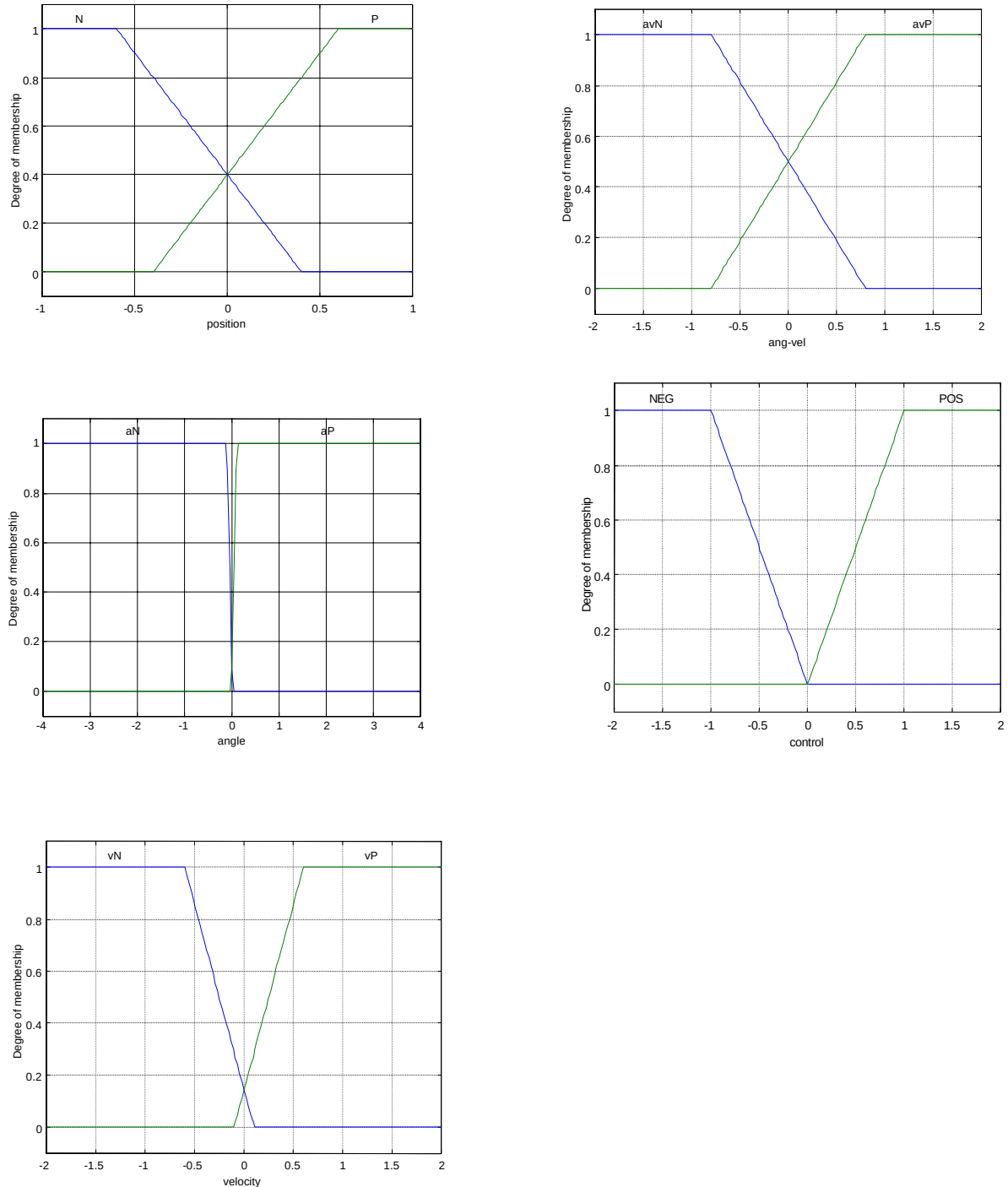


Figure 77: Membership functions

Finally the fuzzy surfaces are plotted in Figure 78. Such a choice of a fuzzy stabilising controller is quite arbitrary. Comment: it works in a wide range of parameter changes.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3 Fuzzy Control

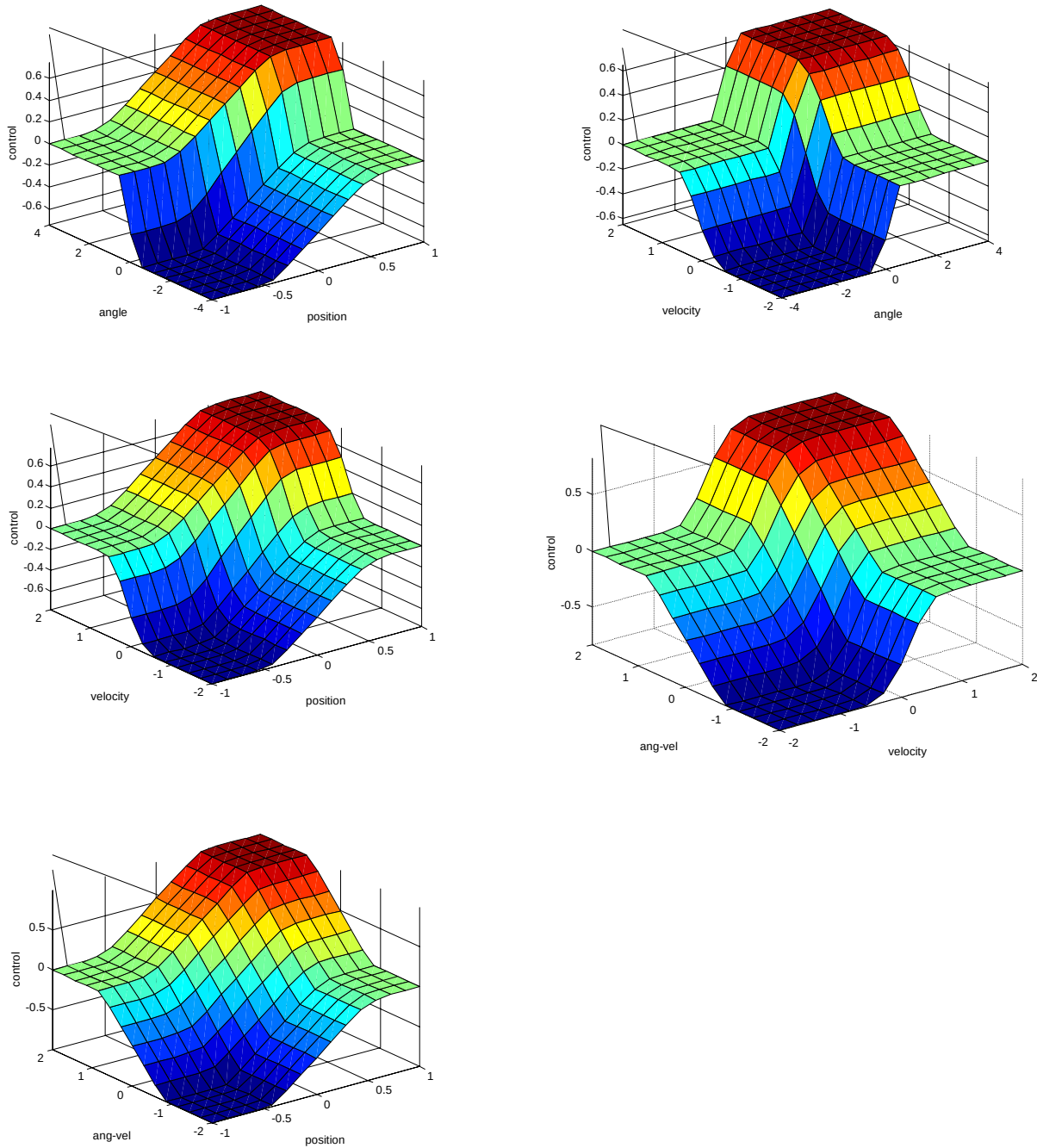


Figure 78: Fuzzy surfaces

The rules are given in Table 5.



1. If (position is N) then (control is NEG) (1)
2. If (position is P) then (control is POS) (1)
3. If (angle is aN) then (control is NEG) (1)
4. If (angle is aP) then (control is POS) (1)
5. If (velocity is vN) then (control is NEG) (1)
6. If (velocity is vP) then (control is POS) (1)
7. If (ang-vel is avN) then (control is NEG) (1)
8. If (ang-vel is avP) then (control is POS) (1)

Table 5: Rules of the fuzzy stabilising controller

Of course, you can build a more complex controller. When doing it pay attention to the following features:

- Is a controller robust to disturbances and parameter changes?
- Does it stabilise more efficiently (faster, in a wider range etc.) than the linear controller?

3.2.2 SIMULATION EXAMPLE 1

In example 1 we demonstrate how the cart-pendulum system is stabilised when the desired cart position is set to zero and the pendulum is up. Open the simulation model from the main fuzzy *pd_fuzzy.mdl* (see Figure 58) – double click on the *Simulink Simulation* button situated under Stabilising FIS Controller title. The model depicted in Figure 79 appears.

Remember to write:

```
>> fismat=readfis('pd_stab');
```

in the *Matlab Command Window*. Check the desired position signal is set to zero and the initial values of the state variables. The state variables related to the cart have the negative sign to meet features of the system modelling (see Figure 70).



The time diagram of simulation of the pendulum angle is given in Figure 80.

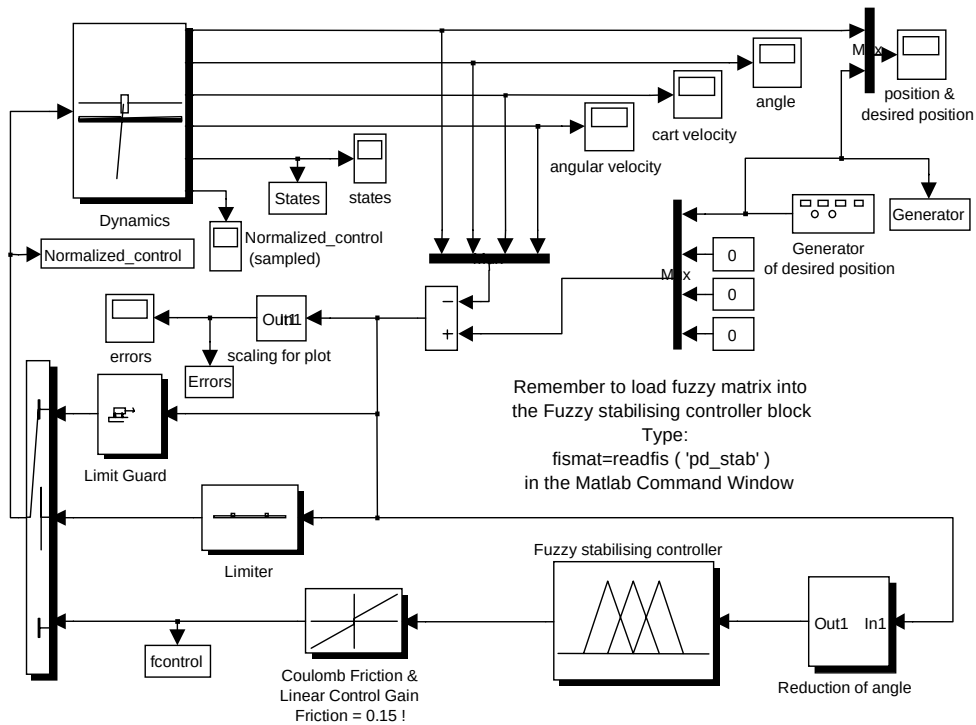


Figure 79: Fuzzy simulation model *pd_mstab.mdl*

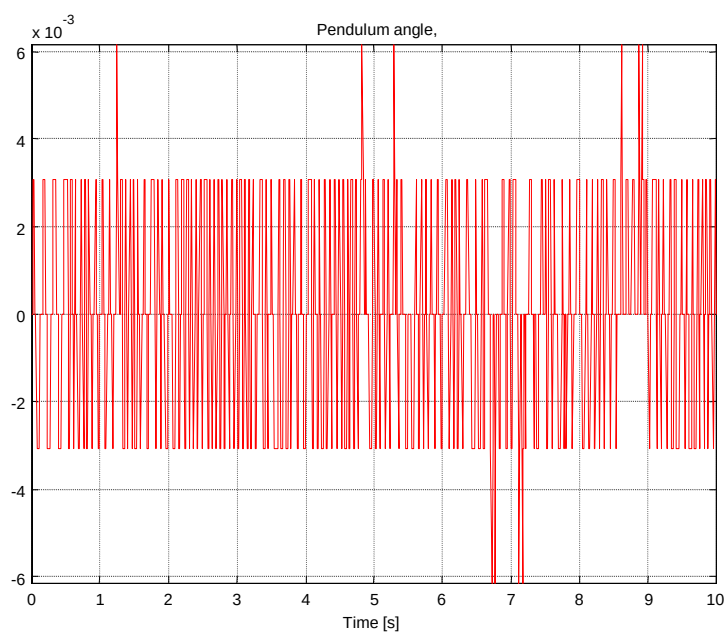


Figure 80: Simulated pendulum angle



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3 Fuzzy Control

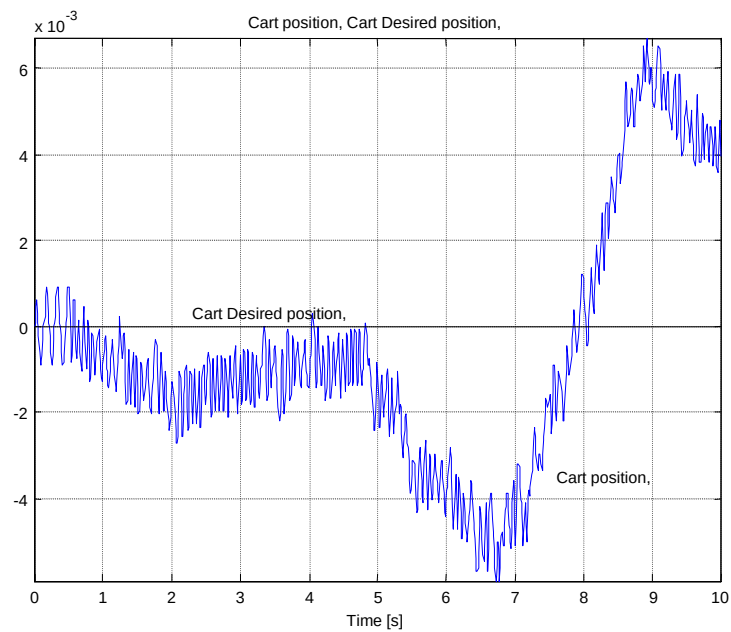


Figure 81: Simulated cart position and cart desired position

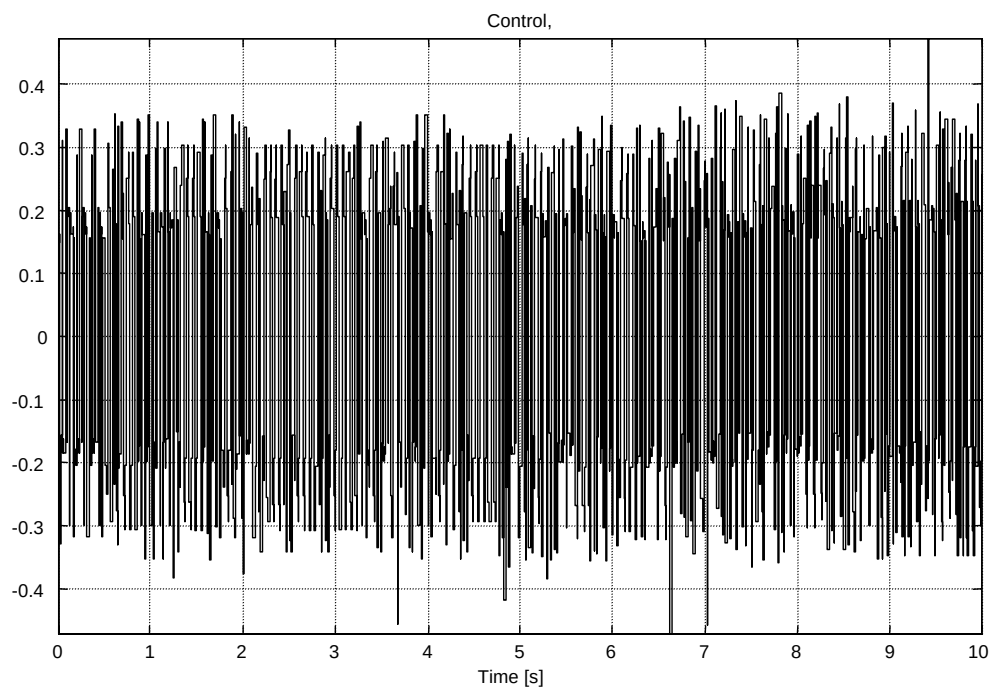
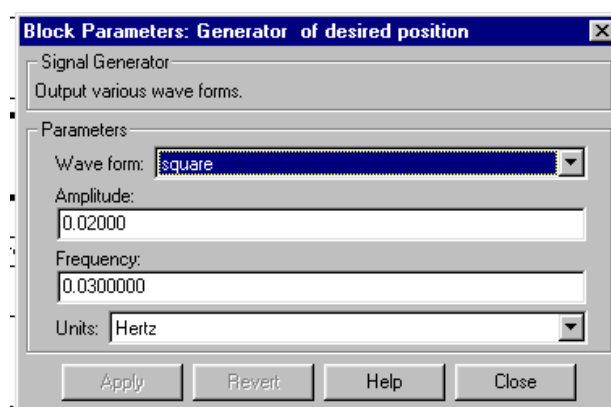


Figure 82: Simulated stabilising control



3.2.3 SIMULATION EXAMPLE 2

Example 2 shows how the cart-pendulum system is stabilised when the desired cart position is a square wave signal. To perform this use the same file as in previous example, but set the Generator of desired position as show below.



The time diagrams of the simulated stabilising positions are given in Figure 83.

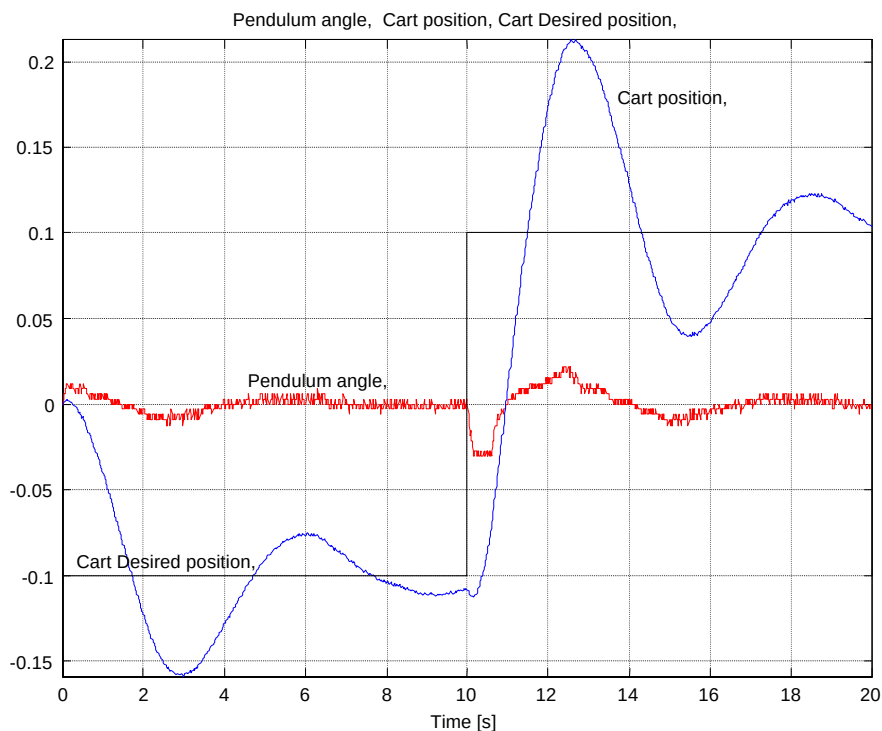


Figure 83: Simulated positions of the system. The cart is excited by a square wave signal.



The simulated control is given in Figure 84. The cart tracks the squared wave signal. The biggest deviations are observed after the switching from -0.1m to 0.1m . The magnitude of the control, usually limited to ± 0.3 , achieves ± 0.5 at switching moments.

From these two examples we can observe that the fuzzy-logic controller is not always the best option to control when comparing the above performances to the ones given by the PID or LQ control.

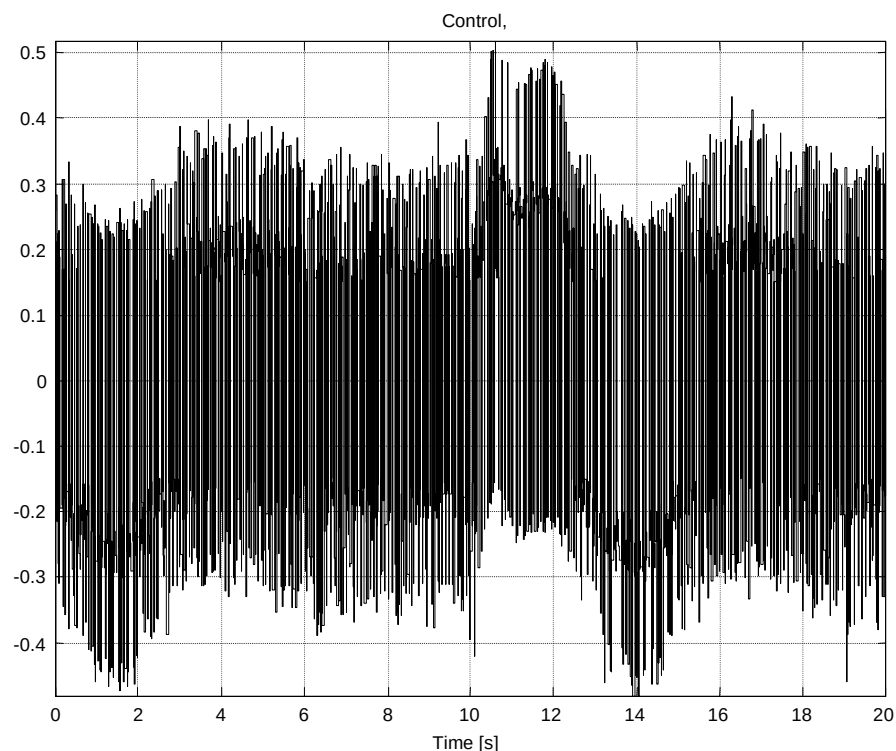


Figure 84: Simulated control. The cart is excited by the square wave signal.

3.2.4 REAL-TIME: EXAMPLE 1

Stabilisation policy can be verified with the real system. If you double click the *Real Time Experiment* button in the main *pd_fuzzy* window, the following window opens (see Figure 85).

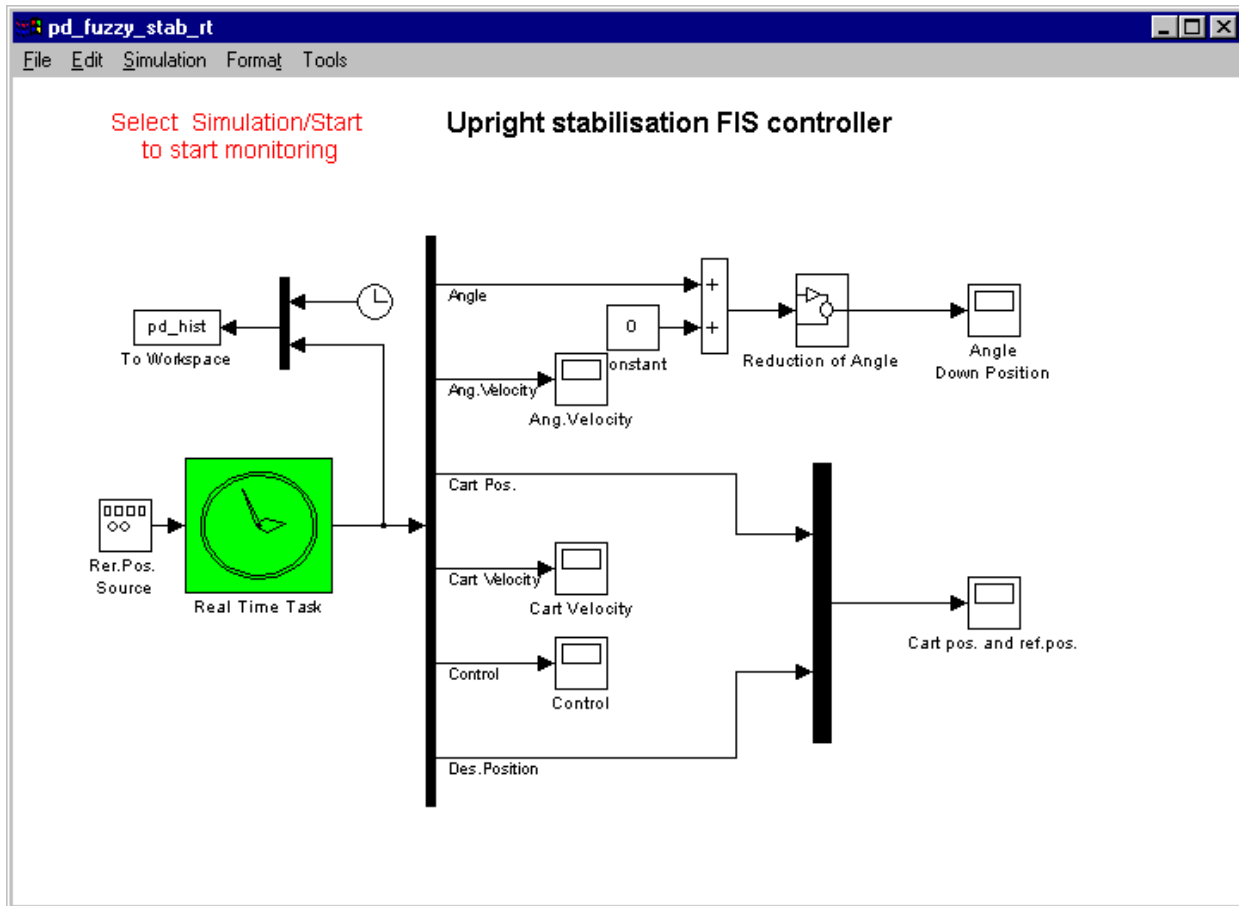


Figure 85: The *stabilisation* controller window

Click the *Real Time Task* block to open the mask (see Figure 86). In this particular case we would like to draw the attention to *Friction* and *Alpha offset* parameters values. *Friction*=0.15 value guarantees damping the cart position to achieve the desired value. *Alpha offset* equal 0 means that the initial angle is set to zero in RTK. In fact the same RTK is used for the crane and the upright stabilisation problem. They differ only in the *Alpha offset* parameter setting. The crane require *Alpha offset* equal to π .

Remember to start the practical holding the pendulum in the upright position by hand.

The real time diagram of the pendulum angle is given in Figure 87. The cart position and the cart desired position are shown in Figure 88. The control is presented in Figure 89.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3 Fuzzy Control

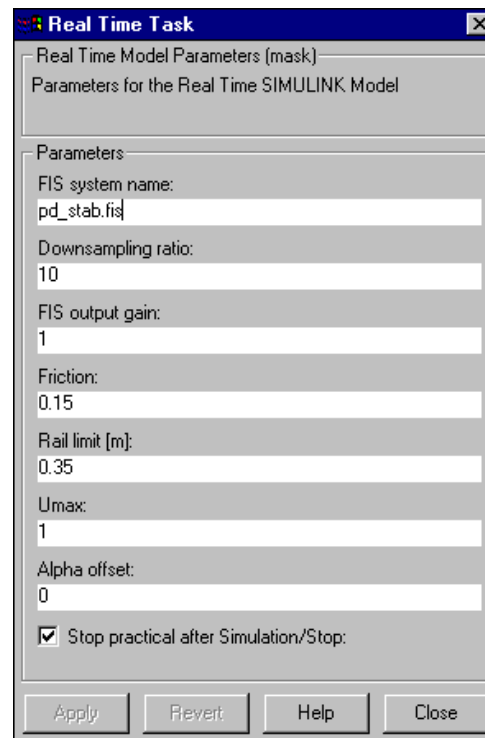


Figure 86: The *Real Time Task* mask

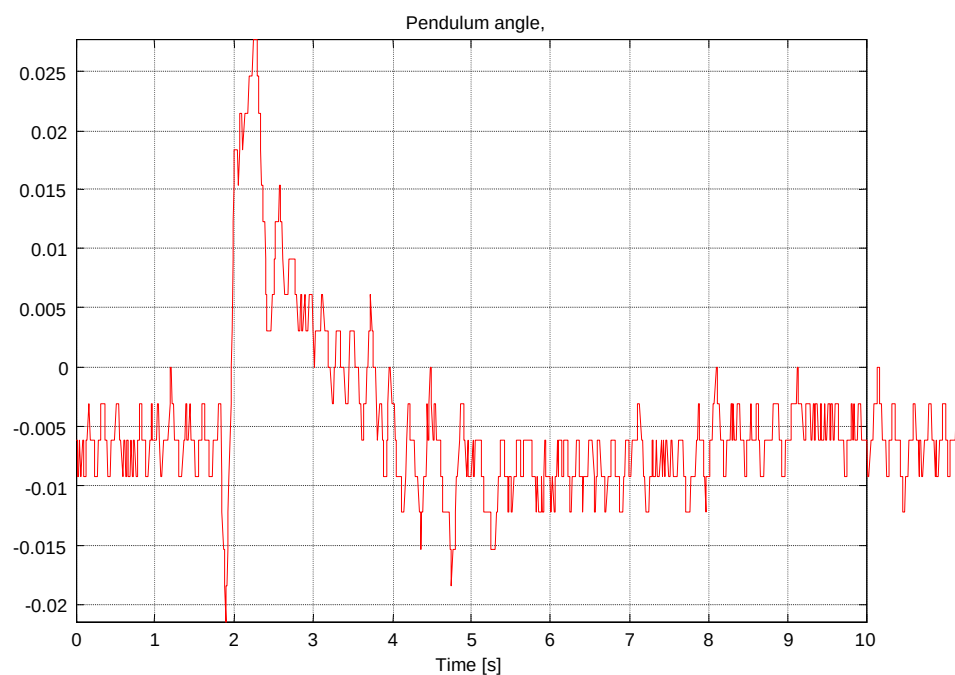


Figure 87: Angle position. Real-time experiment.



Figure 88: Cart position. Real-time experiment.

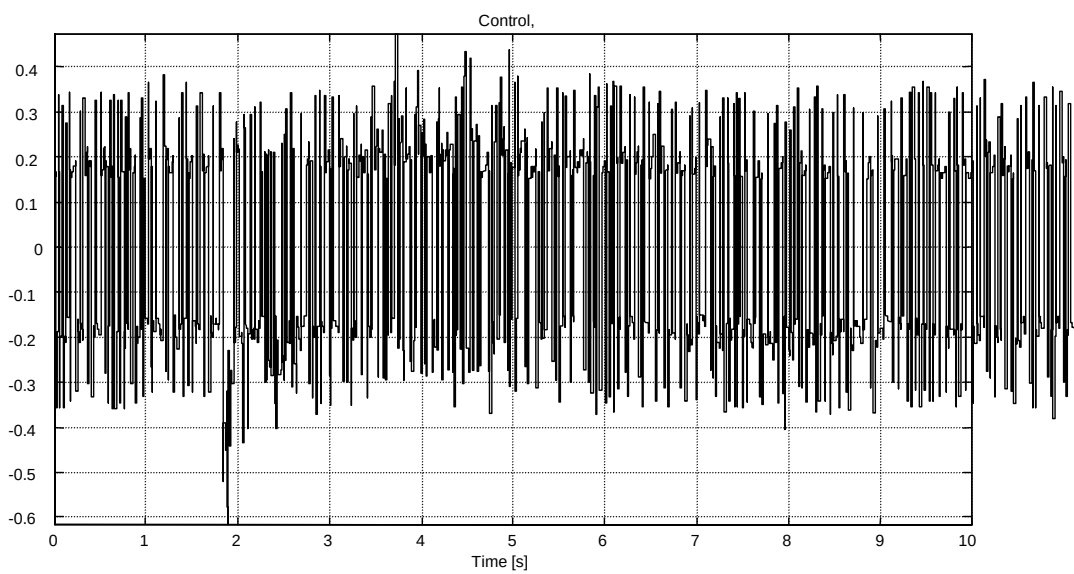


Figure 89: Control. Real-time experiment.



The similarity to the simulation model responses can be seen.

3.2.5 REAL-TIME EXAMPLE 2

In the second example we activate the *Generator of Desired Position* block by setting the amplitude of Reference source higher than zero following a square wave shape(Figure 90). The time diagrams of positions of the real experiment are given in Figure 91. The control is given in Figure 92.

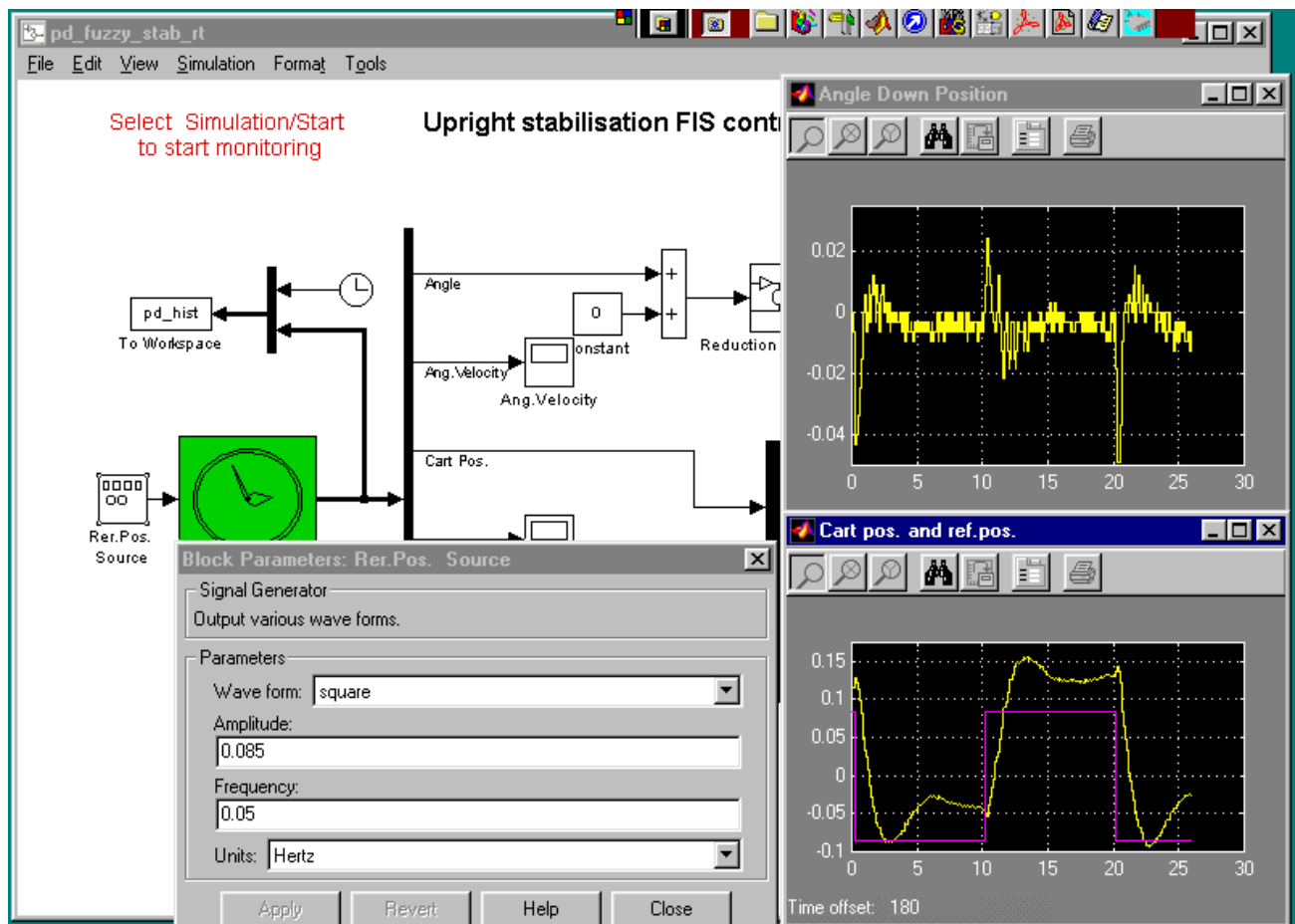


Figure 90

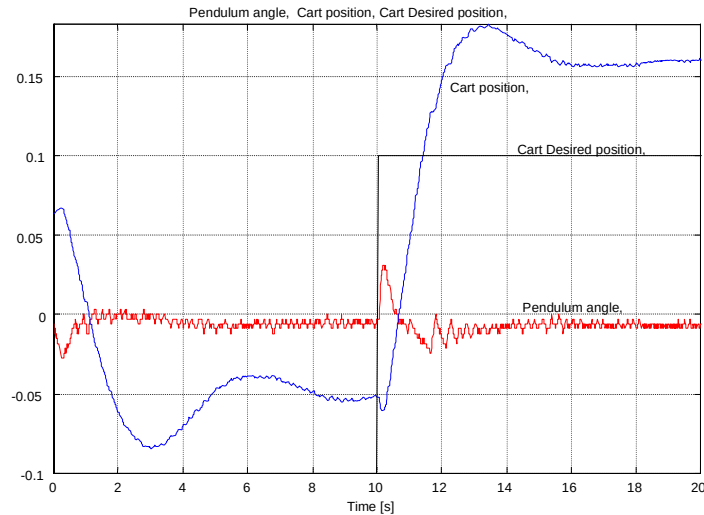


Figure 91: Real-time experiment: positions. The cart is excited by the square wave signal.

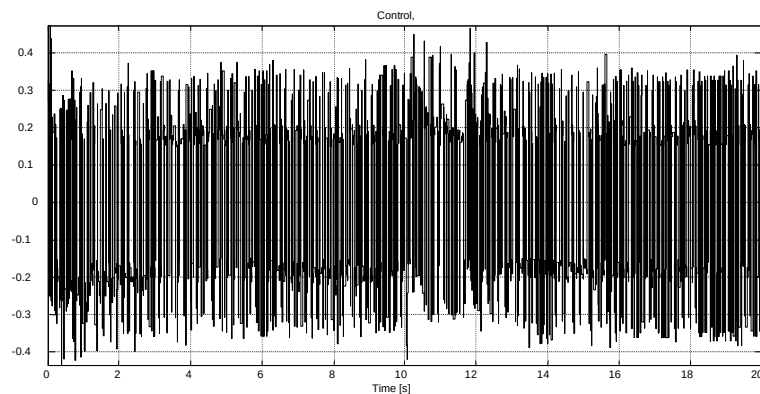


Figure 92: Real-time experiment: control. The cart is excited by the square wave signal.

The pendulum angle, cart position, cart desired position and control are shown in Figure 93. The zoomed time interval is from 9.8 to 11 seconds. In the zoomed data you can detect the inference mechanism of the fuzzy controller. Note the influence of the state variables on control at the switching moment.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3

Fuzzy Control

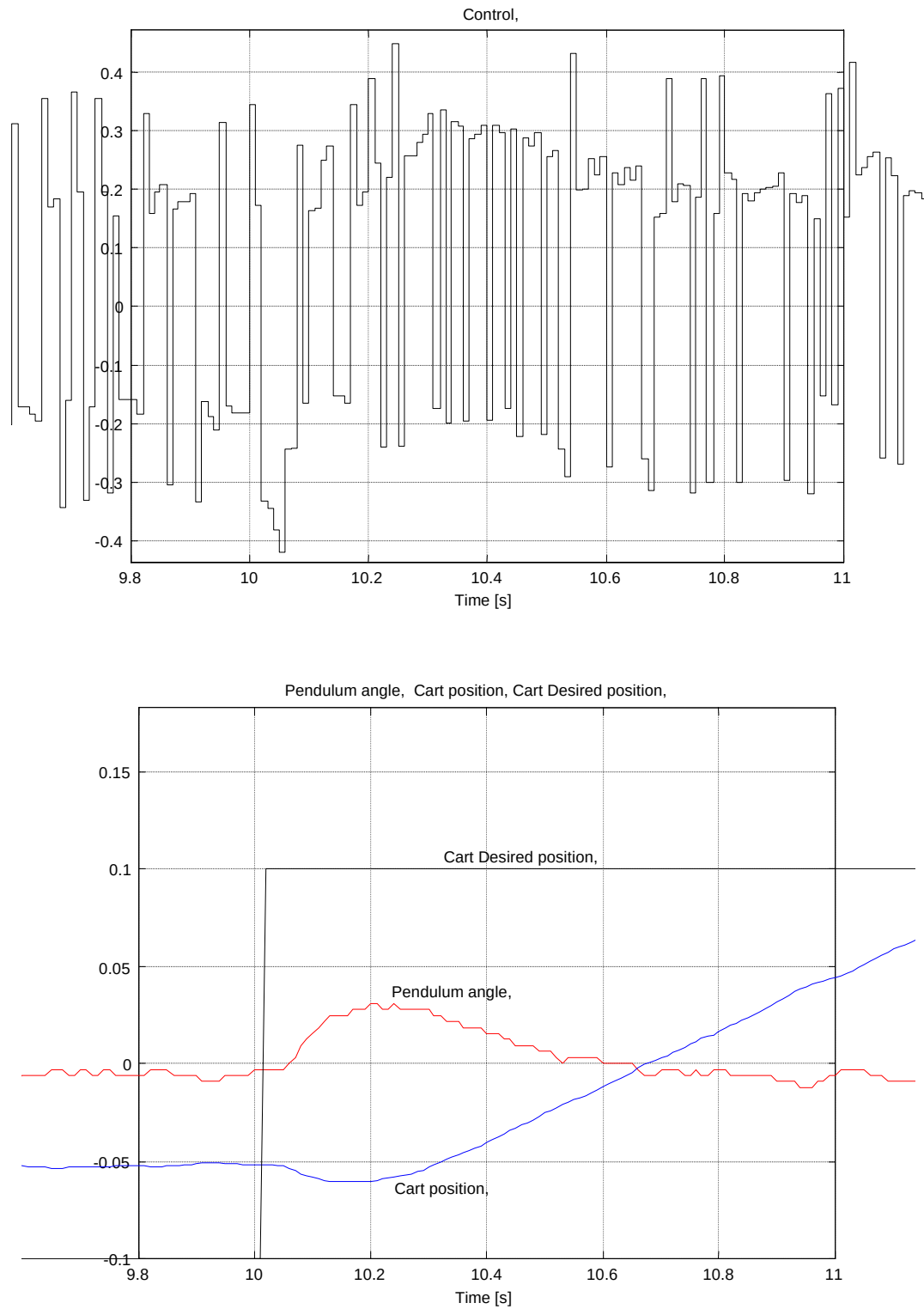


Figure 93: Zoom at the switching moment.



3.3 SWING-UP MODE

3.3.1 FUZZY SET

In the description of this demo we focused mainly on control aspects of pendulum-cart system. If reader is interested in editorial options of the MATLAB Fuzzy Toolbox would refer to sections: 3.1 and 3.2. Next is explained how we build a swing-up fuzzy controller equipped with a cart centring policy. In the RTK swing task, the rule-based control algorithms not concern directly with *Swinging & centring controller* is left out. You can easily recognise which part of the old rule-based controller (see Figure 2) has been replaced by the *Fuzzy swing controller* (see Figure 102). Ideally we could replace all rule-based controllers by a number of corresponding fuzzy controllers? For example, combine the stabilising controller from section 3.2 with a new built swing-up controller and other rule-based controllers designed in the fuzzy technique? It can be done in simulation.

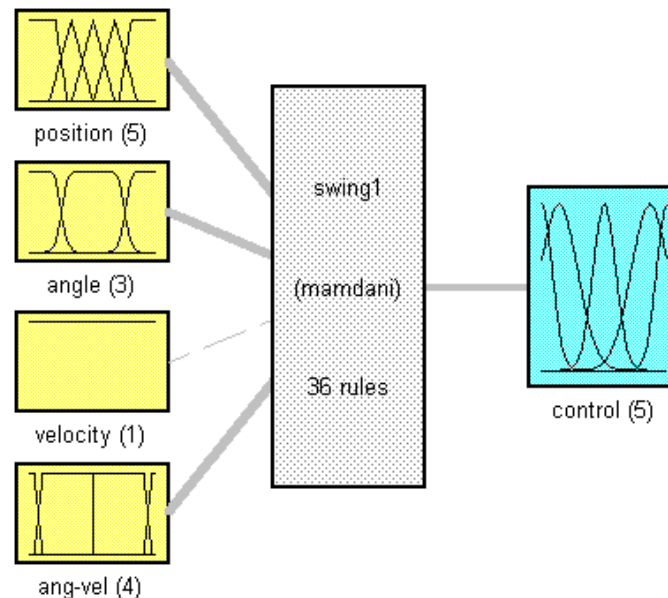
Unfortunately in the real time there are restrictions given by the structure of RTK. We can define a number of fis-matrices but we can embed only one fis-matrix in RTK. Being so restricted we have to fix the number of input and output fuzzy variables and also define the sequence of consecutive variables. The following input/output structure is determined: 4 inputs and 1 output. The sequence and definition of input variables are: cart position, pendulum angle, cart velocity, pendulum angular velocity. The definition of output variable is: force applied to the cart. Of course when building your own controller you do not need to use all input variables. Membership functions for each variable could be choose differently by each designer. You can create: single *if...then*, double *if...and....then*, triple *if...and...and...then* or quadruple *if...and...and...and...then* rules. You can use also the “or” logic operator instead of the “and” operator. This is the flexibility given by the fuzzy -logic.

Another question is: can one build a general fuzzy controller having the above described structure and replacing the entire rule-based functionality? The answer seems to be positive. Nevertheless, the problem is quite complex. We leave it as an open question for a creative user.

Form the MATLAB Command Window go to subdirectory where the *.fis-files are stored. Then write:

```
>> fuzzy pd_swing
```

The *FIS Editor Pd_swing* window opens. From this window you can go further to watch and modify the fuzzy swing controller. The fuzzy project: *swing_up* is depicted in Figure 94.



System swing1: 4 inputs, 1 outputs, 36 rules

Figure 94: Fuzzy project: *swing_up*

There are three active input fuzzy variables: *position*, *angle* and *ang-vel*. The third input variable – *velocity* is deactivated. To build a swing-up algorithm it is enough to relate to the pendulum variables: *angle* and *ang-vel*. The *position* variable is added to introduce a dependency on cart position: a cart centring mechanism. For a fuzzy project we have to acquire knowledge how to swing the pendulum. We can proceed in many ways. First, we must collect data related to successful real-time swing-up experiment or build an accurate reference simulation model of a swinging algorithm (as was presented in section 2.1). Next, define the universe of discourse for each variable. Then, define membership functions and fuzzy rules. Usually we define membership functions having in mind rules corresponding to them.

Now let us begin with defining membership functions for the swing-up controller. The angle and angular velocity membership functions are presented in Figure 95 and Figure 96. We build them using the swing-up formula (see Table 3 Rule-based control algorithm)

$$u =: \text{sign}[x_4 \cdot (|x_2| - \pi/2)] \quad \text{no.8}$$

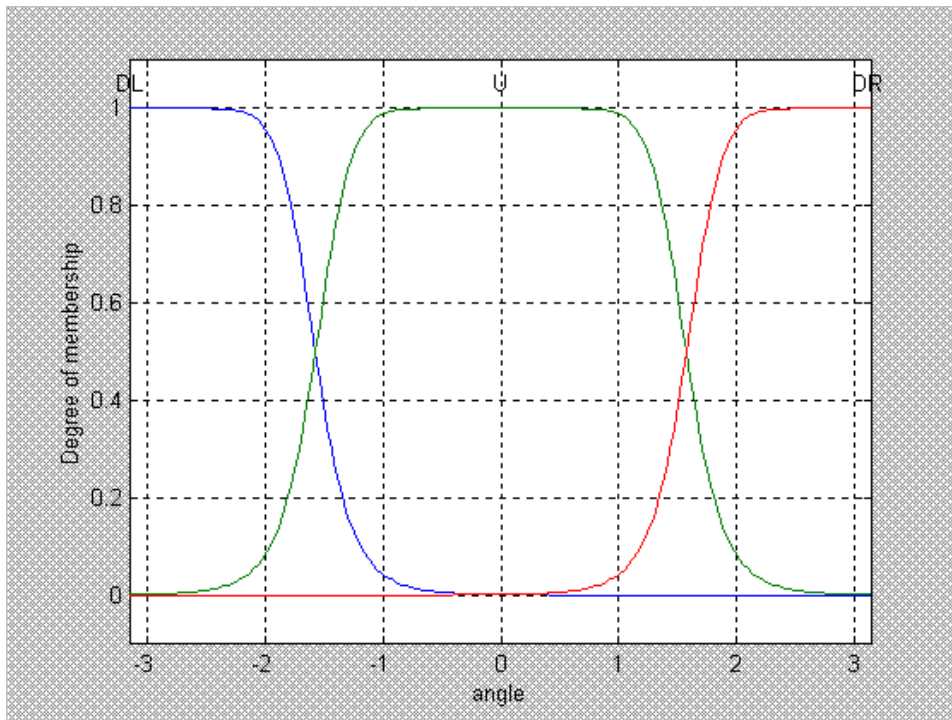


Figure 95: Angle membership functions

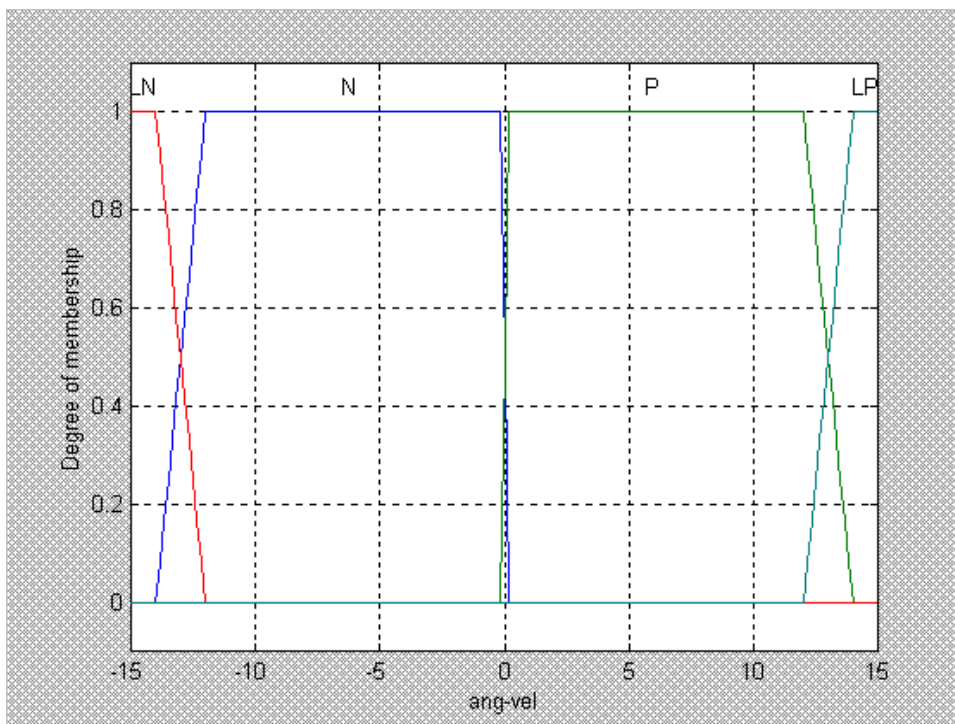


Figure 96: Angular velocity membership functions



There are three membership functions of *angle*: DL – down left, U – up and DR – down right. They overlap each other. The characteristic points in the universe of discourse are $+\pi/2$ and $-\pi/2$. At these points the overlapping functions have the membership value equal to 0.5.

The *ang-vel* fuzzy variable has four membership functions: large negative (LN), negative (N), positive (P) and large positive (LP). N and P are responsible for swinging-up. LN and LP are connected with the soft landing of the pendulum.

The *control* fuzzy variable has five membership functions: negative (N), medium negative (MN), zero (Z), medium positive (MP) and positive (P). They are distributed as shown in Figure 97

Finally we have five membership functions for the *position* variable (see Figure 98): negative (N), medium negative (MN), zero (Z), medium positive (MP) and positive (P). They are evenly distributed. They are used to design the cart centring rules.

An efficient swinging-up control is of the bang-bang type and the membership functions for *angle* and *ang-vel* variables should have such shapes so that they would yield a good approximation of the sharp switching curves.

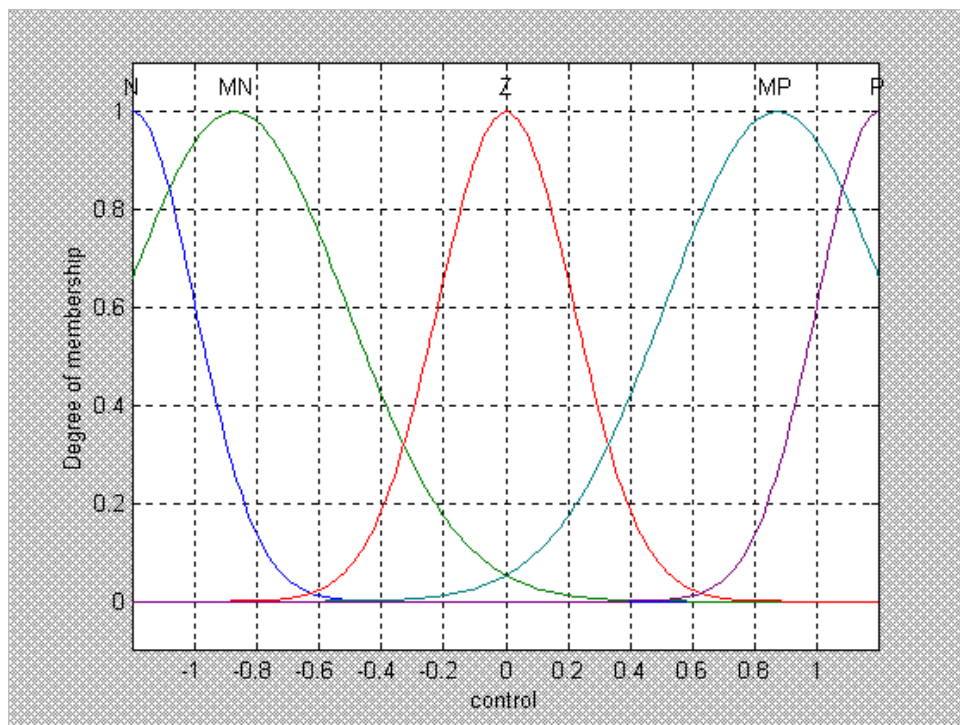


Figure 97: Control membership functions

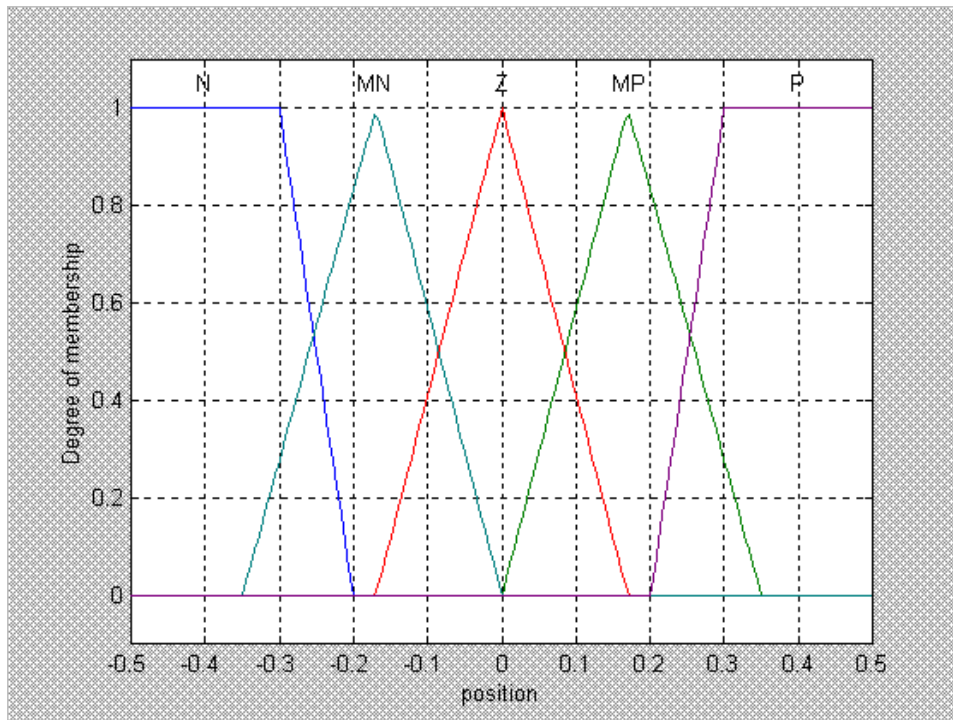


Figure 98: Position membership functions

The swing rules are given in Table 6. The rules no.1 through 12 prevent crossing the rail limits. The rules no.13 through 24 are responsible for swing-up and centring actions of the system. The rules no.25 through 30 generate purely swing-up algorithm. The rules no.31 through 36 correspond to the soft landing algorithm.

Summarising, when creating a fuzzy inference system one must put attention to: appropriate choice of universe of discourse, overlapping of membership functions, shapes of membership functions, covering all possible combinations of input variables by rules: minimum one rule must be fired in every situation.

The options: *View rules* and *view surface* in the pull-down menu of *Edit* of the FIS editor window (see Figure 66) help to design and modify the controller. Watching which rules are fired when travelling through the universe of discourse in the *view rules* editor we can interactively change the controller behaviour. Similarly the *view surface* tool gives a graphical image of control vs. input variables. Then it is easier to judge if the designed fuzzy controller would respond properly. Three surfaces of our controller are presented in Figure 99, Figure 100 and Figure 101.



1. If (position is N) and (angle is DL) and (ang-vel is N) then (control is P) (1)
2. If (position is N) and (angle is U) and (ang-vel is N) then (control is P) (1)
3. If (position is N) and (angle is DR) and (ang-vel is N) then (control is P) (1)
4. If (position is N) and (angle is DL) and (ang-vel is P) then (control is P) (1)
5. If (position is N) and (angle is U) and (ang-vel is P) then (control is P) (1)
6. If (position is N) and (angle is DR) and (ang-vel is P) then (control is P) (1)
7. If (position is P) and (angle is DL) and (ang-vel is N) then (control is N) (1)
8. If (position is P) and (angle is U) and (ang-vel is N) then (control is N) (1)
9. If (position is P) and (angle is DR) and (ang-vel is N) then (control is N) (1)
10. If (position is P) and (angle is DL) and (ang-vel is P) then (control is N) (1)
11. If (position is P) and (angle is U) and (ang-vel is P) then (control is N) (1)
12. If (position is P) and (angle is DR) and (ang-vel is P) then (control is N) (1)
13. If (position is MN) and (angle is DL) and (ang-vel is P) then (control is MN) (1)
14. If (position is MN) and (angle is DR) and (ang-vel is P) then (control is MN) (1)
15. If (position is MN) and (angle is DL) and (ang-vel is N) then (control is P) (1)
16. If (position is MN) and (angle is DR) and (ang-vel is N) then (control is P) (1)
17. If (position is MN) and (angle is U) and (ang-vel is N) then (control is MN) (1)
18. If (position is MN) and (angle is U) and (ang-vel is P) then (control is P) (1)
19. If (position is MP) and (angle is DL) and (ang-vel is P) then (control is N) (1)
20. If (position is MP) and (angle is DR) and (ang-vel is P) then (control is N) (1)
21. If (position is MP) and (angle is DL) and (ang-vel is N) then (control is MP) (1)
22. If (position is MP) and (angle is DR) and (ang-vel is N) then (control is MP) (1)
23. If (position is MP) and (angle is U) and (ang-vel is N) then (control is N) (1)
24. If (position is MP) and (angle is U) and (ang-vel is P) then (control is MP) (1)
25. If (position is Z) and (angle is DL) and (ang-vel is P) then (control is N) (1)
26. If (position is Z) and (angle is DR) and (ang-vel is P) then (control is N) (1)
27. If (position is Z) and (angle is DL) and (ang-vel is N) then (control is P) (1)
28. If (position is Z) and (angle is DR) and (ang-vel is N) then (control is P) (1)
29. If (position is Z) and (angle is U) and (ang-vel is N) then (control is N) (1)
30. If (position is Z) and (angle is U) and (ang-vel is P) then (control is P) (1)
31. If (angle is U) and (ang-vel is LN) then (control is MP) (1)
32. If (angle is U) and (ang-vel is LP) then (control is MN) (1)
33. If (angle is DL) and (ang-vel is LP) then (control is Z) (1)
34. If (angle is DR) and (ang-vel is LP) then (control is Z) (1)
35. If (angle is DL) and (ang-vel is LN) then (control is Z) (1)
36. If (angle is DR) and (ang-vel is LN) then (control is Z) (1)

Table 6 Rules of the fuzzy swing-up controller

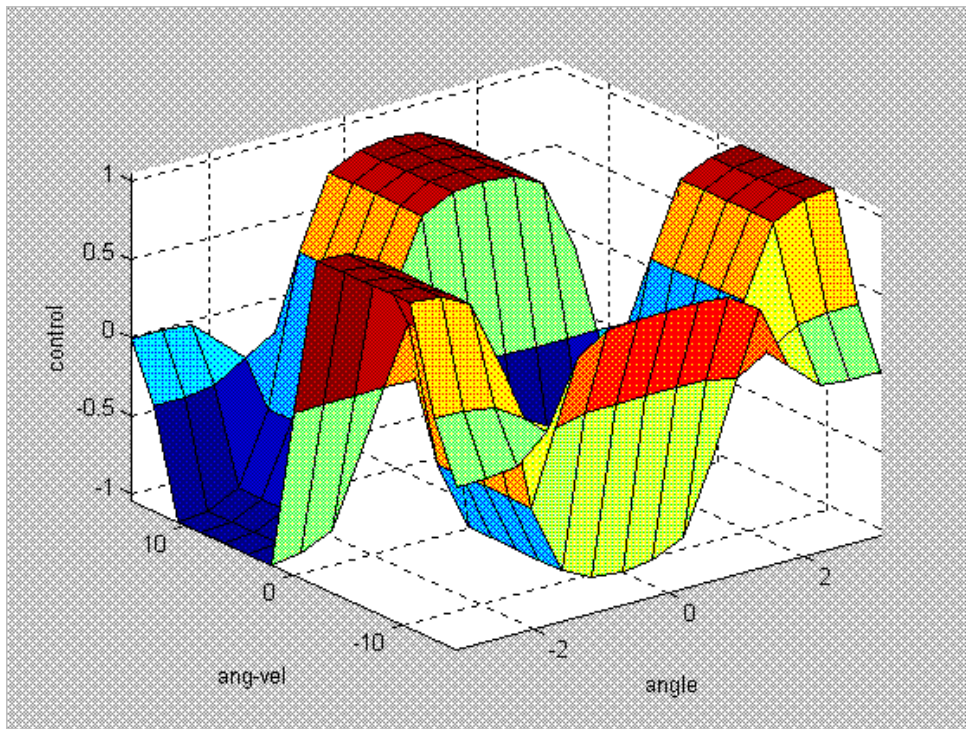


Figure 99: Angular velocity v.s. angle – fuzzy surface

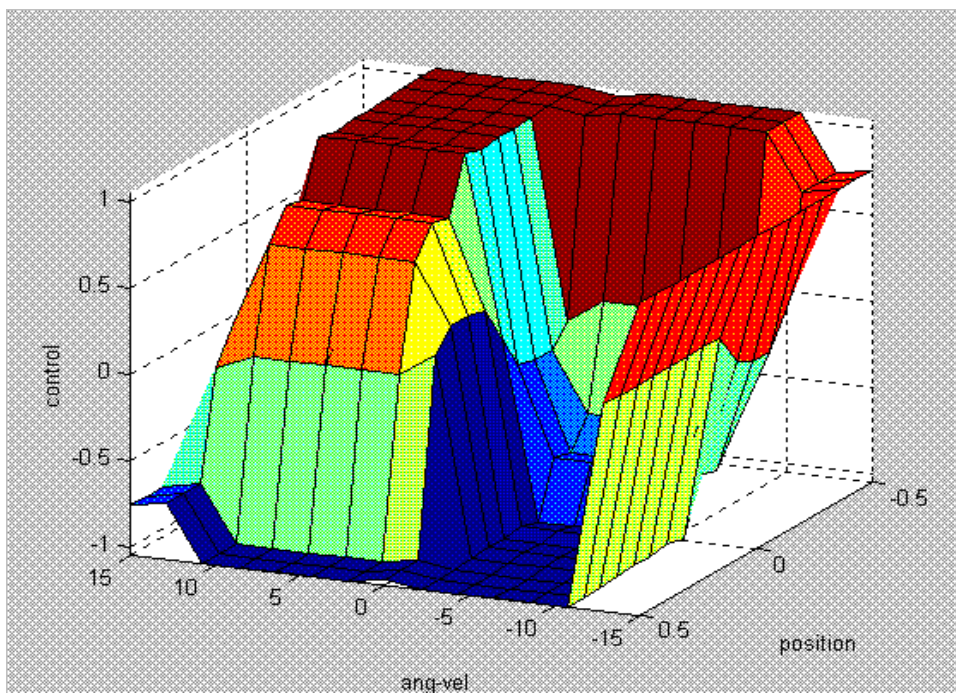


Figure 100: Angular velocity v.s. position – fuzzy surface

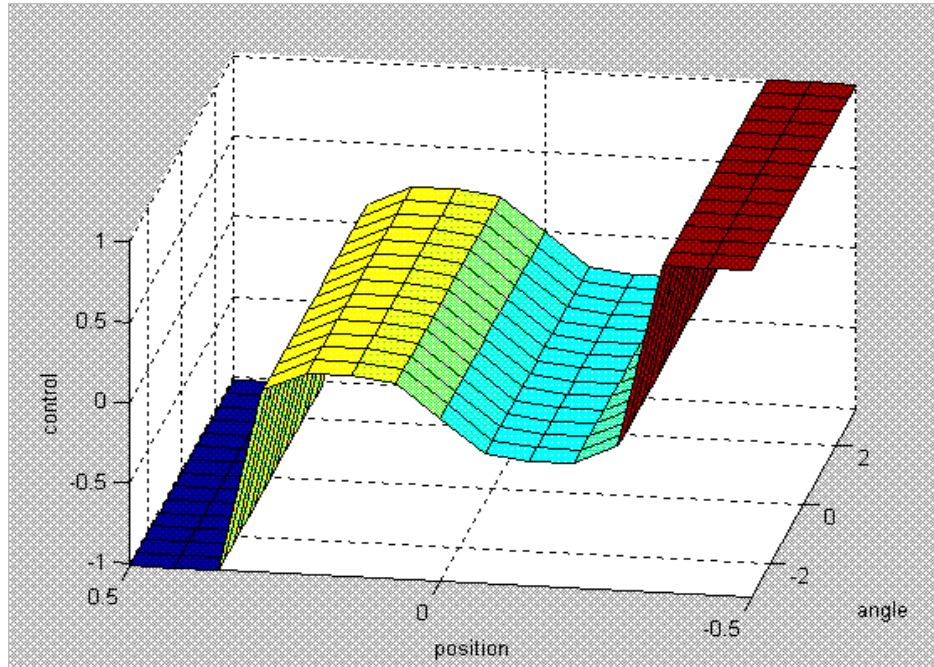
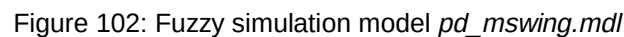


Figure 101: Position v.s. angle – fuzzy surface

3.3.2 SIMULATION EXAMPLE

To perform the swing-up simulation, double click the *Simulink Simulation* block in the main fuzzy window (see Figure 58). The following model appears (Figure 102).



When we set *Swinging Control Gain* equal to 0.14, which means that we use only 14% of the swinging control force the swinging algorithm “pumps” energy into the pendulum very slowly. It takes about 20 s to achieve the stabilisation zone. In the mean time the cart does not travel to far from the middle of the rail. The maximum deviation is less than 20 cm.



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3 Fuzzy Control

The simulation results related to the cart, pendulum and control are presented in Figure 106, Figure 107 and Figure 108 respectively

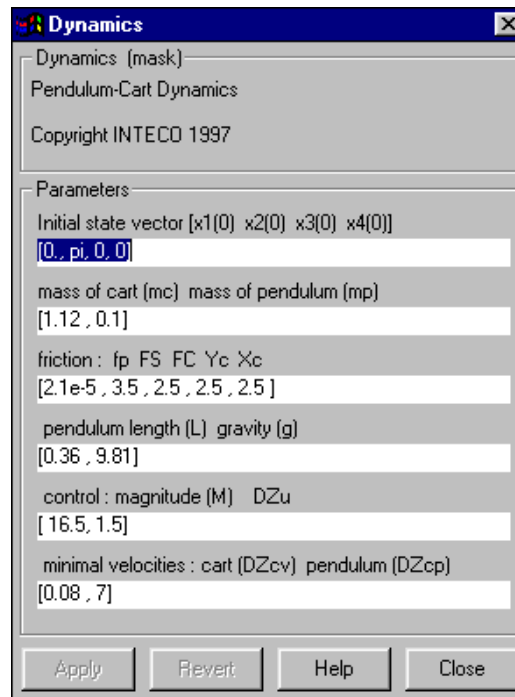


Figure 103: Mask of the *Dynamics* block

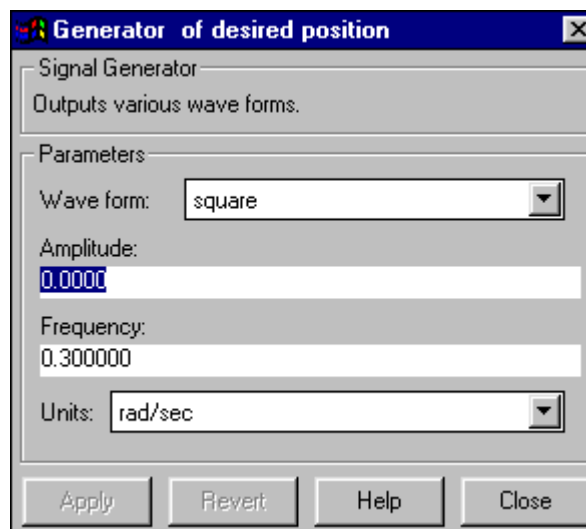


Figure 104: Mask of the *Generator of desired position* block

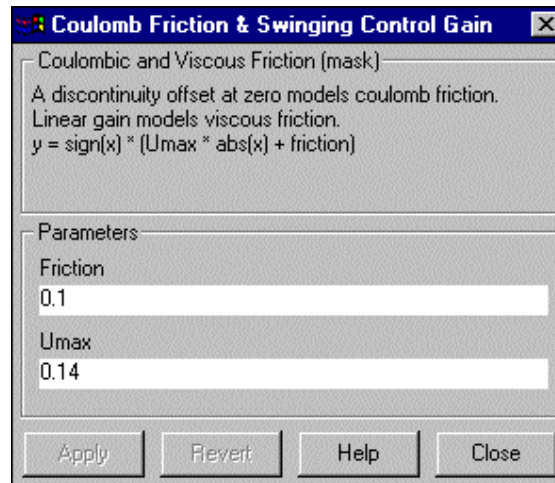


Figure 105: Mask of the *Coulomb Friction & Swinging Control Gain* block

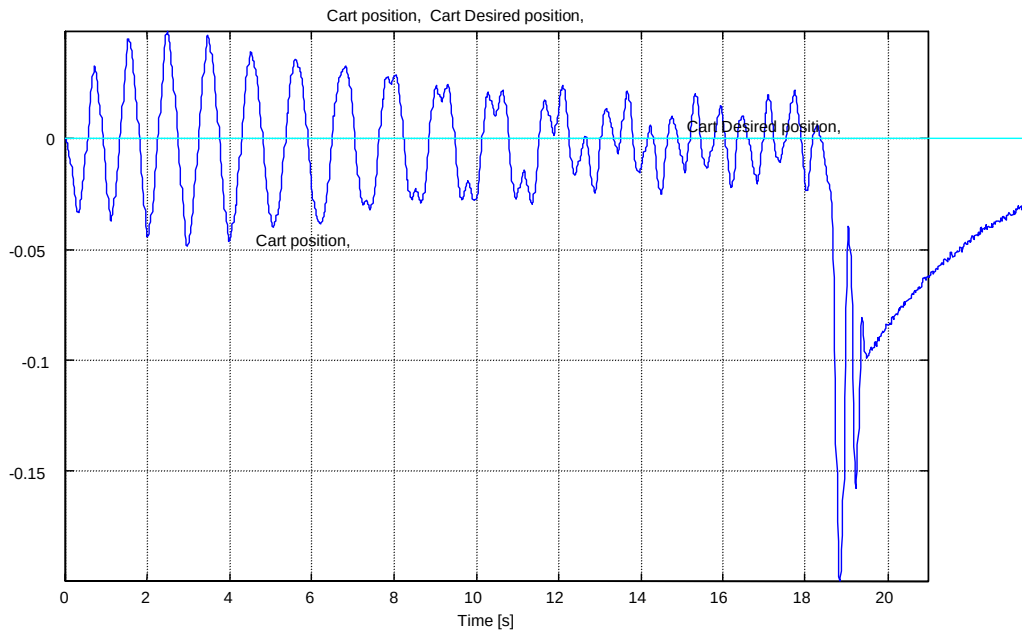


Figure 106: Time diagram of the cart position



DIGITAL PENDULUM SYSTEM Rule-Based And Fuzzy Controllers

CHAPTER 3 Fuzzy Control

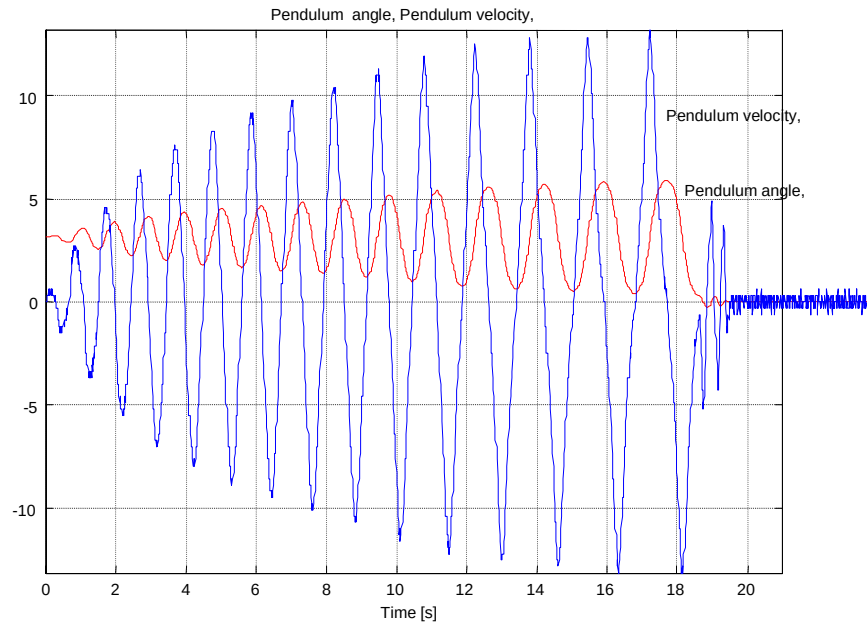


Figure 107: Time diagram of the pendulum angle and velocity

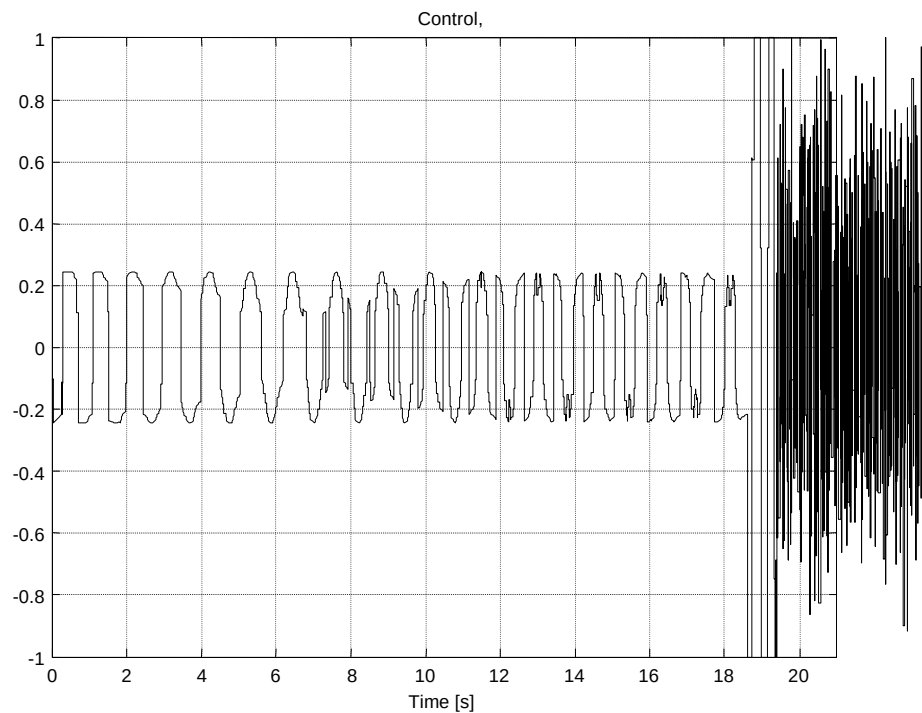


Figure 108: Time diagram of the Control



The control magnitude (see Figure 108) is less than 0.25. We can simply calculate the numerical value of the control magnitude: 0.1 which is the *Friction* value plus the *Swinging Gain* $U_{max}=0.14$ gives 0.24. The control does not preserve the “bang-bang” character.

3.3.3 REAL-TIME EXAMPLE

It will be interesting to compare the simulation results to the ones obtained in the real model. Open the Simulink monitor model from the main fuzzy *pd_fuzzy.mdl* window (see Figure 58) – double click on the *Real-time experiment* button. The window for the *Swing-up* controller (see Figure 109) opens. From here you can start the real-time experiment.

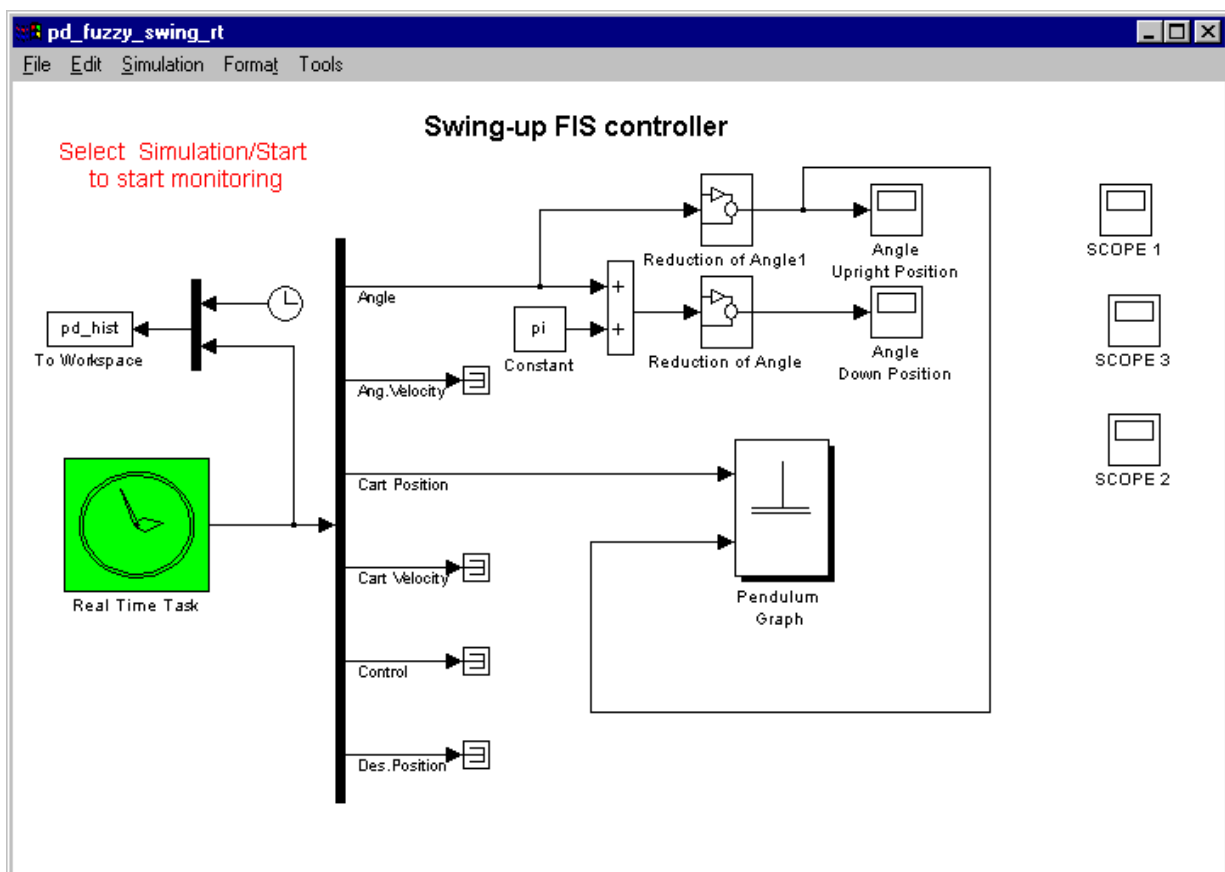


Figure 109: The *Swing-up* controller window

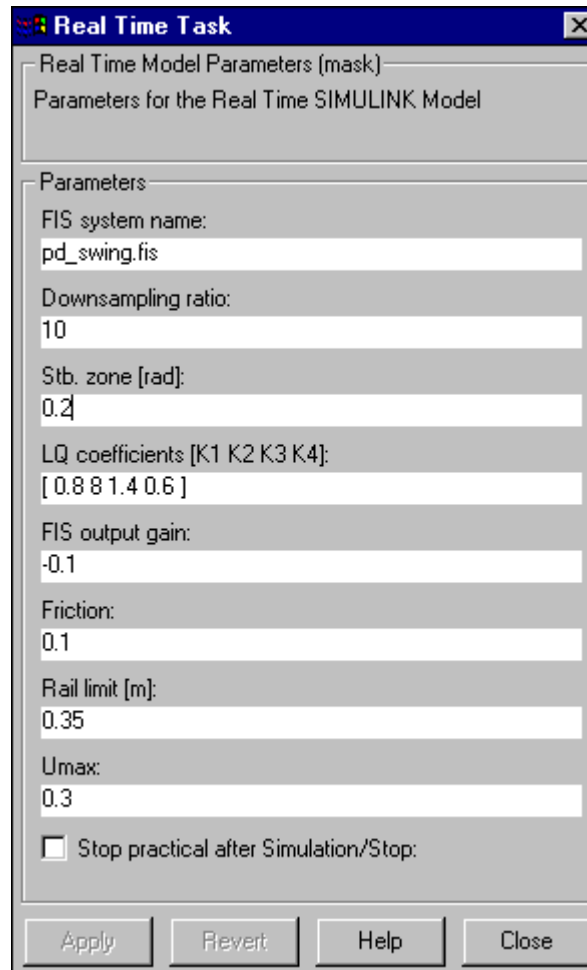


Figure 110: The *Real Time Task* mask

Looking at the list of parameters we make sure that RTK apart from the fuzzy swing-up controller is equipped with additional controllers required to achieve a soft landing in the stabilisation zone.

We can relate back to the rule-based control parameters (Figure 36) and discuss which part of the old rule-based RTK remains and what parameter values are used for the current fuzzy – rule-based controller.

The fuzzy controller replaces the old swinging, landing and centring mechanisms. Therefore the old rule-based parameters: k_{coeff} , R_{coeff} , α , β , γ do not exist any longer. Only two old rule-based mechanisms remain i.e. the algorithm of linear stabilisation (using parameters: $K1$, $K2$, $K3$, $K4$ available for $Stb.zone$) and the algorithm preventing crossing the rail limits (using parameter $Rail\ limit$). Also two old parameters: $Friction$ and U_{max} play the same role, as before.



Notice the differences in parameter values between the old and new parameter sets. Compare the values in Figure 36 with the values in Figure 110.

The results of the experiment are shown in the following figures: the cart position in Figure 111, the pendulum angle in Figure 112, the pendulum angular velocity in Figure 113, the cart velocity in Figure 114 and the control in Figure 115. Additionally, the states related to the cart are depicted pair-wise in Figure 116 and the states related to the pendulum are depicted pair-wise in Figure 117.

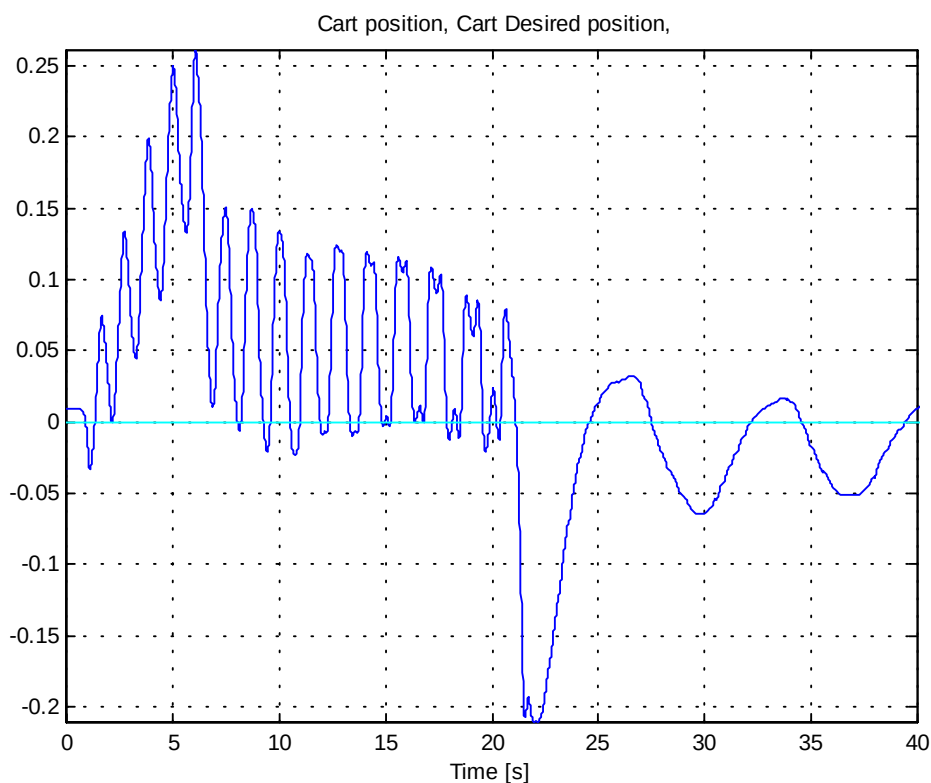


Figure 111: Time diagram of the cart position

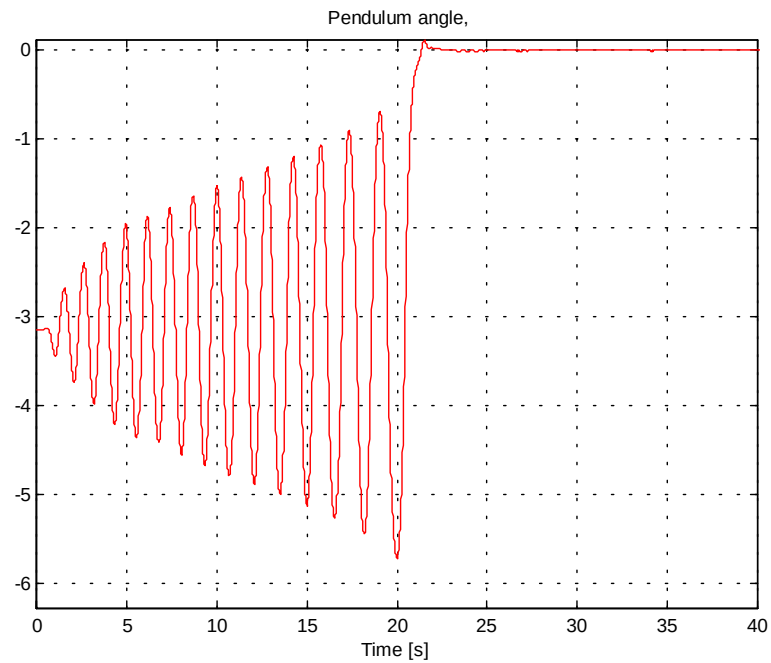


Figure 112: Time diagram of the pendulum angle

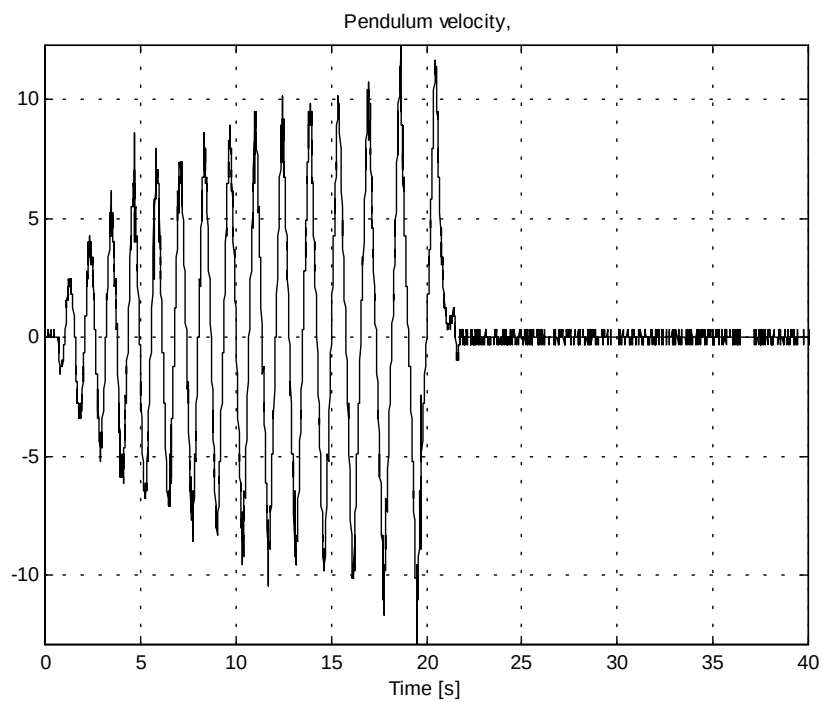


Figure 113: Time diagram of the pendulum angular velocity

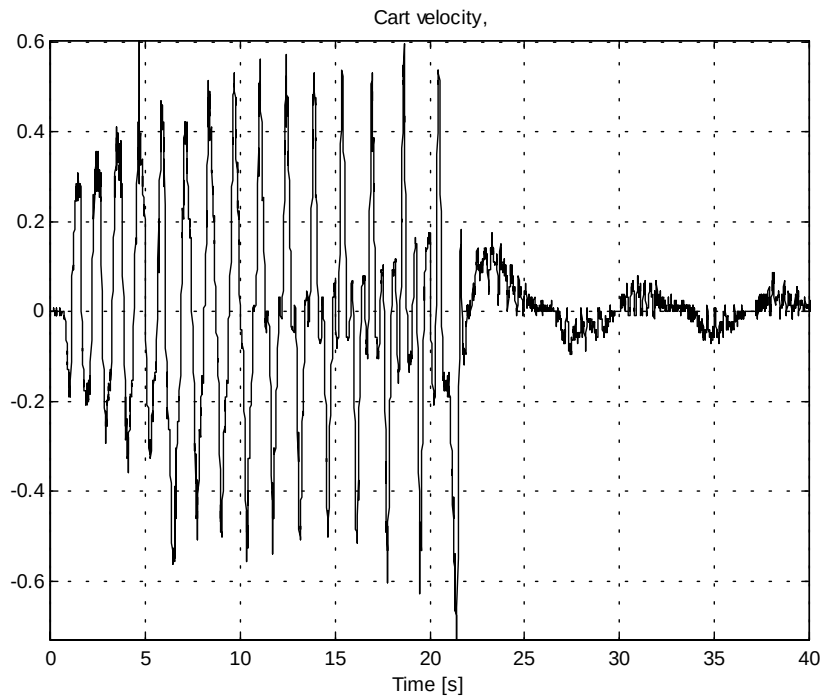


Figure 114: Time diagrams of the cart velocity

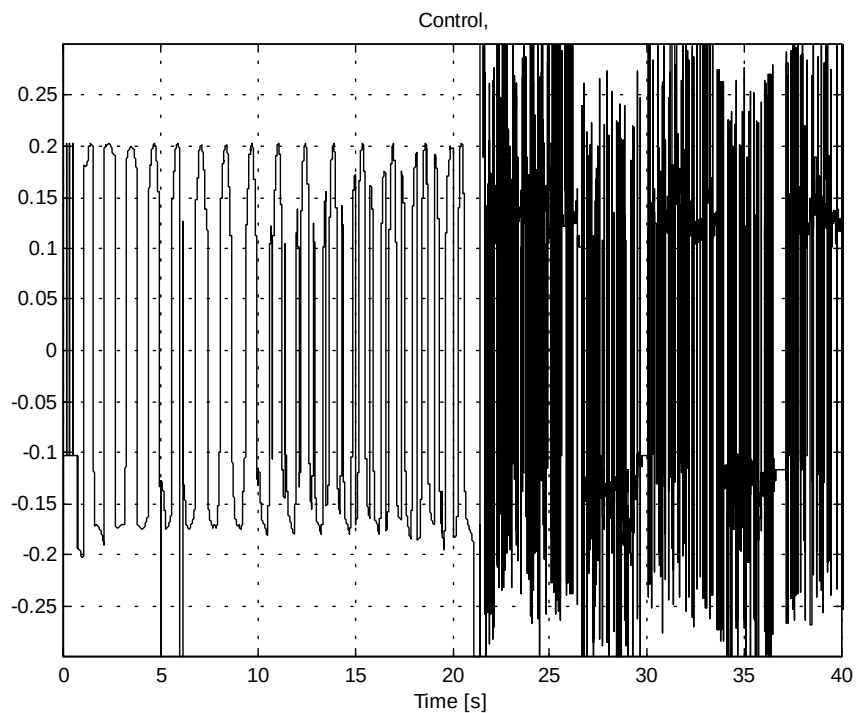


Figure 115: Time diagram of control

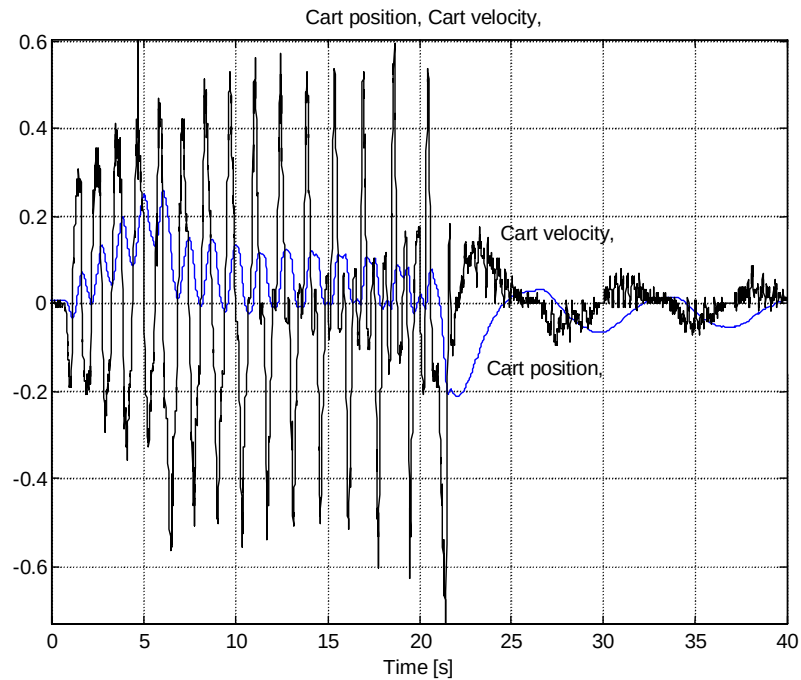


Figure 116: Time diagrams related to the cart

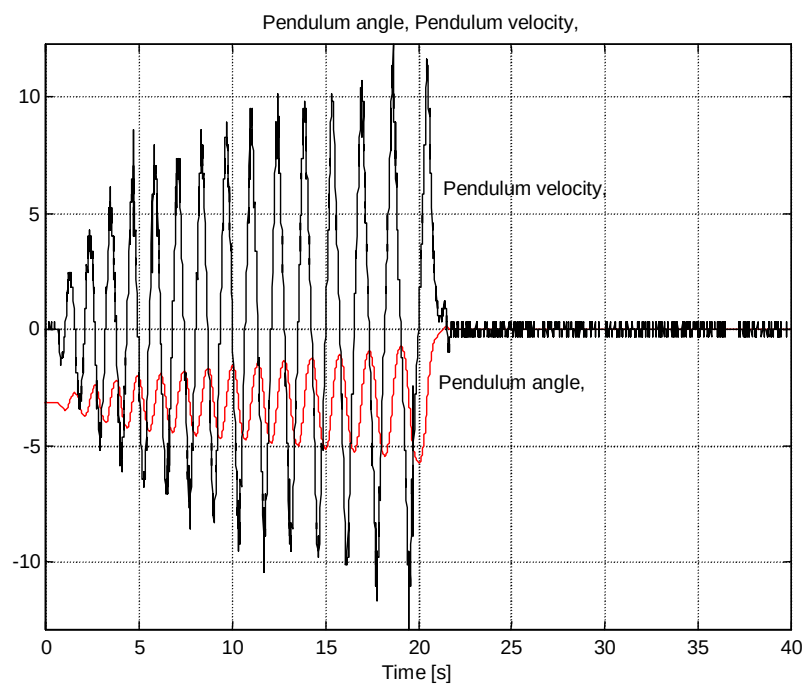


Figure 117: Time diagrams related to the pendulum



3.4 HOW TO BUILD AND RUN YOUR OWN FUZZY CONTROLLER

Follow the steps for simulation:

1. Define your own fuzzy controller. Remember that the fuzzy controller structure is specific and fixed. The structure of RTK impose restrictions. In principle you have two ready structures: *swing-up* RTK, and *crane* or *stabilising* RTK. The *Alpha offset* parameter of the *Real Time Task* block make it possible to distinguish the *crane* RTK from the *stabilising* RTK. In the case of the *stabilising* RTK the *Alpha offset* parameter is set to 0. In the case of the *crane* RTK this parameter is set to π .

If you decide to build a general fuzzy controller (excluding all rule-based algorithms) then the *swing-up* RTK structure must be used.

2. Build a reference model of the fuzzy controller using *FIS Editor* from MATLAB Fuzzy Toolbox.
3. Save your xxxx.fis file to the directory containing other *.fis files i.e. *pd_crane.fis*, etc.
4. Build a Simulink controller. For example modify the *pd_mswing.mdl* model (see Figure 102)

5. Write

```
>>fismat=readfis('xxxx')
```

in the MATLAB Command Window. The “*fismat*” variable is created and ‘xxxx’ *Fismatrix* is loaded under this name to the MATLAB Workspace.

6. Start simulation.

And for Real-time :

7. To perform the practical you must refer to the control problem definition. For example if your controller performs the stabilisation task then refer to the *Real Time Task* mask given in Figure 86. Change the name in the *FIS system name* window: put xxxx.fis instead of *pd_stab.fis*.
8. Start practical



4 Installation Instructions

Be sure that Windows, MATLAB and the basic Digital-Pendulum software are installed and running properly before attempting to install the software of this manual.

Insert the installation disk into the disk drive. See the Software Installation Manual - 33-000M5 for details.



CHAPTER 4DIGITAL PENDULUM SYSTEM

Rule-Based And Fuzzy Controllers

Installation Instructions

Notes



5 Literature

- [1.] MATLAB® v.5.x, Simulink v. 2.x Reference manual. The MathWorks Inc. 1997.
- [2.] MATLAB® v.5.x. Fuzzy Logic Toolbox. User's Guide. The MathWorks Inc. 1997.
- [3.] J. Yen, R. Langari, L. A. Zadech. Industrial Applications Of Fuzzy Logic and Intelligent Systems. IEEE Press, NY 1995.
- [4.] A. Isidori, Nonlinear control systems. Springer-Verlag, NY 1989.
- [5.] E. H. Mamdani. Applications of fuzzy algorithms for control of simple dynamic plant. IEEE Proceedings, 1974.
- [6.] H. J. Zimmerman, Fuzzy set theory and its applications, Kluwer Academic Publishers 1991.



**PENDULUM-CART SYSTEM
RULE-BASED AND FUZZY CONTROLLERS**

**CHAPTER 5
Literature**

Notes